Modelo REGEMAC – Planta de segregación RCD (Informe 2, Grupo 33)

Versión: v1.2 (14-oct-2026) · Módulos: **1 Llegadas** · **2 Caso base** · **2b Caso base optimizado (CBO)** · **3 Plantas de triaje** (Nivel 1 de REGEMAC, FT620C+PS123, KS514) · **3b Costos de operación** · **3c Áridos (interruptor)** · **4 Evaluación económica (FCL, VAN, TIR, Monte Carlo)**

Qué cambió en la v1.2 (14-oct-2026, comentarios de Fernando)

Cambio	Por qué	Dónde
Personal = diferencia contra los N = 8 trabajadores actuales. En la planta, 2 de ellos hacen la transferencia del material (incluido el cargador) y el resto segrega en la cabina. Solo se paga (o se ahorra) la diferencia. Sin supervisor ni operador adicional	Rodrigo piensa reasignar a los mismos trabajadores	`personal_extra`, `PLANTAS[…]['operarios_apoyo']`
2 turnos con sus costos propios: recargo del 2° turno (10%), electricidad en horario punta (18-22 h, abr-sep) a $320/kWh, mantención que crece en parte con las horas	Comparar 1 y 2 turnos sin suponer que todo se duplica	`COSTOS`, §8.6b
Valor del m³ de pozo = $4.200 (lo que REGEMAC cobra a externos por m³ suelto)	Las otras tarifas mezclan otros servicios de REGEMAC	`POZO['valor_m3']`
¿Conviene la chipeadora? Compra vs arriendo Raico vs seguir con RECUPAC, por volumen de madera	Pedido de Fernando	§6.2c
El flete de la astilla se calcula desde el camión de 100 m³ de astilla (4,5 USD/m³ × 100 m³ ÷ 22 t)	Aclarar volúmenes: madera suelta 0,34 t/m³, astilla 0,22 t/m³	`MADERA`

Qué cambió en la v1.1 (14-oct-2026)

Cambio	Por qué	Dónde
Nueva **Planta Nivel 1** (edificio de REGEMAC, planos 2024: cinta 1,2 m, 8 puestos, imán sobrebanda, sin harnero)	Minuta del 13-oct: caso nivel 1; planos que envió Rodrigo	`PLANTAS['Nivel1']`
Captura por fracción granulométrica (finos / medios / gruesos) según el equipo de cada planta	KS514 solo pica los gruesos (la madera de 20-70 mm se va a RDF); FT620C no tiene imán bajo malla	`GRANULOMETRIA`, `captura_fraccion`
Equipo de alimentación: cargador propio + operador (o arriendo)	Rodrigo: no hay máquina disponible; Kiverco lo excluye	`ALIMENTACION`
Áridos de hormigón separado como interruptor `ARIDOS['activo']` (True/False), con chancado por campañas arrendadas o compra	Propuesta de Emilio; Rodrigo quiere vender y reutilizar	`ARIDOS`, §6.10
Valor del pozo con vida útil de 8 años: los m³ ahorrados valen cuando el pozo se llena (2035)	Rodrigo 13-oct	`POZO`, §8
Gráficos del beneficio del fierro y de la madera (anual y en el VAN)	Pedido de Fernando	§6.9 y §8.4b
Costos nuevos: empalme 565 UF, control de polvo, repuestos, internación, losa de astilla, consulta de pertinencia SEIA (y escenario con DIA), año 1 parcial (6 meses)	Análisis de costos diferenciadores (14-oct)	`EXTRA_PLANTA`, `FIN`
Datos actualizados: sueldo $1,1 MM costo empresa, diésel $1.270/L neto, electricidad $250/kWh con potencia, UF/USD/GBP del 14-oct, rf real 2,74% (BCU-10), Kd real 9,2% (deuda de REGEMAC), valor residual por planta	Rodrigo 13-oct e investigación web (registro v3, hoja Investigacion_web)	§1

Todas las fuentes y su **grado de confianza** están en `Registro_modelo_REGEMAC_v3.xlsx` (hojas Parametros e Investigacion_web).

0. Cómo leer este modelo

Pregunta que responde: ¿le conviene a REGEMAC instalar una planta de triaje para el RCD que recibe en su pozo, comparado con seguir como hoy pero haciendo mejoras baratas?

Idea central. Simulamos, hora a hora y durante 10 años, los camiones que llegan al pozo y el destino de cada tonelada en cada escenario. Todos los escenarios usan **los mismos camiones, la misma composición y los mismos precios** (números aleatorios comunes). Así, las diferencias entre escenarios se deben solo a la decisión y no al azar.

Escalera de escenarios

Escenario	Qué hace	Origen de la idea
Caso base	Hoy: 8 trabajadores sacan material a mano en la fosa; la madera se paga a RECUPAC; el resto va al vertedero	Datos REGEMAC 2024-26
CBO (caso base optimizado)	Caso base + **electroimán colgado de la retro** + **chipeadora** (la madera se vende como astilla)	Nicolás (FCL_CBO), adaptado a nuestras reglas
Planta Nivel 1	CBO + **línea de triaje manual** de REGEMAC: tolva, cinta de 1,2 m, imán sobrebanda y 8 operarios en una cabina, sin harnero	Planos REGEMAC 2024 (Rodrigo); minuta 13-oct
Planta FT620C+PS123 / KS514	CBO + **planta de triaje** que procesa parte de lo que llega (lo que no alcanza a procesar va al vertedero)	Cotizaciones Kiverco (Rodrigo)
+ Áridos (opcional)	Chancado del hormigón que llega separado, para vender o usar en el predio	Emilio; Rodrigo

Qué se compara (en este orden).

1. **Todos contra el caso base** (CBO, KS514, FT620C): la comparación justa contra lo que REGEMAC hace hoy.
2. **Plantas contra el CBO**: lo que agrega la planta por sobre lo que REGEMAC podría hacer igual con poca inversión. Así lo pide Rodrigo para la madera: la astilla de lo que hoy ya se recupera va al CBO, y la astilla adicional que segrega la planta va al proyecto.

Recorrido de una tonelada

```
camiones por hora (Poisson) → contenido (% de cada material, sorteado por corrida)
 ├─ Caso base / CBO: fosa → trabajadores (capacidad fija t/h) → [imán en retro] → vertedero
 └─ Planta: acopio de espera (FIFO) → línea (t/h = mín(máquina, operarios)) → recuperado | rechazo
            └─ vaciado al vertedero sin procesar (en horario laboral, controlado según el tamaño del acopio) → [imán en retro]
recuperado → venta (fierro, aluminio, cartón, plástico) | madera → RECUPAC (costo) o astilla (venta)
no permitido en vertedero de inertes (madera, metales, plástico, cartón, yeso) → se mide aparte
```

Resultados por escenario: t recuperadas · t NO permitidas que terminan en el vertedero · m³ de pozo ahorrados · ventas − costos de la madera · **OPEX adicional** · **resultado operativo** · CAPEX · payback simple. VAN y tasa se calculan en el módulo 4.

Principios

1. **Términos reales:** pesos de 2026 sin inflación (Rodrigo pidió evaluar en UF). Por eso no se usa el 3% anual de los Excel del equipo.
2. Cada número tiene fuente, tipo y **confianza**; los IDs (P01, N01, W25…) remiten al `Registro_modelo_REGEMAC_v3.xlsx`.
3. Primero van los parámetros generales (§1), después los de cada escenario (§2) y luego las funciones (§4). Para probar otra cosa, solo se editan §1-§2.
4. OPEX = **costos adicionales** a los que REGEMAC ya tiene hoy. La excepción es el **movimiento de material al pozo** (retro o cargador), que se cuenta en todos los escenarios, también en el caso base, porque cambia con la planta. Lo que importa es la diferencia entre escenarios.

Qué incorporamos de los modelos del equipo (OneDrive, 6-8 oct)

Idea	Origen	Dónde entra	Qué ajustamos
Imán en la retro en horas valle (13 y 17 h), recuperación r = 20%, el personal sigue segregando	Nicolás	`CBO['iman']`	Actúa sobre el fierro que dejaron los trabajadores (sin doble conteo) y con nuestra composición (1,5% de fierro, no 6%)
Imán a batería USD 15.000 + 20% de internación; mantención 5%; retro 5,5 L/h	Nicolás	`CBO['iman']`	—
Chipeadora GardenWood (1,2 t/h, 37 kW, $29,5 MM + IVA, mantención 10%)	Nicolás	`MADERA['chipeo']` `['compra']`	Se agrega **arriendo Raico por semana** (400 UF, 35 h, 700 t)
Astilla $29.000/t + RECUPAC evitado $17.000/t	Nicolás / Rodrigo	`MADERA`	Flete a Arauco Teno (Nicolás supuso 0) y fracción de madera apta (pintada/MDF no sirve)

Idea	Origen	Dónde entra	Qué ajustamos
Sueldos ($780 mil operario, $1 MM supervisor), electricidad $225/kWh, diésel $1.500/L	Pedro	`COSTOS`	Factor costo empresa (cargas sociales)
Supervisor extra y cargador que alimenta la planta (14 L/h)	Pedro	`PLANTAS`	—
Consumo eléctrico de referencia 1,45-4,52 kWh/t (Ferronato)	Pedro	Chequeo en §6.6	Usamos los kW de las cotizaciones
No incorporado: densidad 1,4 t/m³, fierro 6%, madera como ingreso, crecimiento de precios con inflación	Pedro / Nicolás	—	Ver `claude/contraste-onedrive-vs-modelo`

Pendiente de alinear con el equipo: entrada anual (Nicolás usa 129.049 t; aquí ~158 mil t en 2026 según los viajes), régimen tributario (14A 27% vs Pro Pyme 25%).

Estructura del notebook

§	Contenido
1	Parámetros generales (llegadas, composición, normativa, mercado, costos, madera/astilla, simulación)
2	Parámetros por escenario: caso base, CBO, plantas
3	Datos y calibración
4	Funciones del modelo
5	Llegadas: validación
6	Una corrida: caso base → CBO → plantas → comparación → incremental, CAPEX y payback → sensibilidades
7	Monte Carlo del incremental
8	**Evaluación económica**: tasa (WACC), flujo de caja, VAN, TIR, break-even del m³ de pozo, VAN en el Monte Carlo y tornado
9	Exportar

Los datos se leen desde el repositorio público de GitHub (no hay que subir archivos a Colab).

0.1 De dónde sale cada número

Cada fila es un parámetro que se puede editar en §1-§2 (o §8 para finanzas). Los comentarios del código repiten la fuente y el ID del registro.

Tipo de fuente: Dato REGEMAC · Calculado · Cotización · Norma · Fuente externa (web) · Equipo · SUPUESTO · PENDIENTE. **Confianza: alta** = dato de REGEMAC, cotización o dato oficial reciente; **media** = equipo comparable publicado, fuente secundaria seria o cálculo con datos; **baja** = estimación indirecta o supuesto del equipo.

Bloque	Parámetro	Valor	Fuente	Tipo	Confianza
Llegadas	Viajes 2026	~28.200/año	Reportes de envíos ene-ago 2026 anualizados	Dato REGEMAC + calculado	alta
Llegadas	Crecimiento	2,8%/año	CAGR de retiros de la app 2021-2025	Calculado	media
Llegadas	Peso por contenedor	9 m³: 3 t; 10 m³: 7 t	Rodrigo (9-oct)	Dato REGEMAC	media
Composición	Fierro / aluminio / madera / cartón / plástico / yeso	Triangulares, ej. fierro 0,43-1,5-3%	Registro, hoja Composicion	SUPUESTO con rango	baja-media
Composición	Granulometría de cada material (finos / medios / gruesos)	ej. madera 5/20/75%	Nuestro (registro, hoja Captura_por_fraccion)	SUPUESTO	baja
Volumen	Densidad por material	Fierro 0,42, madera 0,34… t/m³	NCh3562:2019 Anexo D	Norma	alta

Bloque	Parámetro	Valor	Fuente	Tipo	Confianza
Mercado	Precio del fierro	$195/kg	Ventas a Ecociclo 2025-26. Mercado oct-2026: 160-170 $/kg (W53)	Dato REGEMAC	alta (riesgo a la baja)
Mercado	Riesgo de comprador único	10%/año, −25% por 6 meses	Nuestro	SUPUESTO	baja
Mercado	RECUPAC	$15,13/kg neto	Rodrigo	Dato REGEMAC	alta
Madera	Astilla / flete / fracción apta	$30.000/t; 4,5 USD por m³ de astilla (camión de 100 m³ ≈ 22 t → ~$20.000/t, sin multiplicar por 2); 70%	Rodrigo y Pedro; densidad de astilla 0,22 t/m³ (W50)	SUPUESTO / calculado	baja / media / baja
CBO	Electroimán	USD 15.000, r = 20%	Nicolás	Equipo	baja
CBO	Chipeadora compra / arriendo	$29,5 MM / 400 UF por semana	Nicolás / Raico	Fuente externa / cotización	media / alta
Plantas	CAPEX Kiverco	KS514 GBP 883.000; FT620C+PS123 GBP 530.000	Cotizaciones SKC 12-09-2025 (vencidas)	Cotización	media
Plantas	CAPEX Nivel 1	Edificio 1.900 UF + máquina ~USD 107 mil + $10 MM	Pinilla 2024 + web (W25-W28)	Dato / SUPUESTO	media / baja
Plantas	Capacidad	KS514 30, FT620C 27, Nivel 1 24 t/h efectivas	Cotizaciones; Nivel 1: 8 operarios × 3 t/h (W30, Finlandia 2,9)	Cotización / SUPUESTO	media / baja
Plantas	Captura por fracción	ver §2.4	Configuración de equipos (cotizaciones, planos) + nuestro	SUPUESTO	baja
Plantas	Potencia	KS514 37, FT620C 46, Nivel 1 25 kW	Cotizaciones / nuestro	Cotización / SUPUESTO	alta / baja
Plantas	Personal	Por turno: 8 en la cabina + 2 de transferencia (cargador y patio). Se paga solo la diferencia con los 8 actuales (+2 con 1 turno). 2° turno con 10% de recargo	Rodrigo (reasigna a los mismos trabajadores)	Dato / SUPUESTO	media
Plantas	Acopio de espera	Tope 7.500 m³; objetivo 1.000 m³	Rodrigo; nuestro	Dato / SUPUESTO	alta / baja
Alimentación	Cargador propio	$100 MM, 12 L/h, operador $1,5 MM/mes	Web (W34, W35, W18)	Fuente externa	media-baja
Costos	Sueldo operario	$1,1 MM/mes costo empresa	Rodrigo 13-oct	Dato REGEMAC	alta
Costos	Electricidad	$250/kWh con cargos de potencia	Pliegos Enel 2026 (W08-W11)	Calculado	media-baja
Costos	Diésel	$1.270/L neto	CNE 14-oct-2026 (W13)	Fuente externa	alta
Costos	Empalme / polvo / repuestos / internación	565 UF / $15 MM / 2-3% / 2,5%	Rodrigo / web (W39, W41, W24)	Dato / SUPUESTO	alta / media / baja / baja
Costos	Mantención	Nivel 1 3%, KS514 4,5%, FT620C 5% del CAPEX	Web (W40)	SUPUESTO	baja-media
Pozo	Vida útil y valor del m³	Lleno en 2034; $4.200 por m³ suelto (tarifa a externos)	Rodrigo 13-oct	Dato REGEMAC	alta / media
Áridos	Hormigón, precio, chancado	10%; $4.500/t; USD 2.000/día	RETC; web (W42-W49)	Dato / calculado	media / media-baja / baja
Finanzas	rf, Kd, impuesto, residual, permisos	Ver §8.0	BCCh, Rodrigo, SII, web	Varios	—

1. Parámetros generales (comunes a todos los escenarios)

In [1]:
```
# ---------- 1.1 Simulación ----------
SIM = dict(
    anio_inicio  = 2027,   # año 1 del proyecto
    horizonte    = 10,     # años (T01)
    n_montecarlo = 200,
    semilla      = 42,
```

```
)
# ---------- 1.2 Llegadas de camiones ----------
LLEG = dict(
    crecimiento          = None,   # %/año; None = CAGR histórico de retiros 2021-2025 (~2,8%). Alternativa CChC: 0.035 (L09)
    t_contenedor_9       = 3.0,    # t por viaje, contenedor 9 m3 (Rodrigo, L06)
    t_contenedor_10      = 7.0,    # t por viaje, contenedor 10 m3 (Rodrigo, L07)
    dens_otros           = 0.525,  # t/m3 para contenedores de 13-23 m3 (L08)
    t_por_viaje_objetivo = None,   # None = pesos por contenedor; ej. 4.43 o 4.8 reescala todo
    cv_peso_viaje        = 0.20,   # variabilidad del peso de cada viaje
    factor_llenado       = 1.00,   # m3 reales / m3 nominales
    usar_feriados        = True,
    domingos_operan      = False,
)
# ---------- 1.3 Composición de lo que llega (% masa: mín, moda, máx) — hoja Composicion ----------
MAT = ['fierro', 'aluminio', 'madera', 'carton', 'plastico', 'yeso', 'inertes']
COMP = dict(
    contenido = {'fierro': (0.0043, 0.015, 0.03), 'aluminio': (0.0001, 0.0005, 0.002), 'madera': (0.006, 0.025, 0.06),
                 'carton': (0.0005, 0.005, 0.02), 'plastico': (0.001, 0.01, 0.03), 'yeso': (0.03, 0.054, 0.07)},   # inertes = resto
    muestrear = True,   # Monte Carlo: sortear contenido por corrida (False = usar la moda)
)
# ---------- 1.4 Normativa: ¿puede ir al vertedero de inertes? (NCh3562:2019, Anexo A, Tabla A.1; D.S. 37 MINVU) ----------
PERMITIDO_VERTEDERO = {'fierro': False, 'aluminio': False, 'madera': False, 'carton': False, 'plastico': False,
                       'yeso': False,   # 17 08 02: no inerte, valorización ND
                       'inertes': True}
# ---------- 1.5 Mercado: precios de venta y costos externos ($ de 2026 por kg) ----------
MERCADO = dict(
    precio          = {'fierro': 195.0, 'aluminio': 1600.0, 'carton': 90.0, 'plastico': 50.0},   # registro limpio v2 (P01)
    vol_mensual     = {'fierro': 0.03, 'aluminio': 0.076, 'carton': 0.10, 'plastico': 0.10},     # σ mensual del log-precio (2025-26)
    reversion       = 0.90,     # AR(1) mensual (1 = paseo aleatorio)
    costo_recupac   = 18.0 / 1.19,   # $/kg neto: Rodrigo (9-oct) "Recupac nos cobra $18 por kilo con IVA" → $15,13 sin IVA (el IVA se recupera)
    # Riesgo de depender de un solo comprador (Ecociclo compra fierro y aluminio, sin contrato, a precio spot — Rodrigo)
    riesgo_comprador = dict(usar=True, materiales=['fierro', 'aluminio'],
                            prob_anual=0.10,   # SUPUESTO: probabilidad por año de perder al comprador
                            descuento=0.25,    # SUPUESTO: caída del precio mientras se busca otro comprador
                            meses=6),          # SUPUESTO: duración del episodio
)
# ---------- 1.6 Densidad aparente por material (t/m3, residuo suelto) — NCh3562:2019 (D.S. 37 MINVU), Anexo D ----------
# Se usan para pasar t ↔ m3 de vertedero con la densidad PROPIA de cada material (2 t de fierro = 2/0,42 = 4,8 m3).
# La densidad de 'inertes' se calcula para que la mezcla calce con lo que REGEMAC realmente recibe (0,525 t/m3, §3.2):
# la norma da 1,24 t/m3 para inerte limpio y advierte un esponjamiento ≥30% en residuos mezclados.
DENS = {'fierro':   0.42,   # Tabla D.3 "Despuntes acero" 420 kg/m3 (D.1: fierro de construcción 920)
        'aluminio': 0.90,   # Tabla D.3 "Despuntes aluminio" 900 kg/m3 (valor de la tabla; parece alto, impacto mínimo)
        'madera':   0.34,   # Tabla D.3 "Despuntes madera" 340 kg/m3
        'carton':   0.05,   # Tabla D.1 n°16 "Cartón" 50 kg/m3
        'plastico': 0.21,   # Tabla D.3 "Embalajes (general)" 210 kg/m3
        'yeso':     0.33}   # Tabla D.3 "Paneles de yeso cartón" 330 kg/m3
# ---------- 1.7 Capacidad técnica máxima de extracción manual en la fosa (fracción del contenido que se puede sacar) ----------
CAPTURA_MAX_FOSA = 0.90
# ---------- 1.8 Costos unitarios (pesos de 2026, términos reales) ----------
COSTOS = dict(
    sueldo_operario_mes   = 1_100_000,   # COSTO EMPRESA: Rodrigo 13-oct "$1.100.000 costo empresa cada uno" [O01, alta]
    sueldo_supervisor_mes = 1_750_000,   # costo empresa; web 1,4-2,1 MM [X11, baja]
    sueldo_operador_mes   = 1_500_000,   # operador de cargador, costo empresa; web 1,2-1,8 MM [A04, baja-media]
    factor_costo_empresa  = 1.0,         # los sueldos ya son costo empresa (v1.0: $780 mil × 1,25)
    precio_kwh            = 250,         # $/kWh con cargos de potencia (BT-4.3 sin punta ≈235; pliegos Enel 2026) [O04, media-baja; sens. 210-330]
    precio_kwh_punta      = 320,         # $/kWh en horas punta (18-22 h, abr-sep): demanda en punta (BT-2/3 o con punta ≈300-360) [W11, media-baja] — solo 2° turno
    horas_punta = [18, 19, 20, 21], meses_punta = [4, 5, 6, 7, 8, 9],
    recargo_turno2        = 0.10,        # SUPUESTO: bono por turno de tarde/noche (la ley no exige recargo nocturno; es práctica común)
    mant_fraccion_variable = 0.5,        # SUPUESTO: la mitad de la mantención crece con las horas de operación (desgaste), la otra mitad es fija
    precio_diesel         = 1_270,       # $/L neto de IVA (CNE 14-oct-2026: 1.530 con IVA) [O05, alta; sens. 1.100-1.500]
```

```
    tc_usd               = 981,         # dólar observado 14-oct-2026 [M01, alta]
    tc_gbp               = 1_290,       # 1.278,87 al 29-sep; ~1.300 implícito al 14-oct [T05, media]
    uf                   = 41_157.43,   # 14-oct-2026 [T06, alta]
    internacion          = 0.20,        # imán importado: flete + internación sobre FOB (Nicolás); el IVA se recupera
    internacion_planta   = 0.025,       # equipos Kiverco: arancel 0% (Acuerdo Chile-Reino Unido) + agente y puerto ~2,5% [X04-X05, alta / baja]
)
# ---------- 1.8a Inversiones adicionales de las plantas (no incluidas en las cotizaciones) ----------
EXTRA_PLANTA = dict(
    empalme_uf      = 565,          # Rodrigo 13-oct: empalme 165 + conexión 100 + cámaras/luminarias 300 UF [X06, alta]
    polvo_clp       = 15_000_000,   # cañón nebulizador + aspersores (web W39: $10-20 MM) [X02, media]
    losa_astilla_uf = 600,          # Arauco exige acopio sobre losa: 400 m² × 1,5 UF/m² (W19) [X01, baja-media] — CBO y plantas
)
# ---------- 1.8b Movimiento de material con retro/cargador (todos los escenarios) ----------
MOVIMIENTO = dict(
    t_por_hora = 80,     # SUPUESTO: t/h que mueve una retro/cargador al pozo
    diesel_lh  = 14,     # L/h (Pedro: cargador 14 L/h)
)   # → costo por t movida = diesel_lh × precio_diesel ÷ t_por_hora (≈ $220/t con diésel a $1.270)
# ---------- 1.8c Pozo ----------
POZO = dict(
    factor_compactacion = 1.0,   # Rodrigo: "se compacta solo con la pasada del bulldozer y los camiones" → ~1
    modo       = 'vida_util',    # 'vida_util': los m3 ahorrados se cobran cuando el pozo se llena (el caso base dispone afuera desde ese año
                                 #   y el proyecto todavía tiene espacio) | 'anual': cada m3 ahorrado vale valor_m3 el mismo año (v1.0)
    anio_lleno = 2034,           # Rodrigo 13-oct: "al pozo le quedan 8 años de vida útil" → último año con espacio [P07, alta]
    valor_m3   = 4_200,          # $/m3 suelto: lo que REGEMAC cobra a externos por disponer (Rodrigo 13-oct) [P06, media]
                                 #   referencias: costo propio $1.740; afuera cobran $3.500-4.000 (conservador); gate fee web 4.000-8.000 (W55, baja)
)
# ---------- 1.10 Evaluación económica (módulo 4) — fuentes en la tabla del §8.0 ----------
FIN = dict(
    tasa_real_base = None,           # None = WACC real calculado en §8.1 (CAPM con datos de Damodaran); o fijar un valor, ej. 0.09
    tasas_rango    = [0.06, 0.07, 0.08, 0.09, 0.10, 0.11, 0.12],   # Rodrigo: "muestren el VAN para un rango de tasas" (reales)
    capm = dict(rf_real=0.0274,      # BCU/BTU-10 Banco Central, 9-oct-2026 [M03, alta] (v1.0: TIPS 2,91%)
                beta_u=0.82,         # Damodaran ene-2026, Environmental & Waste Services (beta desapalancada)
                d_e=0.2145,          # Damodaran ene-2026, D/E de la misma industria
                erp=0.0533,          # Damodaran ene-2026: prima madura 4,23% + riesgo país Chile 1,10%
                premio=0.02,         # SUPUESTO justificado: tecnología nueva y venta de material que hoy no se vende (Rodrigo)
                kd_real=0.092),      # deuda de REGEMAC 10,9-13,9% nominal (prom. 12,5%) → real con 3% de inflación [M04, media]
    impuesto = 0.27,                 # régimen 14A (Rodrigo 13-oct) [T02, alta]
    vida_util = {'maquinaria': (15, 5), 'obras': (10, 3)},   # años (normal, acelerada) — tabla de vida útil SII
    depreciacion_acelerada = False,
    valor_residual = {'maquinaria': 0.25, 'obras': 0.0},     # por defecto; cada planta define 'valor_residual_equipo' (KS514 15%, FT620C 30%, Nivel 1 15%) [X09, N11]
    dias_cobro = 30, dias_pago = 30, # SUPUESTO: capital de trabajo
    seguro = 0.01,                   # SUPUESTO: % del CAPEX por año
    permisos_planta = 20_000_000,    # SUPUESTO: $ en el año 0 (PAS 140, autorización sanitaria Seremi, OGUC), como gasto
    seia = dict(pertinencia_uf=75,   # consulta de pertinencia (siempre) [X07, baja]
                dia=False,           # True = escenario con Declaración de Impacto Ambiental (letra o.8, ≥30 t/día si se considera "tratamiento")
                dia_uf=1_250, dia_retraso_meses=12),   # web: 600-2.000 UF; tramitación promedio 401 días [X08, media-baja]
    rampa_anio1 = 0.85,              # SUPUESTO: año 1 la planta logra el 85% del margen de materiales
    valores_m3 = [0, 1_740, 3_750, 4_200, 8_000, 15_000],   # $/m3 de pozo para la sensibilidad (costo propio, afuera, tarifa REGEMAC, gate fee alto)
)
# ---------- 1.9 Madera: astilla y formas de chipear ----------
# Arauco (Planta Teno) solo compra astilla de 5-50 mm, ≤15% de humedad, pasada por imán y entregada en su planta (ficha técnica).
MADERA = dict(
    precio_astilla = 30_000,   # $/t: Rodrigo $29.000, Pedro ~$30.000 (unidad t, por confirmar con Arauco) [P04, baja]
    flete_usd_m3   = 4.5,      # USD por m3 estéreo (suelto) de ASTILLA: Pedro/Arauco 4-5 USD, Arauco retira en planta. Es el precio del servicio:
                               #   se supone que ya incluye la vuelta del camión (no se multiplica por 2) [P05, media]
    camion_m3      = 100,      # m3 de astilla por camión de Arauco (Pedro)
    densidad_astilla = 0.22,   # t/m3 suelto de astilla reciclada al 15-20% de humedad (COFORD 0,18-0,25; W50) [media]
                               #   ≠ madera suelta en el pozo 0,34 t/m3 (NCh3562): RECUPAC cobra por kg (la masa no cambia al chipear),
                               #   el flete por m3 de astilla, y el pozo por m3 de madera suelta
    frac_apta      = 0.70,     # SUPUESTO: madera limpia apta para astilla; el resto (pintada, MDF, impregnada) sigue a RECUPAC
```

```
    chipeo = {
        'compra':   dict(capex_clp=29_500_000 * 1.05, cap_th=1.2, factor_horas=1.5, kw=37, carga=0.75, mantencion=0.10),  # GardenWood GWTP60PH (Nicolás)
        # Raico (Ricardo, 9-oct): "no tenemos máquinas más pequeñas ... arriendo por 1 semana, 35 horas de triturado a 400 UF,
        # con una producción de 700 t/semana". La madera se junta hasta completar un lote (o hasta dias_max) y se arrienda por semanas.
        'arriendo': dict(uf_semana=400, horas_semana=35, t_semana=700, lote_min_t=700, dias_max=365),
    },
)
MADERA['flete_astilla'] = MADERA['flete_usd_m3'] * COSTOS['tc_usd'] / MADERA['densidad_astilla']   # $/t ≈ 20.000
# camión: 100 m3 × 0,22 t/m3 = 22 t de astilla; 4,5 USD × 100 m3 × $981 ≈ $441 mil por viaje
URL = 'https://raw.githubusercontent.com/fernando25132/Evalpro/main/data/'
F_ENVIOS     = URL + 'Reportes_Envios/envios-regemac-2026-completo-estandarizado.csv'
F_RETIROS    = URL + 'Info_App/retiros-mensuales-obra-regemac-estandarizado.csv'
F_MATERIALES = URL + 'Materiales_Reciclados/materiales-regemac-limpio-v2.csv'
# ---------- 1.11 Alimentación de la planta (Rodrigo: "no tenemos máquina disponible para alimentar la planta") ----------
ALIMENTACION = dict(
    cargador_propio = dict(capex_clp=100_000_000,   # cargador 2,5-3 m3: chino nuevo $66-105 MM, CAT 938M usado $106-196 MM (W34) [A02, media-baja]
                           diesel_lh=12,             # Liebherr L 550: 10,7 L/h (W35) [A03, alta]
                           mantencion=0.06, valor_residual=0.25),   # SUPUESTO [A05]
    arriendo = dict(clp_hora=45_000, diesel_lh=12),  # con operador, diésel aparte; retro 25-35 mil/h, cargador 40-60 mil/h (W32-W33) [A06, baja]
)
# ---------- 1.12 Granulometría: % de la masa de cada material en finos 0-20 / medios 20-70 / gruesos 70-300 mm ----------
# SUPUESTO (confianza baja): clavos y alambre caen en finos y medios; la madera de obra viene mayormente en piezas grandes.
GRANULOMETRIA = {'fierro': (0.15, 0.30, 0.55), 'aluminio': (0.10, 0.40, 0.50), 'madera': (0.05, 0.20, 0.75),
                 'carton': (0.05, 0.25, 0.70), 'plastico': (0.10, 0.30, 0.60), 'yeso': (0.40, 0.30, 0.30)}
# ---------- 1.13 Áridos: chancado del hormigón que llega separado (propuesta de Emilio) ----------
ARIDOS = dict(
    activo   = False,               # INTERRUPTOR: True = se chanca el hormigón separado en las plantas de aplica_a
    aplica_a = ['Planta Nivel 1', 'Planta FT620C + PS123', 'Planta KS514'],
    frac_hormigon  = 0.10,          # RETC 2024: 11.990 t de hormigón separado de 118.457 t [R01, media]. El mixto no cumple como base (CBR 41, CDH UC 2024)
    rendimiento    = 0.90,          # t de árido por t de hormigón [R02, baja]
    frac_venta     = 0.50,          # el resto se usa en el predio (caminos, cobertura) [R03, baja]
    precio_venta_t = 4_500,         # $/t neto ex planta: base natural 12,9-14 mil/m3+IVA puesta en obra; reciclado 0,74-0,89× (W42-W45) [R04, media-baja]
    valor_uso_interno_t = 0,        # PENDIENTE: lo que REGEMAC deja de comprar para caminos/cobertura [R05]
    modo = 'arriendo',              # 'arriendo' (campañas) | 'compra'
    arriendo = dict(usd_dia=2_000,  # RM 70GO! Texas 2025: USD 1.799-1.999/día de 6 h (W46); en Chile solo a cotización [R06, baja]
                    t_dia=480,      # ~80 t/h reales × 6 h (W48) [R07, media]
                    usd_campana=1_000, lote_min_t=3_000),   # cargo por arriendo; se junta hormigón hasta el lote
    compra = dict(capex_usd=320_000, # RM 70GO! usado 2022-23 USD 350-460 mil; 2017-18 ~131 mil (W47) [R09, media-baja]
                  opex_t=3_500,     # $/t con diésel, operador y desgaste (W49: 4-8 USD/t) [media-baja]
                  mantencion=0.05),
    diesel_l_t = 0.25,              # L/t (W48) [R08, media]
    personal_t = 300,               # $/t: operador y cargador durante la campaña (SUPUESTO)
)
```

2. Parámetros por escenario

2.1 Caso base (sin proyecto): REGEMAC sigue operando como hoy

Los trabajadores sacan material del frente de descarga de la fosa. Su **capacidad de extracción** (t/h por trabajador y material) se calibra para reproducir lo recuperado en 2026 (kg/viaje del registro). Como la capacidad es fija, lo recuperado **no crece** con las llegadas.

In [2]:
```
BASE = dict(
    nombre = 'Caso base',
    trabajadores = 8,                                # segregación manual en la fosa (Rodrigo)
    horas = [8, 9, 10, 11, 12, 14, 15, 16, 17],      # horario de trabajo (colación 13 h)
    dias = [0, 1, 2, 3, 4],                          # lun-vie
    kg_viaje_2026 = {'fierro': 21.0, 'aluminio': 0.19, 'madera': 5.0, 'carton': 0.4, 'plastico': 0.0},   # calibración (B01-B05)
```

```
    cv_mensual = {'fierro': 0.20, 'aluminio': 0.30, 'madera': 0.45, 'carton': 0.35, 'plastico': 0.35},
    tendencia_anual = {'madera': 0.0, 'carton': 0.0},   # 0 = se mantiene 2026; -0.2 = sigue bajando 20%/año
    vende = {'fierro': True, 'aluminio': True, 'carton': True, 'plastico': False},
    madera = dict(destino='recupac'),                   # 'recupac' (costo), 'fosa' (no se saca) o 'astilla' (ver CBO)
    frac_no_permitido_apartado = 0.10,                  # del no permitido NO recuperado, lo que se aparta en acopio
    acopio_np = dict(usar_limite_tiempo=True, dias_max=7, usar_limite_stock=True, stock_max_t=50),
)
```

2.2 Caso base optimizado (CBO): lo que REGEMAC podría hacer sin planta

Idea de Nicolás (`FCL_CBO_electroiman_chipeadoraa.xlsx`), adaptada para que sea comparable con el resto del modelo:

- **Electroimán en la retro.** La retro ya existe; se le cuelga un imán (a batería o hidráulico) que trabaja en las horas en que los trabajadores no están en el frente (13 h colación y 17 h, según el gráfico de llegadas). Recupera una fracción **r** del fierro que llega en esas horas **y que los trabajadores no sacaron**. Supuesto: r = 20% (Nicolás; sensibilidad 12-50% en §6.2).
- **Chipeadora.** La madera recuperada apta se chipea y se vende como astilla en vez de pagar a RECUPAC. Hay dos formas de hacerlo (`modo_chipeo`):
  - `compra` : GardenWood, chipea a diario.
  - `arriendo` : Raico, que no tiene máquinas chicas y arrienda por semana (400 UF, 35 h, ~700 t). La madera se junta hasta completar el lote o hasta un máximo de un año.
- **Nuestras decisiones se mantienen:** mismas llegadas y composición (1,5% de fierro, no 6%), los mismos 8 trabajadores con su capacidad calibrada, precios en $ reales, la misma normativa y las mismas reglas de acopio que el caso base.
- `trabajadores_extra` permite probar la alternativa (a) de la minuta del 22-sep: más personal en la fosa.

In [3]:
```
CBO = dict(
    nombre = 'CBO',
    trabajadores_extra = 0,                 # Nicolás: los actuales operan imán y chipeadora; minuta 22-sep: opción de más personal
    iman = dict(
        usar = True,
        horas = [13, 17],                   # horas valle (Nicolás: 14,3% del fierro llega en esas horas)
        recuperacion = 0.20,                # r: fracción del fierro que llega en esas horas y queda en la fosa (Nicolás, sens. 12-50%)
        capex_usd = 15_000,                 # imán a batería tipo Hydraram EMG-80 (Nicolás, sin precio público: cotizar)
        mantencion = 0.05,                  # % del CAPEX por año (Nicolás)
        diesel_lh = 5.5,                    # retro CAT 420 (Nicolás)
        frac_diesel_incremental = 1.0,      # 1 = todo el diésel es adicional (conservador)
    ),
    madera = dict(destino='astilla', modo_chipeo='compra'),   # modo: 'compra' | 'arriendo' (Raico)
)
```

2.3 Plantas de triaje: Nivel 1 de REGEMAC y cotizaciones SKC/Kiverco (12-sep-2025)

	Nivel 1 (REGEMAC)	FT620C + PS123	KS514
Separación previa	Ninguna: todo pasa por la cinta	Trómel: solo el sobretamaño va a la cabina	Harnero 70/20 mm: solo los gruesos van a la cabina
Imán	Sobrebanda sobre todo el flujo	Solo en el sobretamaño	En medios y después de la cabina
Qué limita la capacidad	Los 8 operarios (~3 t/h cada uno)	La máquina (~35 t/h nominal)	La máquina (~45 t/h a 1 t/m³)
CAPEX	Edificio 1.900 UF + máquina fabricada en Chile	GBP 530.000 + obras 12%	GBP 883.000 + obras 12%
Valor residual	15% (fijo)	30% (móvil, hay mercado de usados)	15% (estática)

Flujo por hora: llegadas → **acopio de espera** → línea en horario de operación → imán / harnero o trómel / picking → recuperado, rechazo inerte (vertedero permitido) y rechazo no permitido (acopio NP).

¿Qué es el acopio de espera? Es el montón de RCD que los camiones descargan frente a la planta y que todavía no se procesa. Llegan ~600 t por día hábil y la planta procesa ~200-240 t por turno, así que el montón crecería sin parar. Para que no crezca, los trabajadores **vacían parte del montón al vertedero sin procesar**, y solo lo hacen **en horario laboral**. El layout de REGEMAC (16-08-2024) reserva 2.400 m² de acopio primario: con ~3 m de altura caben ~7.200 m³, casi el tope de 7.500 m³ que dio Rodrigo.

Regla de vaciado (`modo='control'`). Cada hora de trabajo:

`vaciado (t/h) = tasa_base + kp × (tamaño del acopio − objetivo)`, acotado entre 0 y la capacidad de la retro o cargador.

- `tasa_base` es el vaciado promedio que cuadra el balance: (llegadas − capacidad de proceso) ÷ horas de trabajo. Con `kp = 0` queda una tasa fija.
- El término `kp` hace que los trabajadores vacíen más cuando el montón está grande y menos cuando está chico. `retardo_h` modela que reaccionan con atraso.
- Si el acopio llega al tope físico (`tope_m3`), los camiones que llegan descargan directo en el vertedero.

Capacidad horaria = mín(capacidad nominal × factor de densidad × disponibilidad ; operarios de picking × t/h revisadas por operario ÷ fracción que llega al picking).

Captura por fracción (nuevo en v1.1). Cada material se reparte en finos (0-20 mm), medios (20-70) y gruesos (70-300) según `GRANULOMETRIA`, y cada planta captura distinto en cada tamaño según sus equipos (`captura_fraccion`). La captura efectiva es la suma ponderada (tabla del §2.4).

Planta = CBO + planta (`mejoras_cbo = True`): el imán de la retro sigue trabajando sobre lo que la planta no alcanza a procesar, y la madera recuperada tiene el mismo destino que en el CBO.

Alimentación (nuevo en v1.1). REGEMAC no tiene una máquina libre para cargar la tolva (la retro y el bulldozer ya trabajan 4-6 h/día) y Kiverco excluye el "equipo de carga". Por defecto se compra un cargador frontal con operador (`alimentacion='cargador_propio'`); `'arriendo'` lo arrienda por hora con operador; `'por_tonelada'` es la forma de la v1.0.

Personal (v1.2). Hoy hay N = 8 trabajadores en la fosa, que además depositan material en el pozo. Rodrigo piensa reasignarlos a la planta: **2 hacen la transferencia** del material a la tolva (uno opera el cargador) y **el resto segrega en la cabina**. Solo cuenta la **diferencia** con los 8 de hoy: si la planta necesita 10 por turno se pagan 2 sueldos más; si necesita 7, se ahorra 1. El 2° turno lleva 10% de recargo.

Costos de operación adicionales: personal (diferencia con los 8 actuales), electricidad (kW × horas, más cara en punta para el 2° turno), diésel del cargador, mantención (% del CAPEX, la mitad crece con las horas) y movimiento al pozo.

In [4]:
```
# Horarios de turno (colación excluida)
T1 = [8, 9, 10, 11, 12, 14, 15, 16]          # 8 h efectivas
T2 = [17, 18, 19, 21, 22, 23]                # 6 h efectivas (supuesto)
PLANTAS = {
 'Nivel1': dict(
    nombre = 'Planta Nivel 1',
    # CAPEX por ítem: (monto, moneda, categoría tributaria). Edificio: informe Pinilla Rev.0 (27-03-2024) 1.615,25 UF sin IVA + ~18% no incluido
    capex_items = {'Edificio y estructura (Pinilla 2024)':           (1_900, 'uf', 'obras'),        # [N01, media]
                   'Tolva, grizzly y alimentador vibratorio':         (30_000, 'usd', 'maquinaria'), # web W27 [N02, baja]
                   'Cinta de alimentación inclinada (~12 m)':         (22_000, 'usd', 'maquinaria'), # web W26 [N03, baja]
                   'Cinta de triaje 1,2 m (~15 m)':                   (35_000, 'usd', 'maquinaria'), # plano L01 "cinta 120"; web W25 [N04, baja]
                   'Imán sobrebanda autolimpiante':                   (20_000, 'usd', 'maquinaria'), # web W28 [N05, baja]
                   'Motores, tablero, montaje':                       (10_000_000, 'clp', 'maquinaria')},  # [N06, baja]
    cap_nominal_th = 60,                     # tope físico de la cinta (no manda) [N09, baja]
    factor_densidad = 1.0, disponibilidad = 0.92,
    fraccion_a_picking = 1.0,                # sin harnero: todo pasa por la cabina [N08]
    operarios_picking = 8,                   # plano L01: 8 puestos (4 por lado); PFD: 6 [N07]
    operarios_apoyo = 2,                     # transferencia del material a la planta (uno opera el cargador) y patio: 2 de los 8 actuales
    trabajadores_fosa = 0,
    tasa_revision_th = 3.0,                  # t/h por operario sobre RCD sin precribar: Finlandia 2,9; web 10-25 t/h con 6-8 (W30); PFD 6,5 [F07, baja]
    captura_fraccion = {'fierro': (0.6, 0.8, 0.9), 'aluminio': (0, 0.2, 0.5), 'madera': (0, 0.3, 0.65),   # (finos, medios, gruesos)
                        'carton': (0, 0.3, 0.6), 'plastico': (0, 0.3, 0.6), 'yeso': (0, 0.3, 0.4)},     # imán sobre todo el flujo; picking con finos que tapan
    horas = T1, dias = [0, 1, 2, 3, 4], turnos = 1,
    vende = {'fierro': True, 'aluminio': True, 'carton': True, 'plastico': True},
    acopio_espera = dict(modo='control', objetivo_m3=1_000, kp=0.05, tasa_base='auto', retardo_h=1, horas_vaciado=None,
                         cap_vaciado_th=80, tope_m3=7_500, horas_max=120, usar_limite_tiempo=True),
    acopio_np = dict(usar_limite_tiempo=True, dias_max=14, usar_limite_stock=True, stock_max_t=150),
    mejoras_cbo = True,
    kw = 25, energia = 'red', generador_lh = None, carga_electrica = 0.80,   # alimentador 7,5 + cintas 5,5+4 + imán 5,5 + A/C 3 [N10, baja]
    alimentacion = 'cargador_propio', factor_alimentacion = 1.0, cargador_lh = 12,
    supervisores_extra = 0,                  # SUPUESTO v1.2: supervisa el jefe actual (sensibilidad: 1)
    mantencion = 0.03,                       # cintas 2-3%/año (W40) [O06]
```

```
    repuestos = 0.02,                       # % del equipo en el año 0 [X03]
    valor_residual_equipo = 0.15,           # fijo, fabricación local [N11, baja]
    inicio_meses = 6,                       # meses desde ene-2027 hasta operar [N12, baja]
 ),
 'FT620C+PS123': dict(
    nombre = 'Planta FT620C + PS123',
    capex_gbp = 530_000,                    # cotización opción base (trómel + PS123 4 bahías + genset)
    cap_nominal_th = 35,                    # SUPUESTO: no viene en la cotización; resumen técnico FT620: 35-60 t/h en RCD pesado → pedir a SKC
    factor_densidad = 0.85, disponibilidad = 0.90,
    fraccion_a_picking = 0.30,              # sobretamaño del trómel ≈ gruesos (granulometría de la cotización KS514: 35/35/30)
    operarios_picking = 8, operarios_apoyo = 2, trabajadores_fosa = 0,
    tasa_revision_th = 4.0,
    captura_fraccion = {'fierro': (0, 0, 0.9), 'aluminio': (0, 0, 0.6), 'madera': (0, 0, 0.75),       # bajo malla sin imán ni picking
                        'carton': (0, 0, 0.7), 'plastico': (0, 0, 0.7), 'yeso': (0, 0, 0.5)},       # sobretamaño: imán + soplador + picking
    horas = T1, dias = [0, 1, 2, 3, 4], turnos = 1,
    vende = {'fierro': True, 'aluminio': True, 'carton': True, 'plastico': True},
    acopio_espera = dict(modo='control', objetivo_m3=1_000, kp=0.05, tasa_base='auto', retardo_h=1, horas_vaciado=None,
                         cap_vaciado_th=80, tope_m3=7_500, horas_max=120, usar_limite_tiempo=True),
    acopio_np = dict(usar_limite_tiempo=True, dias_max=14, usar_limite_stock=True, stock_max_t=150),
    mejoras_cbo = True,
    pct_obras = 0.12,                       # SUPUESTO: radier, grúa, tablero (no incluidos en la cotización)
    kw = 46,                                # ficha FT620: alimentador 2×1,1 + tambor 4×5,5 + PS123 21 kW (cotización)
    energia = 'red',                        # 'red' | 'generador' (trae generador 60 kVA: 9,5 L/h al 50% de carga)
    generador_lh = 9.5, carga_electrica = 0.80,
    alimentacion = 'cargador_propio', factor_alimentacion = 1.0, cargador_lh = 12,
    supervisores_extra = 0,                 # SUPUESTO v1.2: supervisa el jefe actual (sensibilidad: 1)
    mantencion = 0.05,                      # trómel: mallas, rodamientos, orugas 4-6% (W40) [O06]
    repuestos = 0.03,                       # [X03]
    valor_residual_equipo = 0.30,           # trómel móvil con mercado de usados: TTS620 2018 ~USD 200 mil (W36) [X09, media]
    inicio_meses = 6,                       # 12-16 semanas fabricación + 6-8 transporte + 5 días [X10]
 ),
 'KS514': dict(
    nombre = 'Planta KS514',
    capex_gbp = 883_000,                    # cotización (entrega en faena RM; sin grúas ni obra civil)
    cap_nominal_th = 45,                    # cotización (estimada para 1.000 kg/m3)
    factor_densidad = 0.75,                 # SUPUESTO: menor t/h con RCD de ~0,5 t/m3 (cinta limita por volumen)
    disponibilidad = 0.90,
    fraccion_a_picking = 0.30,              # cotización: 70-300 mm = 30%
    operarios_picking = 8, operarios_apoyo = 2, trabajadores_fosa = 0,
    tasa_revision_th = 4.0,
    captura_fraccion = {'fierro': (0, 0.85, 0.95), 'aluminio': (0, 0, 0.6), 'madera': (0, 0, 0.75),  # medios: imán + aire (livianos → RDF)
                        'carton': (0, 0, 0.7), 'plastico': (0, 0, 0.7), 'yeso': (0, 0, 0.5)},      # gruesos: cabina + imán después
    horas = T1, dias = [0, 1, 2, 3, 4], turnos = 1,
    vende = {'fierro': True, 'aluminio': True, 'carton': True, 'plastico': True},
    acopio_espera = dict(modo='control', objetivo_m3=1_000, kp=0.05, tasa_base='auto', retardo_h=1, horas_vaciado=None,
                         cap_vaciado_th=80, tope_m3=7_500, horas_max=120, usar_limite_tiempo=True),
    acopio_np = dict(usar_limite_tiempo=True, dias_max=14, usar_limite_stock=True, stock_max_t=150),
    mejoras_cbo = True,
    pct_obras = 0.12,
    kw = 37,                                # cotización: 32,9 kW de motores + A/C cabina ~4 kW
    energia = 'red', generador_lh = 9.5, carga_electrica = 0.80,
    alimentacion = 'cargador_propio', factor_alimentacion = 1.0, cargador_lh = 12,
    supervisores_extra = 0,                 # SUPUESTO v1.2: supervisa el jefe actual (sensibilidad: 1)
    mantencion = 0.045,                     # [O06]
    repuestos = 0.03,
    valor_residual_equipo = 0.15,           # estática, sin mercado de usados (W37) [X09, baja]
    inicio_meses = 6,
 ),
}
def captura_de(sc):
```

```
    # Captura efectiva por material = Σ fracción del material en cada tamaño × captura del equipo en ese tamaño
    if 'captura_fraccion' not in sc: return sc['captura']
    return {m: float(np.dot(GRANULOMETRIA[m], sc['captura_fraccion'][m])) for m in sc['captura_fraccion']}
import numpy as np, pandas as pd
display(pd.DataFrame({sc['nombre']: captura_de(sc) for sc in PLANTAS.values()}).style.format('{:.0%}').set_caption('2.4 Captura efectiva por material (% del contenido procesado)')
```

2.4 Captura efectiva por material (% del contenido procesado)

	Planta Nivel 1	Planta FT620C + PS123	Planta KS514
fierro	82%	50%	78%
aluminio	33%	30%	30%
madera	55%	56%	56%
carton	49%	49%	49%
plastico	45%	42%	42%
yeso	21%	15%	15%

3. Datos y análisis histórico

In [5]:
```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
pd.set_option('display.float_format', lambda x: f'{x:,.2f}')
plt.rcParams.update({'figure.figsize': (11, 4), 'axes.grid': True, 'grid.alpha': .3})
try:
    import holidays
    FERIADOS = holidays.Chile(years=range(2021, 2045))
except ImportError:
    FERIADOS = {}; print('Paquete holidays no disponible: sin feriados (pip install holidays)')
env = pd.read_csv(F_ENVIOS, low_memory=False)
ret = pd.read_csv(F_RETIROS)
mat = pd.read_csv(F_MATERIALES)
print('envíos', env.shape, '| retiros app', ret.shape, '| materiales', mat.shape)
```

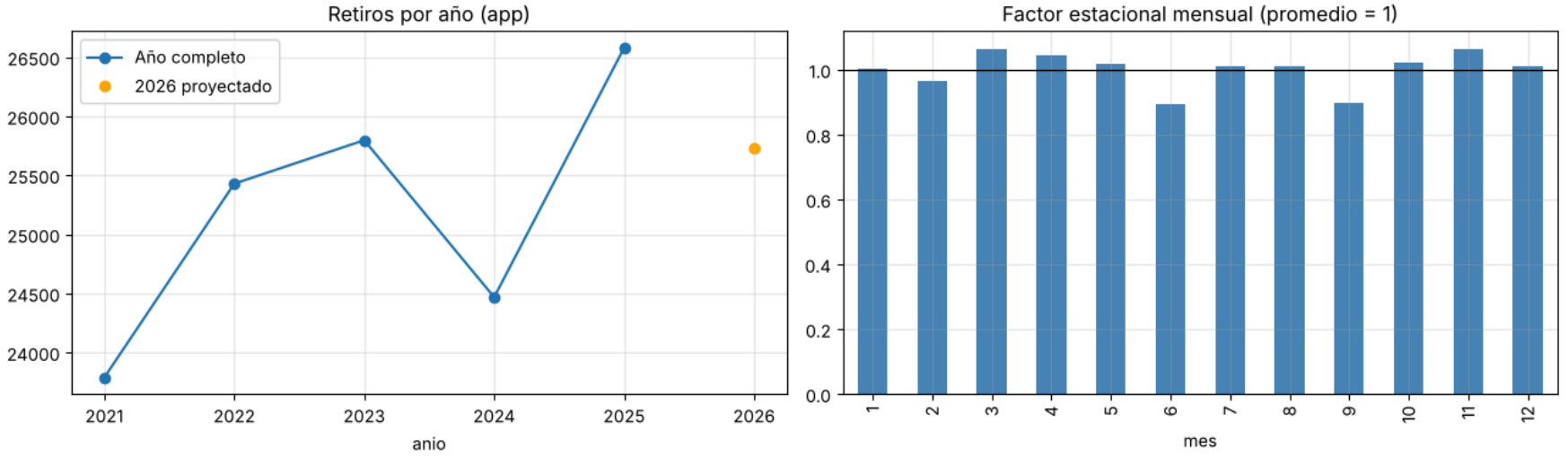
```
envíos (20091, 29) | retiros app (51409, 6) | materiales (2399, 20)
```

3.1 Tendencia anual y estacionalidad (retiros de la app 2021-2026)

In [6]:
```
r = ret.groupby(['anio','mes']).retiros.sum().unstack()
anual = r.sum(axis=1); meses_2026 = r.loc[2026].notna().sum()
completo = r.loc[2021:2025]
estac = completo.div(completo.mean(axis=1), axis=0).mean(); estac = estac / estac.mean()
anual_2026_proy = r.loc[2026].sum() / estac.iloc[:meses_2026].sum() * 12
cagr = (anual.loc[2025] / anual.loc[2021]) ** (1/4) - 1
tab = pd.DataFrame({'retiros_app': anual}); tab.loc[2026,'retiros_app_proy'] = anual_2026_proy
display(tab); print(f'CAGR retiros 2021-2025: {cagr:.2%} | meses con datos en 2026: {meses_2026}')
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
anual.loc[2021:2025].plot(marker='o', ax=ax[0], label='Año completo'); ax[0].plot([2026], [anual_2026_proy], 'o', color='orange', label='2026 proyectado')
ax[0].set_title('Retiros por año (app)'); ax[0].legend()
estac.plot(kind='bar', ax=ax[1], color='steelblue'); ax[1].axhline(1, color='k', lw=.8); ax[1].set_title('Factor estacional mensual (promedio = 1)')
plt.tight_layout(); plt.show()
```

anio	retiros_app	retiros_app_proy
2021	23,788.00	NaN
2022	25,429.00	NaN
2023	25,797.00	NaN
2024	24,468.00	NaN
2025	26,579.00	NaN
2026	17,162.00	25,730.90

```
CAGR retiros 2021-2025: 2.81% | meses con datos en 2026: 8
```



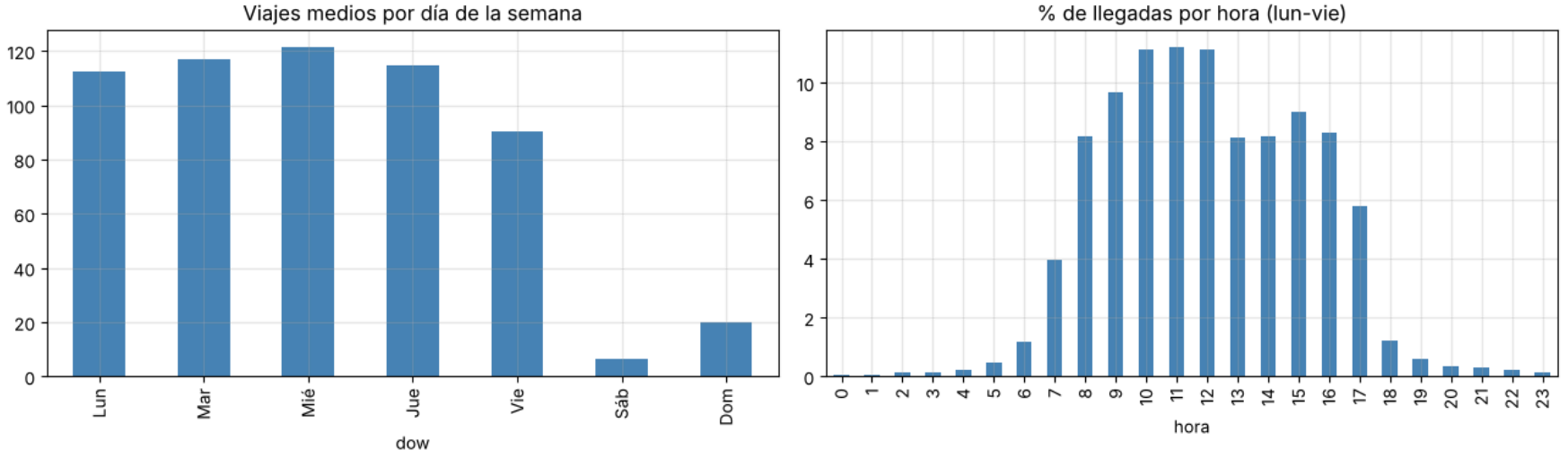
3.2 Día de la semana, hora y contenedores (envíos ene-ago 2026)

In [7]:
```
n = env[env.tipo_envio == 'normal'].copy()
n['fecha'] = pd.to_datetime(n.fecha)
n['hora'] = pd.to_datetime(n.hora_llegada_pozo, format='%I:%M %p', errors='coerce').dt.hour
n['dow'] = n.fecha.dt.dayofweek
dia = n.groupby('fecha').size().rename('viajes').to_frame()
dia['dow'] = dia.index.dayofweek; dia['mes'] = dia.index.month
dia['viajes_desest'] = dia.viajes / dia.mes.map(estac)
f_dow = dia.groupby('dow').viajes_desest.mean().reindex(range(7)).fillna(0)
cal = pd.date_range(dia.index.min(), dia.index.max())
cal = cal[[not (LLEG['usar_feriados'] and d in FERIADOS) for d in cal]]
for d_ in (5, 6):   # sábados/domingos: media por día calendario (incluye días sin viajes)
    f_dow[d_] = dia[dia.dow == d_].viajes_desest.sum() / max(sum(1 for d in cal if d.dayofweek == d_), 1)
if not LLEG['domingos_operan']: f_dow[6] = 0.0
nombres = ['Lun','Mar','Mié','Jue','Vie','Sáb','Dom']
hab = dia[dia.dow < 5]
cv_dia = (hab.viajes / (hab.dow.map(f_dow) * hab.mes.map(estac))).std()
cv_gamma = np.sqrt(max(cv_dia**2 - 1/hab.viajes.mean(), 0))
perfil_lv  = n[n.dow < 5].hora.value_counts(normalize=True).reindex(range(24), fill_value=0)
perfil_sab = n[n.dow == 5].hora.value_counts(normalize=True).reindex(range(24), fill_value=0)
mix = n.tipo_contenedor_m3.value_counts(normalize=True); m3_tipos = mix.index.values.astype(float); p_tipos = mix.values
```

```
def t_base(m3): return np.where(m3 == 9, LLEG['t_contenedor_9'], np.where(m3 == 10, LLEG['t_contenedor_10'], m3 * LLEG['dens_otros']))
t_viaje_contenedores = (t_base(m3_tipos) * p_tipos).sum()
esc_peso = 1.0 if LLEG['t_por_viaje_objetivo'] is None else LLEG['t_por_viaje_objetivo'] / t_viaje_contenedores
def t_esperado(m3): return t_base(m3) * esc_peso
t_viaje_medio = (t_esperado(m3_tipos) * p_tipos).sum()
m3_viaje_medio = (m3_tipos * p_tipos).sum() * LLEG['factor_llenado']
display(pd.DataFrame({'viajes_medios': dia.groupby('dow').viajes.mean().reindex(range(7)), 'factor_modelo': f_dow}).rename(index=dict(enumerate(nombres))).T)
print(f'CV diario: {cv_dia:.3f} (gamma {cv_gamma:.3f}) | contenedores 9 m3 {mix.get(9.0,0):.1%}, 10 m3 {mix.get(10.0,0):.1%}, otros {1-mix.get(9.0,0)-mix.get(10.0,0):.1%}')
print(f'Peso medio por viaje: {t_viaje_medio:.2f} t (alternativas: 4,43 → ~125 mil t/año; 4,8 → ~135 mil t/año)')
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
dia.groupby('dow').viajes.mean().reindex(range(7)).rename(index=dict(enumerate(nombres))).plot(kind='bar', ax=ax[0], color='steelblue', title='Viajes medios por día de la semana
(perfil_lv*100).plot(kind='bar', ax=ax[1], color='steelblue', title='% de llegadas por hora (lun-vie)'); plt.tight_layout(); plt.show()
```

dow	Lun	Mar	Mié	Jue	Vie	Sáb	Dom
viajes_medios	112.62	116.76	121.50	114.88	90.27	6.71	20.00
factor_modelo	112.69	116.86	121.34	114.93	90.48	6.31	0.00

```
CV diario: 0.212 (gamma 0.189) | contenedores 9 m3 43.6%, 10 m3 42.7%, otros 13.7%
Peso medio por viaje: 5.59 t (alternativas: 4,43 → ~125 mil t/año; 4,8 → ~135 mil t/año)
```



3.3 Calibración del nivel de llegadas

In [8]:
```
meses_env = sorted(n.fecha.dt.month.unique())
viajes_2026 = len(n) / estac.loc[meses_env].sum() * 12
g = cagr if LLEG['crecimiento'] is None else LLEG['crecimiento']
habiles_2026 = [d for d in pd.date_range('2026-01-01','2026-12-31') if not (LLEG['usar_feriados'] and d in FERIADOS)]
k_base = viajes_2026 / np.array([f_dow[d.dayofweek] * estac[d.month] for d in habiles_2026]).sum()
print(f'Viajes 2026 (envíos, anualizado): {viajes_2026:,.0f} | crecimiento: {g:.2%}/año | escala: {k_base:.3f}')
```

```
Viajes 2026 (envíos, anualizado): 28,206 | crecimiento: 2.81%/año | escala: 0.997
```

3.4 Materiales recuperados hoy (registro limpio v2)

In [9]:
```
mm_ = mat[mat.incluir_modelo].copy()
trips = (r.stack().dropna() * len(n) / r.loc[2026, meses_env].sum())        # viajes/mes a nivel envíos
```

```
trips.index.names = ['anio','mes']
rec_hist = mm_.groupby(['material','anio','mes']).agg(t=('t','sum'), clp=('monto_clp','sum')).reset_index()
rec_hist = rec_hist.merge(trips.rename('viajes').reset_index(), on=['anio','mes'])
rec_hist['kg_viaje'] = rec_hist.t * 1000 / rec_hist.viajes
tabla_kgv = rec_hist.groupby(['material','anio']).kg_viaje.mean().unstack().round(2)
display(tabla_kgv.rename_axis(index='kg recuperados por viaje'))
print('La caída de madera y cartón en 2025-26 es por decisión de REGEMAC (dejó de priorizarlos), no por menor contenido.')
```

kg recuperados por viaje	anio 2024	2025	2026
aluminio	0.19	0.18	0.19
carton	1.85	1.06	0.40
fierro	18.76	18.91	20.90
madera	26.25	8.66	4.91
neumaticos	NaN	0.59	NaN
plastico	0.77	0.59	NaN

```
La caída de madera y cartón en 2025-26 es por decisión de REGEMAC (dejó de priorizarlos), no por menor contenido.
```

4. Funciones del modelo

In [10]:

```python
from contextlib import contextmanager
@contextmanager
def temporal(d, clave, valor):
    # Cambia d[clave] solo dentro del bloque 'with' (para sensibilidades sin tocar los parámetros)
    viejo = d[clave]; d[clave] = valor
    try: yield
    finally: d[clave] = viejo

def es_planta(sc): return 'operarios_picking' in sc

def simular_llegadas(seed=None, detalle_horario=False, fecha_ini=None, fecha_fin=None):
    rng = np.random.default_rng(seed)
    fechas = pd.date_range(fecha_ini or f"{SIM['anio_inicio']}-01-01", fecha_fin or f"{SIM['anio_inicio']+SIM['horizonte']-1}-12-31")
    ok = np.array([not (LLEG['usar_feriados'] and d in FERIADOS) for d in fechas])
    dow = fechas.dayofweek.values; mes = fechas.month.values; anio = fechas.year.values
    sh = 1 / cv_gamma**2
    lam = k_base * f_dow.values[dow] * estac.loc[mes].values * (1 + g) ** (anio - 2026) * ok * rng.gamma(sh, 1/sh, len(fechas))
    viajes = rng.poisson(lam)
    cont = rng.multinomial(viajes, p_tipos)
    t_mean = t_esperado(m3_tipos)
    sd = np.sqrt(cont @ (LLEG['cv_peso_viaje'] * t_mean) ** 2)
    df = pd.DataFrame({'fecha': fechas, 'anio': anio, 'mes': mes, 'dow': dow, 'viajes': viajes,
                       'entrada_t': np.maximum(cont @ t_mean + rng.standard_normal(len(fechas)) * sd, 0),
                       'entrada_m3': cont @ m3_tipos * LLEG['factor_llenado']})
    if not detalle_horario: return df
    perf = np.where(dow[:, None] == 5, perfil_sab.values, perfil_lv.values)
    return df, pd.DataFrame([rng.multinomial(v, p) for v, p in zip(viajes, perf)], index=fechas, columns=range(24))

def sortear_contenido(rng):
    c = {m: (rng.triangular(*COMP['contenido'][m]) if COMP['muestrear'] else COMP['contenido'][m][1]) for m in COMP['contenido']}
    c['inertes'] = 1 - sum(c.values()); return c

def precios_mensuales(rng, n_mes):
    out = {}
    for m, p0 in MERCADO['precio'].items():
```

```
        z = np.zeros(n_mes)
        for k in range(1, n_mes): z[k] = MERCADO['reversion'] * z[k-1] + MERCADO['vol_mensual'][m] * rng.standard_normal()
        out[m] = p0 * np.exp(z)
    rc = MERCADO['riesgo_comprador']
    if rc['usar']:      # episodios en que se pierde al comprador único: el precio cae 'descuento' durante 'meses'
        ev = rng.random(n_mes) < rc['prob_anual'] / 12
        caida = np.zeros(n_mes, bool)
        for k in np.flatnonzero(ev): caida[k:k + rc['meses']] = True
        for m in rc['materiales']: out[m] = np.where(caida, out[m] * (1 - rc['descuento']), out[m])
    return out

def acopio_no_permitidos(ingreso, reglas):
    # Acopio FIFO diario con reglas propias del escenario. Devuelve stock y salidas al vertedero (por tiempo / exceso).
    lotes, n_d = [], len(ingreso)
    stock, s_t, s_e = np.zeros(n_d), np.zeros(n_d), np.zeros(n_d)
    for i in range(n_d):
        if ingreso[i] > 0: lotes.append([i, ingreso[i]])
        if reglas['usar_limite_tiempo']:
            while lotes and i - lotes[0][0] >= reglas['dias_max']: s_t[i] += lotes.pop(0)[1]
        tot = sum(l[1] for l in lotes)
        if reglas['usar_limite_stock'] and tot > reglas['stock_max_t']:
            exc = tot - reglas['stock_max_t']; s_e[i] = exc
            while exc > 1e-9:
                q = min(lotes[0][1], exc); lotes[0][1] -= q; exc -= q
                if lotes[0][1] <= 1e-9: lotes.pop(0)
            tot = reglas['stock_max_t']
        stock[i] = tot
    return stock, s_t, s_e

try:
    from numba import njit
except ImportError:
    njit = lambda f: f
@njit
def cola_planta(arr, cap, max_h, max_t, usar_t, usar_s):
    # Modo 'reglas' (v0.5) — acopio FIFO por hora: llega arr[h], se procesa hasta cap[h]; salen al vertedero cohortes con antigüedad ≥ max_h o el exceso sobre max_t.
    n = len(arr); coh = np.zeros(n); head = 0; stock = 0.0
    proc = np.zeros(n); d_t = np.zeros(n); d_s = np.zeros(n); st = np.zeros(n)
    for h in range(n):
        coh[h] = arr[h]; stock += arr[h]
        c = cap[h]
        while c > 1e-12 and head <= h:
            q = min(coh[head], c); coh[head] -= q; c -= q; stock -= q; proc[h] += q
            if coh[head] <= 1e-12: head += 1
        if usar_t:
            while head <= h and h - head >= max_h:
                d_t[h] += coh[head]; stock -= coh[head]; coh[head] = 0.0; head += 1
        if usar_s and stock > max_t:
            exc = stock - max_t; d_s[h] = exc
            while exc > 1e-12 and head <= h:
                q = min(coh[head], exc); coh[head] -= q; exc -= q; stock -= q
                if coh[head] <= 1e-12: head += 1
        st[h] = stock
    return proc, d_t, d_s, st

@njit
def cola_control(arr, cap, trab, obj, kp, base, cap_v, ret, tope):
    # Acopio de espera FIFO por hora con vaciado controlado:
    #  1) llega arr[h]; si el acopio supera el tope, lo que llega se descarga directo en el vertedero (desvío)
    #  2) la planta procesa hasta cap[h] (lo más antiguo primero)
```

```
    #  3) en horas de trabajo (trab[h]=1) se vacía al vertedero: base + kp·(acopio observado − objetivo), entre 0 y cap_v
    n = len(arr); coh = np.zeros(n); head = 0; stock = 0.0
    proc = np.zeros(n); d_v = np.zeros(n); d_s = np.zeros(n); st = np.zeros(n)
    for h in range(n):
        coh[h] = arr[h]; stock += arr[h]
        if stock > tope:
            q = min(stock - tope, coh[h]); coh[h] -= q; stock -= q; d_s[h] = q
        c = cap[h]
        while c > 1e-12 and head <= h:
            q = min(coh[head], c); coh[head] -= q; c -= q; stock -= q; proc[h] += q
            if coh[head] <= 1e-12: head += 1
        if trab[h] > 0:
            s_obs = st[h - ret] if (ret > 0 and h - ret >= 0) else stock
            v = min(max(base + kp * (s_obs - obj), 0.0), cap_v, stock)
            d_v[h] = v
            while v > 1e-12 and head <= h:
                q = min(coh[head], v); coh[head] -= q; v -= q; stock -= q
                if coh[head] <= 1e-12: head += 1
        st[h] = stock
    return proc, d_v, d_s, st

def dens_inertes(cont, f=1.0):
    # Densidad de 'inertes' tal que la mezcla calce con la densidad media recibida (f escala esa densidad, ver con_densidad).
    rho_mix = t_viaje_medio / m3_viaje_medio * f
    resto = 1 / rho_mix - sum(cont[m] / DENS[m] for m in DENS)
    return max(cont['inertes'] / max(resto, 1e-6), 0.4)

def volumen(masas, cont):
    d = dict(DENS, inertes=dens_inertes(cont))
    return sum(masas[m] / d[m] for m in masas)

def preparar_corrida(seed):
    # Sorteos comunes a todos los escenarios (números aleatorios comunes): llegadas por hora, contenido, precios.
    rng = np.random.default_rng(seed)
    lleg, hor = simular_llegadas(seed=seed, detalle_horario=True)
    v = lleg.viajes.values.astype(float)
    t_h = np.divide(lleg.entrada_t.values, v, out=np.zeros_like(v), where=v > 0)[:, None] * hor.values
    cont = sortear_contenido(rng)
    meses = (lleg.anio.values - lleg.anio.values[0]) * 12 + lleg.mes.values - 1
    precios = {m: s_[meses] for m, s_ in precios_mensuales(rng, meses.max() + 1).items()}
    mult = {m: rng.gamma(1/cv**2, cv**2, meses.max() + 1)[meses] for m, cv in BASE['cv_mensual'].items()}
    return dict(lleg=lleg, t_h=t_h, cont=cont, precios=precios, mult=mult, meses=meses, f_dens=1.0)

def con_densidad(cr, rho):
    # Misma historia, pero con otra densidad media de lo recibido (t/m3): mismos viajes y m3, más o menos toneladas.
    f = rho / (t_viaje_medio / m3_viaje_medio)
    lleg = cr['lleg'].copy(); lleg['entrada_t'] = lleg.entrada_t * f
    return dict(cr, lleg=lleg, t_h=cr['t_h'] * f, f_dens=f)

# tasa de extracción manual por trabajador (t/h), calibrada con 2026
_h26 = sum(1 for d in habiles_2026 if d.dayofweek in BASE['dias']) * len(BASE['horas'])
TASA_EXTR = {m: kg / 1000 * viajes_2026 / _h26 / BASE['trabajadores'] for m, kg in BASE['kg_viaje_2026'].items()}

def extraccion_manual(E_dia, lleg, trabajadores, cr, horas, dias):
    # t/día que sacan los trabajadores del frente de descarga: mín(capacidad, CAPTURA_MAX_FOSA × lo que llegó).
    h_dia = np.where(np.isin(lleg.dow.values, dias) & (lleg.viajes.values > 0), len(horas), 0)
    anio_rel = lleg.anio.values - lleg.anio.values[0]
    R = {}
    for m in MAT:
        if m not in TASA_EXTR or trabajadores == 0: R[m] = np.zeros(len(lleg)); continue
```

```
        tend = (1 + BASE['tendencia_anual'].get(m, 0)) ** anio_rel
        cap = trabajadores * TASA_EXTR[m] * h_dia * cr['mult'][m] * tend
        R[m] = np.minimum(cap, CAPTURA_MAX_FOSA * E_dia[m])
    return R

def cerrar_balance(o, lleg, cont, R, E_vert_np_dia, vert_perm_mat, sep_np, sc, cr, extra=None):
    # Arma columnas comunes: acopio NP, vertedero (t y m3), ahorro, ingresos y costos.
    st, s_t, s_e = acopio_no_permitidos(sep_np, sc['acopio_np'])
    o['acopio_np_ingreso_t'] = sep_np; o['acopio_np_stock_t'] = st; o['acopio_np_a_vert_t'] = s_t + s_e
    o['vert_permitido_t'] = sum(vert_perm_mat.values())
    o['vert_noperm_t'] = sum(E_vert_np_dia.values()) + s_t + s_e
    o['vertedero_t'] = o.vert_permitido_t + o.vert_noperm_t
    for m in MAT: o[f'rec_{m}_t'] = R[m]
    o['recuperado_t'] = sum(R.values())
    # volumen: materiales al vertedero con su densidad; lo del acopio NP con densidad media de lo separado
    d = dict(DENS, inertes=dens_inertes(cont, cr.get('f_dens', 1.0)))
    v_np = sum(E_vert_np_dia[m] / d[m] for m in E_vert_np_dia)
    v_perm = sum(vert_perm_mat[m] / d[m] for m in vert_perm_mat)
    o['vertedero_m3'] = v_perm + v_np + (s_t + s_e) / d['yeso']
    # áridos (si el interruptor está activo): el árido vendido sale del predio → menos t y m3 al vertedero
    for k, v in flujo_aridos(o, sc, cr).items(): o[k] = v
    o['vert_permitido_t'] = o.vert_permitido_t - o.aridos_venta_t
    o['vertedero_t'] = o.vertedero_t - o.aridos_venta_t
    o['vertedero_m3'] = o.vertedero_m3 - o.aridos_venta_t / d['inertes']
    # ahorro = lo que entró y NO terminó en el vertedero ni quedó en inventario (acopio de espera / acopio NP)
    inv_m3 = (extra['acopio_espera_m3'] if extra and 'acopio_espera_m3' in extra else 0) + st / d['yeso']
    o['ahorro_vertedero_m3'] = o.entrada_m3 - o.vertedero_m3 - np.diff(np.asarray(inv_m3, dtype=float) + np.zeros(len(o)), prepend=0)
    pr = cr['precios']
    ing = {m: (R[m] * 1000 * pr[m] if sc['vende'].get(m, False) else np.zeros(len(o))) for m in pr}
    o['ing_fierro_clp'], o['ing_aluminio_clp'] = ing['fierro'], ing['aluminio']
    o['ing_otros_clp'] = ing['carton'] + ing['plastico']
    o['ingresos_metales_clp'] = sum(ing.values())
    for k, v in flujo_madera(R['madera'], sc).items(): o[k] = v
    o['costo_madera_clp'] = o.costo_recupac_clp + o.costo_flete_astilla_clp + o.costo_chipeo_clp
    o['neto_madera_clp'] = o.ingreso_astilla_clp - o.costo_madera_clp          # astilla − RECUPAC − flete − chipeo
    o['neto_aridos_clp'] = o.ingreso_aridos_clp - o.costo_aridos_clp
    o['ingresos_clp'] = o.ingresos_metales_clp + o.ingreso_astilla_clp + o.ingreso_aridos_clp
    o['neto_materiales_clp'] = o.ingresos_metales_clp + o.neto_madera_clp + o.neto_aridos_clp
    for c_ in ['vertedero_t','vertedero_m3','vert_noperm_t','recuperado_t','ahorro_vertedero_m3']: o[f'acum_{c_}'] = o[c_].cumsum()
    if extra:
        for k, v in extra.items(): o[k] = v
    return o

# ---------- Áridos (interruptor ARIDOS['activo']) ----------
def aridos_en(sc): return ARIDOS['activo'] and sc.get('nombre') in ARIDOS['aplica_a']

def flujo_aridos(o, sc, cr):
    # Chancado del hormigón que llega separado (t/día y $/día). Arriendo: se junta hasta lote_min_t y se arrienda por días.
    n_d = len(o); z = lambda: np.zeros(n_d)
    out = dict(aridos_t=z(), aridos_venta_t=z(), ingreso_aridos_clp=z(), costo_aridos_clp=z())
    if not aridos_en(sc): return out
    A = ARIDOS; h = o.entrada_t.values * A['frac_hormigon']
    if A['modo'] == 'compra':
        p = A['compra']; chanc = h
        costo = h * p['opex_t'] + p['capex_usd'] * COSTOS['tc_usd'] * (1 + COSTOS['internacion']) * p['mantencion'] / 365
    else:
        p = A['arriendo']; chanc, costo = z(), z(); stock = 0.0
        var_t = A['diesel_l_t'] * COSTOS['precio_diesel'] + A['personal_t']
        for i in range(n_d):
            stock += h[i]
```

```
            if stock >= p['lote_min_t'] or (i == n_d - 1 and stock > 0):
                chanc[i] = stock
                costo[i] = (np.ceil(stock / p['t_dia']) * p['usd_dia'] + p['usd_campana']) * COSTOS['tc_usd'] + stock * var_t
                stock = 0.0
    prod = chanc * A['rendimiento']; venta = prod * A['frac_venta']
    out.update(aridos_t=prod, aridos_venta_t=venta, costo_aridos_clp=costo,
               ingreso_aridos_clp=venta * A['precio_venta_t'] + (prod - venta) * A['valor_uso_interno_t'])
    return out

# ---------- Bloques del CBO: imán, madera/astilla, costos ----------
def iman_de(sc):
    # Imán del escenario: el propio, o el del CBO si el escenario hereda las mejoras del CBO.
    if 'iman' in sc: return sc['iman']
    return CBO['iman'] if sc.get('mejoras_cbo') else None

def madera_de(sc):
    if 'madera' in sc: return sc['madera']
    return CBO['madera'] if sc.get('mejoras_cbo') else {'destino': 'recupac'}

def aplicar_iman(E_f, R, cr, sc):
    # Fierro (t/día) que saca el imán: r × (fracción de lo del día que llega en las horas del imán) × fierro que dejaron los trabajadores.
    # Suma a R['fierro'] respetando el tope técnico CAPTURA_MAX_FOSA. Devuelve lo aportado por el imán.
    im = iman_de(sc); n_d = len(E_f)
    if not im or not im['usar']: return np.zeros(n_d)
    t_h = cr['t_h']; tot = t_h.sum(axis=1)
    w = np.divide(t_h[:, im['horas']].sum(axis=1), tot, out=np.zeros(n_d), where=tot > 0)
    dia_ok = np.isin(cr['lleg'].dow.values, BASE['dias'])
    resto = np.maximum(E_f - R['fierro'], 0)
    R_im = np.minimum(im['recuperacion'] * w * resto * dia_ok, np.maximum(CAPTURA_MAX_FOSA * E_f - R['fierro'], 0))
    R['fierro'] = R['fierro'] + R_im
    return R_im

def campanas_chipeo(apta, modo):
    # Arriendo (Raico): la madera se junta hasta completar un lote (o hasta que pasan dias_max) y se chipea en una campaña.
    p = MADERA['chipeo'][modo]; n_d = len(apta)
    chip, costo = np.zeros(n_d), np.zeros(n_d); stock, edad = 0.0, 0
    for i in range(n_d):
        stock += apta[i]; edad = edad + 1 if stock > 0 else 0
        if stock > 0 and (stock >= p['lote_min_t'] or edad >= p['dias_max'] or i == n_d - 1):
            chip[i] = stock
            costo[i] = np.ceil(stock / p['t_semana']) * p['uf_semana'] * COSTOS['uf']   # semanas de arriendo Raico
            stock, edad = 0.0, 0
    return chip, costo

def flujo_madera(Rm, sc):
    # Destino de la madera recuperada (t/día) → t a RECUPAC, t de astilla y sus ingresos/costos ($/día).
    md_ = madera_de(sc); n_d = len(Rm)
    out = dict(t_recupac=np.asarray(Rm, float).copy(), t_astilla=np.zeros(n_d), ingreso_astilla_clp=np.zeros(n_d),
               costo_flete_astilla_clp=np.zeros(n_d), costo_chipeo_clp=np.zeros(n_d))
    if md_['destino'] == 'astilla':
        apta = Rm * MADERA['frac_apta']; out['t_recupac'] = Rm - apta
        modo = md_['modo_chipeo']
        if modo == 'compra':
            p = MADERA['chipeo']['compra']
            chip, costo = apta, apta / p['cap_th'] * p['factor_horas'] * p['kw'] * p['carga'] * COSTOS['precio_kwh']
        else:
            chip, costo = campanas_chipeo(apta, modo)
        out.update(t_astilla=chip, costo_chipeo_clp=costo, ingreso_astilla_clp=chip * MADERA['precio_astilla'],
                   costo_flete_astilla_clp=chip * MADERA['flete_astilla'])
    out['costo_recupac_clp'] = out['t_recupac'] * 1000 * MERCADO['costo_recupac']
```

```
    return out

def personal_extra(sc):
    if 'operarios_picking' in sc:
        return (sc['operarios_picking'] + sc['operarios_apoyo']) * sc.get('turnos', 1) + sc['trabajadores_fosa'] - BASE['trabajadores']
    return sc.get('trabajadores_extra', 0)

MONEDA = lambda mon: {'uf': COSTOS['uf'], 'usd': COSTOS['tc_usd'], 'gbp': COSTOS['tc_gbp'], 'clp': 1.0}[mon]
ITEMS_PLANTA = ['Planta (cotización)', 'Obras civiles, grúa, tablero']   # + capex_items de cada planta → base de la mantención

def capex(sc):
    # Inversión del escenario (CLP), por ítem.
    it = {}
    im = iman_de(sc)
    if im and im['usar']: it['Electroimán'] = im['capex_usd'] * (1 + COSTOS['internacion']) * COSTOS['tc_usd']
    if 'capex_gbp' in sc:
        it['Planta (cotización)'] = sc['capex_gbp'] * COSTOS['tc_gbp']
        it['Internación (agente, puerto)'] = it['Planta (cotización)'] * COSTOS['internacion_planta']
        it['Obras civiles, grúa, tablero'] = it['Planta (cotización)'] * sc['pct_obras']
    for k, (monto, mon, cat) in sc.get('capex_items', {}).items(): it[k] = monto * MONEDA(mon)
    if es_planta(sc):
        equipo = it.get('Planta (cotización)', 0) + sum(it[k] for k, v in sc.get('capex_items', {}).items() if v[2] == 'maquinaria')
        it['Repuestos iniciales'] = equipo * sc.get('repuestos', 0)
        if sc.get('energia', 'red') == 'red': it['Empalme eléctrico'] = EXTRA_PLANTA['empalme_uf'] * COSTOS['uf']
        it['Control de polvo'] = EXTRA_PLANTA['polvo_clp']
        if sc.get('alimentacion') == 'cargador_propio': it['Cargador frontal'] = ALIMENTACION['cargador_propio']['capex_clp']
    md_ = madera_de(sc)
    if md_['destino'] == 'astilla':
        it['Losa acopio astilla'] = EXTRA_PLANTA['losa_astilla_uf'] * COSTOS['uf']
        if md_['modo_chipeo'] == 'compra': it['Chipeadora'] = MADERA['chipeo']['compra']['capex_clp']
    if aridos_en(sc) and ARIDOS['modo'] == 'compra':
        it['Chancador'] = ARIDOS['compra']['capex_usd'] * COSTOS['tc_usd'] * (1 + COSTOS['internacion'])
    return it

def costo_mov_t():
    return MOVIMIENTO['diesel_lh'] * COSTOS['precio_diesel'] / MOVIMIENTO['t_por_hora']

def agregar_opex(o, sc, es_planta=False):   # (el argumento es_planta tapa a la función del mismo nombre solo aquí)
    # Costos de operación ($/día): adicionales a los de hoy + movimiento al pozo (en todos). Resultado operativo = neto materiales − OPEX.
    n_d = len(o); dia_op = (np.isin(o.dow.values, BASE['dias']) & (o.viajes.values > 0)).astype(float)
    fac = COSTOS['factor_costo_empresa'] * 12 / 365
    tur = sc.get('turnos', 1); f_tur = 1 + (tur - 1) * (1 + COSTOS['recargo_turno2'])     # turnos equivalentes en sueldo (2° turno con recargo)
    if es_planta:   # personas por turno (cabina + transferencia) en turnos equivalentes, menos los N actuales
        eq = (sc['operarios_picking'] + sc['operarios_apoyo']) * f_tur + sc['trabajadores_fosa'] - BASE['trabajadores']
    else:
        eq = personal_extra(sc)
    o['opex_personal_clp'] = (eq * COSTOS['sueldo_operario_mes'] + sc.get('supervisores_extra', 0) * f_tur * COSTOS['sueldo_supervisor_mes']) * fac
    f_mant = 1 - COSTOS['mant_fraccion_variable'] + COSTOS['mant_fraccion_variable'] * len(sc.get('horas', T1)) / len(T1)   # más horas → más desgaste
    energia, diesel, mant = np.zeros(n_d), np.zeros(n_d), np.zeros(n_d)
    im = iman_de(sc); cx = capex(sc)
    if im and im['usar']:
        diesel += dia_op * len(im['horas']) * im['diesel_lh'] * im['frac_diesel_incremental'] * COSTOS['precio_diesel']
        mant += cx['Electroimán'] * im['mantencion'] / 365
    if es_planta:
        h = o.horas_op.values
        if sc.get('energia', 'red') == 'generador':       # generador diésel 60 kVA (FT620C): 9,5 L/h al 50% de carga (ficha Kiverco)
            diesel += h * sc['generador_lh'] * COSTOS['precio_diesel']
        else:
            hp = o.horas_op_punta.values
            energia += ((h - hp) * COSTOS['precio_kwh'] + hp * COSTOS['precio_kwh_punta']) * sc['kw'] * sc['carga_electrica']
```

```
        modo = sc.get('alimentacion', 'por_tonelada')
        if modo == 'cargador_propio':                                   # cargador comprado + operador (en personal), todo el turno
            p = ALIMENTACION['cargador_propio']
            alim = h * p['diesel_lh'] * COSTOS['precio_diesel']
            mant += cx['Cargador frontal'] * p['mantencion'] / 365 * f_mant
        elif modo == 'arriendo':                                        # cargador arrendado con operador, por hora de planta
            p = ALIMENTACION['arriendo']
            alim = h * (p['clp_hora'] + p['diesel_lh'] * COSTOS['precio_diesel'])
        elif modo == 'por_hora':                                        # v1.0: solo el diésel de un cargador dedicado
            alim = h * sc['cargador_lh'] * COSTOS['precio_diesel']
        else:                                                           # v1.0: por tonelada cargada a la tolva
            alim = o.procesado_t.values * costo_mov_t() * sc.get('factor_alimentacion', 1.0)
        o['opex_alimentacion_clp'] = alim; diesel += alim
        base_mant = sum(cx.get(k, 0) for k in ITEMS_PLANTA + list(sc.get('capex_items', {})))
        mant += base_mant * sc['mantencion'] / 365 * f_mant
    if 'Chipeadora' in cx: mant += cx['Chipeadora'] * MADERA['chipeo']['compra']['mantencion'] / 365
    # movimiento al pozo (TODOS los escenarios): todo lo que termina en el vertedero se mueve una vez con retro/cargador
    o['opex_movimiento_clp'] = o.vertedero_t.values * costo_mov_t(); diesel += o.opex_movimiento_clp.values
    if 'opex_alimentacion_clp' not in o: o['opex_alimentacion_clp'] = 0.0
    o['opex_energia_clp'], o['opex_diesel_clp'], o['opex_mantencion_clp'] = energia, diesel, mant
    o['opex_clp'] = o.opex_personal_clp + energia + diesel + mant
    o['resultado_operativo_clp'] = o.neto_materiales_clp - o.opex_clp
    return o

def caso_fosa(cr, sc):
    # Caso base y CBO: todo se descarga en la fosa; los trabajadores (y el imán, si hay) sacan material; el resto va al vertedero.
    sc = {**BASE, **sc}                          # lo que el escenario no define se toma del caso base
    lleg, cont = cr['lleg'], cr['cont']
    E = {m: lleg.entrada_t.values * cont[m] for m in MAT}
    R = extraccion_manual(E, lleg, sc['trabajadores'] + sc.get('trabajadores_extra', 0), cr, sc['horas'], sc['dias'])
    R_im = aplicar_iman(E['fierro'], R, cr, sc)
    if madera_de(sc)['destino'] == 'fosa': R['madera'] = np.zeros(len(lleg))
    no_rec = {m: E[m] - R[m] for m in MAT}
    np_mats = [m for m in MAT if not PERMITIDO_VERTEDERO[m]]
    sep = sum(no_rec[m] for m in np_mats) * sc['frac_no_permitido_apartado']
    E_vert_np = {m: no_rec[m] * (1 - sc['frac_no_permitido_apartado']) for m in np_mats}
    vert_perm = {m: no_rec[m] for m in MAT if PERMITIDO_VERTEDERO[m]}
    o = lleg.copy(); o['procesado_t'] = 0.0; o['bypass_t'] = 0.0; o['acopio_espera_m3'] = 0.0
    o = cerrar_balance(o, lleg, cont, R, E_vert_np, vert_perm, sep, sc, cr, {'rec_fierro_iman_t': R_im})
    return agregar_opex(o, sc)

def caso_base(cr): return caso_fosa(cr, BASE)
def caso_cbo(cr, sc=None): return caso_fosa(cr, sc or CBO)

def horas_operacion(lleg, sc):
    # Matriz días × 24 con 1 en horas de operación de la planta.
    op = np.zeros((len(lleg), 24))
    dia_ok = np.isin(lleg.dow.values, sc['dias']) & np.array([not (LLEG['usar_feriados'] and d in FERIADOS) for d in lleg.fecha])
    op[np.ix_(dia_ok, sc['horas'])] = 1.0
    return op

def capacidad_th(sc):
    c_maq = sc['cap_nominal_th'] * sc['factor_densidad'] * sc['disponibilidad']
    c_op = sc['operarios_picking'] * sc['tasa_revision_th'] / sc['fraccion_a_picking']
    return min(c_maq, c_op), c_maq, c_op

def caso_planta(cr, sc):
    lleg, cont, t_h = cr['lleg'], cr['cont'], cr['t_h']
    n_d = len(lleg); cap, c_maq, c_op = capacidad_th(sc)
    cap_h = (horas_operacion(lleg, sc) * cap).ravel()
```

```
    rho = t_viaje_medio / m3_viaje_medio * cr.get('f_dens', 1.0)
    p = sc['acopio_espera']; arr = t_h.ravel()
    if p['modo'] == 'control':
        trab = horas_operacion(lleg, dict(sc, horas=p['horas_vaciado'] or sc['horas'])).ravel()
        base = max(arr.sum() - cap_h.sum(), 0) / max(trab.sum(), 1) if p['tasa_base'] == 'auto' else p['tasa_base']
        proc, d_v, d_s, st = cola_control(arr, cap_h, trab, p['objetivo_m3'] * rho, p['kp'], base, p['cap_vaciado_th'], p['retardo_h'], p['tope_m3'] * rho)
    else:
        proc, d_v, d_s, st = cola_planta(arr, cap_h, p['horas_max'], p['tope_m3'] * rho, p['usar_limite_tiempo'], True)
        d_v = d_v + d_s; d_s = np.zeros_like(d_s); base = np.nan      # en 'reglas' todo sale del montón (remanejo)
    dia = lambda x: x.reshape(n_d, 24).sum(axis=1)
    P_d, B_d = dia(proc), dia(d_v) + dia(d_s)          # procesado / al vertedero sin procesar (t/día)
    sin_mat = ((cap_h > 0) & (proc < cap_h - 1e-9)).astype(float)   # horas de turno en que la planta se quedó sin material
    # material procesado
    cap_ = captura_de(sc)
    Rp = {m: P_d * cont[m] * cap_.get(m, 0) for m in MAT}
    # lo no procesado: extracción manual opcional en la fosa
    Eb = {m: B_d * cont[m] for m in MAT}
    Rb = extraccion_manual(Eb, lleg, sc['trabajadores_fosa'], cr, BASE['horas'], BASE['dias'])
    R_im = aplicar_iman(Eb['fierro'], Rb, cr, sc)       # imán de la retro sobre lo no procesado (si mejoras_cbo)
    R = {m: Rp[m] + Rb[m] for m in MAT}
    # destinos
    sep_np = Rp['yeso'] + sum(Rp[m] for m in ['plastico','carton','aluminio'] if not sc['vende'].get(m, False))
    for m in ['yeso'] + [m for m in ['plastico','carton','aluminio'] if not sc['vende'].get(m, False)]: R[m] = np.zeros(n_d) + Rb[m]
    dest_mad = madera_de(sc)['destino']
    if dest_mad == 'fosa': R['madera'] = Rb['madera']
    np_mats = [m for m in MAT if not PERMITIDO_VERTEDERO[m]]
    E_vert_np = {m: (P_d * cont[m] - Rp[m]) + (Eb[m] - Rb[m]) + (Rp['madera'] if (m == 'madera' and dest_mad == 'fosa') else 0) for m in np_mats}
    vert_perm = {m: P_d * cont[m] + Eb[m] - Rb[m] for m in MAT if PERMITIDO_VERTEDERO[m]}
    o = lleg.copy()
    extra = {'procesado_t': P_d, 'bypass_t': B_d, 'vaciado_t': dia(d_v), 'desvio_tope_t': dia(d_s), 'horas_sin_material': dia(sin_mat),
             'acopio_espera_m3': st.reshape(n_d, 24)[:, -1] / rho, 'acopio_espera_max_m3': st.reshape(n_d, 24).max(axis=1) / rho,
             'horas_op': horas_operacion(lleg, sc).sum(axis=1), 'rec_fierro_iman_t': R_im,
             'horas_op_punta': horas_operacion(lleg, sc)[:, COSTOS['horas_punta']].sum(axis=1) * np.isin(lleg.mes.values, COSTOS['meses_punta'])}
    o = cerrar_balance(o, lleg, cont, R, E_vert_np, vert_perm, sep_np, sc, cr, extra)
    o = agregar_opex(o, sc, es_planta=True)
    o.attrs.update(cap=cap, c_maq=c_maq, c_op=c_op, acopio_h=st / rho, vaciado_h=d_v, desvio_h=d_s, proc_h=proc, tasa_base=base)
    return o

def resumen(o):
    a = o.groupby('anio').agg(entrada_t=('entrada_t','sum'), procesado_t=('procesado_t','sum'), bypass_t=('bypass_t','sum'),
        recuperado_t=('recuperado_t','sum'), fierro_t=('rec_fierro_t','sum'), madera_t=('rec_madera_t','sum'),
        vertedero_t=('vertedero_t','sum'), vert_permitido_t=('vert_permitido_t','sum'), vert_noperm_t=('vert_noperm_t','sum'), vertedero_m3=('vertedero_m3','sum'),
        ahorro_m3=('ahorro_vertedero_m3','sum'), iman_t=('rec_fierro_iman_t','sum'), astilla_t=('t_astilla','sum'),
        ingresos_clp=('ingresos_clp','sum'), costo_madera_clp=('costo_madera_clp','sum'), neto_clp=('neto_materiales_clp','sum'),
        ing_fierro_clp=('ing_fierro_clp','sum'), ing_aluminio_clp=('ing_aluminio_clp','sum'), ing_otros_clp=('ing_otros_clp','sum'),
        neto_madera_clp=('neto_madera_clp','sum'), neto_aridos_clp=('neto_aridos_clp','sum'), aridos_t=('aridos_t','sum'), aridos_venta_t=('aridos_venta_t','sum'),
        opex_clp=('opex_clp','sum'), opex_energia_clp=('opex_energia_clp','sum'), opex_personal_clp=('opex_personal_clp','sum'),
        opex_diesel_clp=('opex_diesel_clp','sum'), opex_mantencion_clp=('opex_mantencion_clp','sum'), opex_mov_clp=('opex_movimiento_clp','sum'), opex_alim_clp=('opex_alimentaci
        acopio_max_m3=('acopio_espera_m3','max'))
    a['pct_procesado'] = a.procesado_t / a.entrada_t
    a['pct_recuperado'] = a.recuperado_t / a.entrada_t       # Rodrigo: % de recuperación sobre lo recibido (medido 2025: ~0,7%)
    return a

def anios_pozo(r_, r0):
    # Años de vida útil del pozo que se ganan: m3 que se dejaron de disponer en el horizonte ÷ m3 que se disponen por año al final (caso de referencia).
    # Es una razón: no depende del factor de compactación si es el mismo en ambos escenarios.
    return (r0.vertedero_m3 - r_.vertedero_m3).sum() / r0.vertedero_m3.iloc[-1]

def delta(r_, r0):
    # Diferencia anual entre dos escenarios (r_ − r0), en unidades legibles.
```

```
    return pd.DataFrame({'Δ fierro (t)': r_.fierro_t - r0.fierro_t, 'Δ fierro imán (t)': r_.iman_t - r0.iman_t,
        'Δ madera (t)': r_.madera_t - r0.madera_t, 'Δ astilla (t)': r_.astilla_t - r0.astilla_t,
        'Δ NO permitido al vertedero (t)': r_.vert_noperm_t - r0.vert_noperm_t, 'Δ m3 ahorrados': r_.ahorro_m3 - r0.ahorro_m3,
        'Δ fierro ($MM)': (r_.ing_fierro_clp - r0.ing_fierro_clp)/1e6, 'Δ madera neta ($MM)': (r_.neto_madera_clp - r0.neto_madera_clp)/1e6,
        'Δ aluminio y otros ($MM)': (r_.ing_aluminio_clp + r_.ing_otros_clp - r0.ing_aluminio_clp - r0.ing_otros_clp)/1e6,
        'Δ áridos ($MM)': (r_.neto_aridos_clp - r0.neto_aridos_clp)/1e6,
        'Δ neto materiales ($MM)': (r_.neto_clp - r0.neto_clp)/1e6, 'Δ OPEX ($MM)': (r_.opex_clp - r0.opex_clp)/1e6,
        'Δ mov. al pozo ($MM)': (r_.opex_mov_clp - r0.opex_mov_clp)/1e6, 'Δ alimentación planta ($MM)': (r_.opex_alim_clp - r0.opex_alim_clp)/1e6,
        'Δ resultado operativo ($MM)': (r_.resultado_clp - r0.resultado_clp)/1e6})
```

4.1 Densidades por material (t → m3)

Cada tonelada que sale del pozo (vendida, a RECUPAC o astilla) ahorra el volumen que **ese material** ocuparía, no el del RCD promedio. La tabla muestra los m3 por tonelada y el aporte de cada material al volumen de un m3 de RCD recibido, con la composición en su moda.

In [11]:
```
c_moda = {m: v[1] for m, v in COMP['contenido'].items()}; c_moda['inertes'] = 1 - sum(c_moda.values())
d_ = dict(DENS, inertes=dens_inertes(c_moda))
tab_d = pd.DataFrame({'densidad (t/m3)': d_, 'm3 por t': {m: 1/v for m, v in d_.items()}, '% masa (moda)': c_moda})
tab_d['% del volumen'] = tab_d['% masa (moda)'] * tab_d['m3 por t'] / (tab_d['% masa (moda)'] * tab_d['m3 por t']).sum()
display(tab_d.style.format({'densidad (t/m3)': '{:.2f}', 'm3 por t': '{:.2f}', '% masa (moda)': '{:.2%}', '% del volumen': '{:.1%}'}))
print(f'Densidad de la mezcla recibida: {t_viaje_medio/m3_viaje_medio:.3f} t/m3 → inertes {d_["inertes"]:.2f} t/m3 (NCh3562 Tabla D.3: inerte limpio 1,24; con esponjamiento ≥30%
print('Ojo: son densidades de residuo SUELTO. En el pozo el material se compacta; el m3 de pozo ahorrado real depende de esa compactación (dato a pedir a Rodrigo).')
```

	densidad (t/m3)	m3 por t	% masa (moda)	% del volumen
fierro	0.42	2.38	1.50%	1.9%
aluminio	0.90	1.11	0.05%	0.0%
madera	0.34	2.94	2.50%	3.9%
carton	0.05	20.00	0.50%	5.2%
plastico	0.21	4.76	1.00%	2.5%
yeso	0.33	3.03	5.40%	8.6%
inertes	0.60	1.67	89.05%	77.9%

```
Densidad de la mezcla recibida: 0.525 t/m3 → inertes 0.60 t/m3 (NCh3562 Tabla D.3: inerte limpio 1,24; con esponjamiento ≥30%)
Ojo: son densidades de residuo SUELTO. En el pozo el material se compacta; el m3 de pozo ahorrado real depende de esa compactación (dato a pedir a Rodrigo).
```

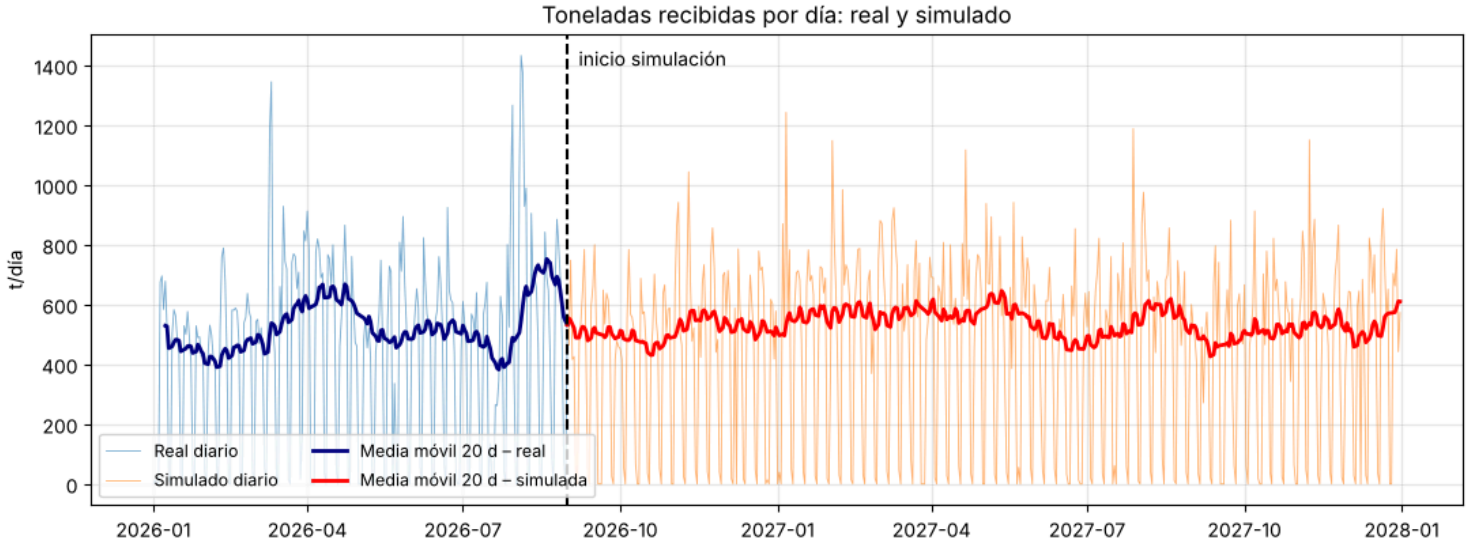
5. Llegadas: validación y real vs simulado

In [12]:
```
lleg, horario = simular_llegadas(seed=SIM['semilla'], detalle_horario=True)
sim_hab = lleg[(lleg.dow < 5) & (lleg.viajes > 0) & (lleg.anio == SIM['anio_inicio'])]
display(pd.DataFrame({'real 2026': [hab.viajes.mean(), hab.viajes.std(), hab.viajes.quantile(.95)],
                      f"simulado {SIM['anio_inicio']}": [sim_hab.viajes.mean(), sim_hab.viajes.std(), sim_hab.viajes.quantile(.95)]},
                     index=['media viajes/día (lun-vie)', 'desv. estándar', 'P95']))
```

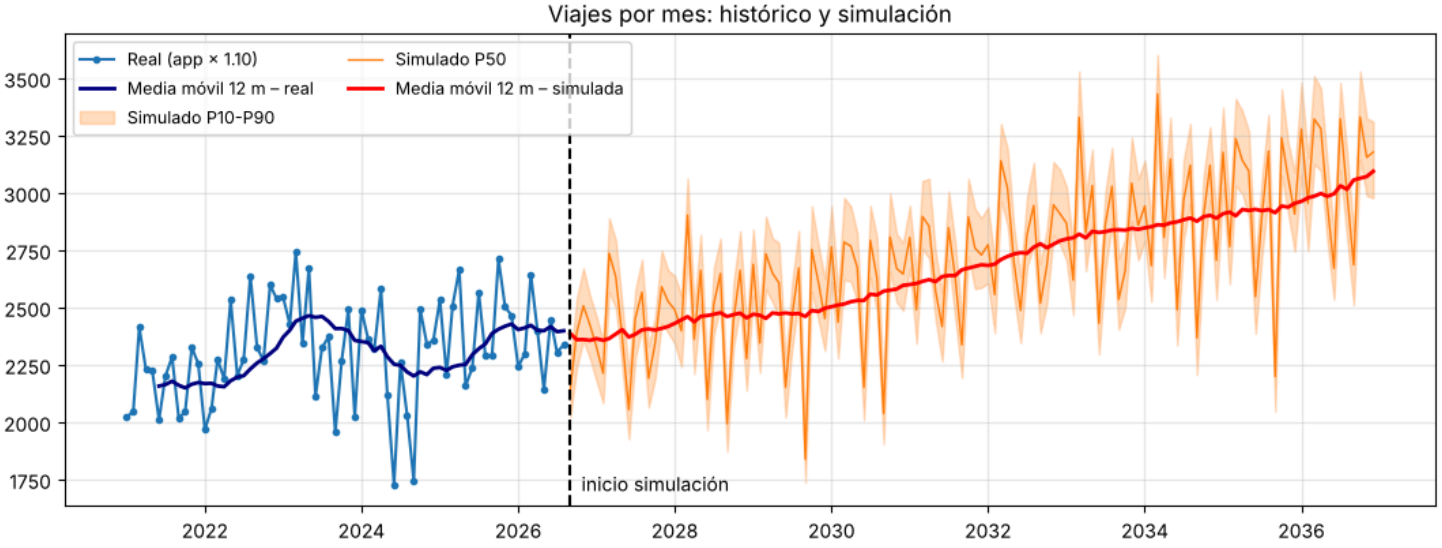
	real 2026	simulado 2027
media viajes/día (lun-vie)	111.29	113.84
desv. estándar	25.38	26.45
P95	150.10	165.50

In [13]:
```
# 5.1 Diario: real ene-ago 2026 + simulado desde sep-2026 (media móvil 20 días)
n['t_viaje'] = t_esperado(n.tipo_contenedor_m3.fillna(9).astype(float).values)
real_d = n.groupby('fecha').agg(t=('t_viaje','sum')).t
real_d = real_d.reindex(pd.date_range('2026-01-01', real_d.index.max()), fill_value=0)
corte = real_d.index.max()
```

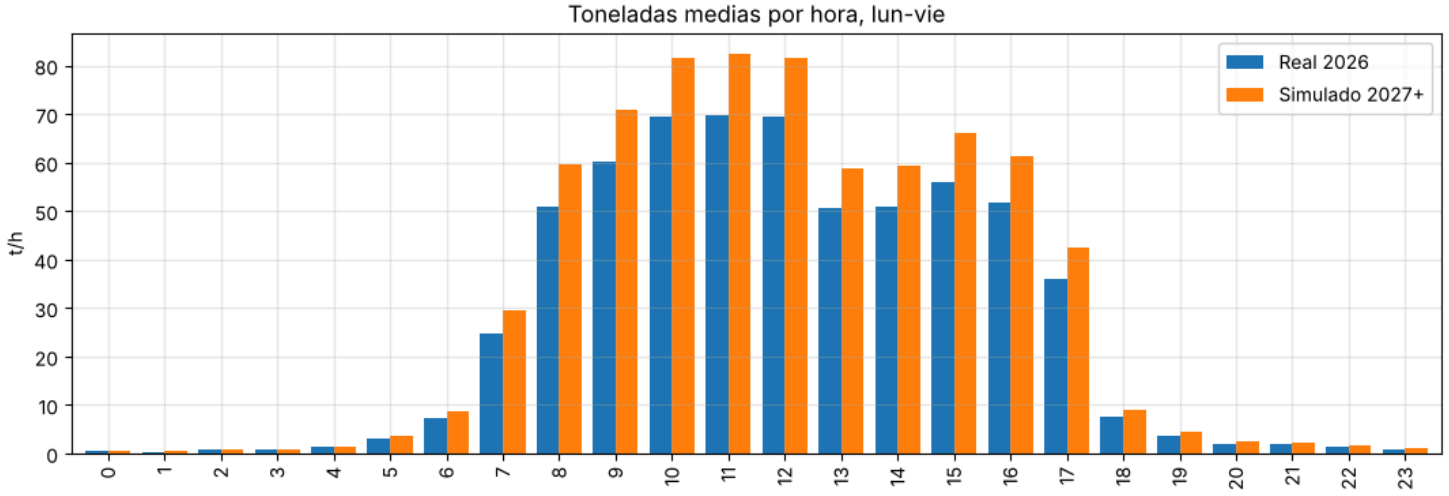
```
sim_d = simular_llegadas(seed=SIM['semilla'], fecha_ini=corte + pd.Timedelta(days=1), fecha_fin='2027-12-31').set_index('fecha').entrada_t
s_all = pd.concat([real_d, sim_d]); mmv = s_all[s_all > 0].rolling(20, min_periods=5).mean()
fig, ax = plt.subplots(figsize=(13, 4.5))
ax.plot(real_d.index, real_d, lw=.6, alpha=.6, color='tab:blue', label='Real diario'); ax.plot(sim_d.index, sim_d, lw=.6, alpha=.6, color='tab:orange', label='Simulado diario')
ax.plot(mmv[:corte].index, mmv[:corte], color='navy', lw=2, label='Media móvil 20 d — real'); ax.plot(mmv[corte:].index, mmv[corte:], color='red', lw=2, label='Media móvil 20 d
ax.axvline(corte, color='k', ls='--'); ax.annotate('inicio simulación', (corte, ax.get_ylim()[1]*.93), xytext=(6, 0), textcoords='offset points')
ax.set_title('Toneladas recibidas por día: real y simulado'); ax.set_ylabel('t/día'); ax.legend(loc='lower left', ncol=2, fontsize=9); plt.show()
```



In [14]:
```
# 5.2 Mensual 2021-2036 con banda P10-P90 (200 corridas)
esc_app = len(n) / r.loc[2026, meses_env].sum()
real_m = r.stack().dropna() * esc_app; real_m.index = pd.to_datetime([f'{a}-{m:02d}-01' for a, m in real_m.index]); real_m = real_m[real_m.index <= corte]
fin = f"{SIM['anio_inicio']+SIM['horizonte']-1}-12-31"
corr = pd.concat([simular_llegadas(seed=1000+i, fecha_ini=corte + pd.Timedelta(days=1), fecha_fin=fin).set_index('fecha').viajes.resample('MS').sum() for i in range(200)], axis=
band = corr.quantile([.1, .5, .9], axis=1).T
fig, ax = plt.subplots(figsize=(13, 4.5))
ax.plot(real_m.index, real_m, color='tab:blue', marker='.', label=f'Real (app × {esc_app:.2f})'); ax.plot(real_m.index, real_m.rolling(12, min_periods=6).mean(), color='navy', l
ax.fill_between(band.index, band[.1], band[.9], color='tab:orange', alpha=.25, label='Simulado P10-P90'); ax.plot(band.index, band[.5], color='tab:orange', lw=1, label='Simulado
mm2 = pd.concat([real_m, band[.5]]).rolling(12, min_periods=6).mean(); ax.plot(mm2[band.index.min():].index, mm2[band.index.min():], color='red', lw=2, label='Media móvil 12 m -
ax.axvline(corte, color='k', ls='--'); ax.annotate('inicio simulación', (corte, ax.get_ylim()[0]), xytext=(6, 8), textcoords='offset points')
ax.set_title('Viajes por mes: histórico y simulación'); ax.legend(loc='upper left', ncol=2, fontsize=9); plt.show()
```



In [15]:
```
# 5.3 Perfil horario lun-vie: real vs simulado
hs = horario[(horario.index.dayofweek < 5) & (horario.sum(axis=1) > 0)]
real_h = n[n.dow < 5].groupby(['fecha','hora']).size().unstack(fill_value=0).reindex(columns=range(24), fill_value=0).mean()
pd.DataFrame({'Real 2026': real_h * t_viaje_medio, f"Simulado {SIM['anio_inicio']}+": hs.mean() * t_viaje_medio}).plot(kind='bar', figsize=(13, 4), width=.8, color=['tab:blue','
plt.title('Toneladas medias por hora, lun-vie'); plt.ylabel('t/h'); plt.show()
```



6. Una corrida de todos los escenarios (mismas llegadas, composición y precios)

Primero se sortea una "historia" de 10 años (`cr`): camiones por hora, contenido de cada material y precios mensuales. Después cada escenario se aplica a esa misma historia.

In [16]:
```
cr = preparar_corrida(SIM['semilla'])
print('Contenido sorteado (% masa):', {k: f'{v:.2%}' for k, v in cr['cont'].items()}, f"| densidad inertes: {dens_inertes(cr['cont']):.2f} t/m3")
ESC = {'Caso base': caso_base(cr), 'CBO': caso_cbo(cr)}
SC_DE = {'Caso base': BASE, 'CBO': CBO}            # parámetros de cada escenario (para CAPEX)
for k, sc in PLANTAS.items():
    ESC[sc['nombre']] = caso_planta(cr, sc); SC_DE[sc['nombre']] = sc
    c, cm, co = capacidad_th(sc)
    pt = sc['operarios_picking'] + sc['operarios_apoyo']
    print(f"{sc['nombre']}: capacidad efectiva {c:.1f} t/h  (máquina {cm:.1f} | operarios {co:.1f})  → {'manda la MÁQUINA' if cm <= co else 'mandan los OPERARIOS'}"
          f" | personal por turno {sc['operarios_picking']} cabina + {sc['operarios_apoyo']} transferencia = {pt} → {pt - BASE['trabajadores']:+d} vs los {BASE['trabajadores']}")
R_ = {k: resumen(o) for k, o in ESC.items()}       # resumen anual de cada escenario
DIAS = ['Lun', 'Mar', 'Mié', 'Jue', 'Vie', 'Sáb', 'Dom']
# Un color fijo por escenario en todos los gráficos
COLOR = {'Caso base': '#7f7f7f', 'CBO': 'tab:blue', 'Planta Nivel 1': 'tab:purple', 'Planta FT620C + PS123': 'tab:green', 'Planta KS514': 'tab:orange'}
# Un color fijo por componente del beneficio
COLOR_COMP = {'Fierro': '#8B4513', 'Madera (neta)': '#DAA520', 'Aluminio, cartón, plástico': '#17becf', 'Áridos (neto)': '#bdbdbd',
              'Espacio de pozo': '#2ca02c', 'OPEX': '#d62728', 'Seguros y permisos': '#ffbb78', 'Impuestos': '#ff9896',
              'CAPEX, residual y capital de trabajo': '#525252'}
col_de = lambda nombre: COLOR.get(nombre.split(' vs ')[0], 'k')
```

```
Contenido sorteado (% masa): {'fierro': '2.07%', 'aluminio': '0.07%', 'madera': '4.37%', 'carton': '1.06%', 'plastico': '0.60%', 'yeso': '6.61%', 'inertes': '85.23%'} | densidad
inertes: 0.66 t/m3
Planta Nivel 1: capacidad efectiva 24.0 t/h  (máquina 55.2 | operarios 24.0)  → mandan los OPERARIOS | personal por turno 8 cabina + 2 transferencia = 10 → +2 vs los 8 actuales
Planta FT620C + PS123: capacidad efectiva 26.8 t/h  (máquina 26.8 | operarios 106.7)  → manda la MÁQUINA | personal por turno 8 cabina + 2 transferencia = 10 → +2 vs los 8 actual
es
Planta KS514: capacidad efectiva 30.4 t/h  (máquina 30.4 | operarios 106.7)  → manda la MÁQUINA | personal por turno 8 cabina + 2 transferencia = 10 → +2 vs los 8 actuales
```

6.1 Caso base: validación contra los datos de REGEMAC

Datos de Rodrigo (9-oct): ventas de reciclaje $108,7 MM en 2025 y $81,4 MM entre enero y agosto de 2026; madera a RECUPAC 258 t en 2025 y 93 t entre enero y agosto de 2026; lo que efectivamente salió del pozo en 2025 suma ~875 t, alrededor de **0,7% de lo recibido**. El 3% es una estimación de la operación y se usa como escenario, no como dato.

In [17]:
```
b1 = R_['Caso base'].iloc[0]
display(pd.DataFrame({'REGEMAC 2026 (ene-ago × 12/8)': [401.0*12/8, 93*12/8, 81.4*12/8, np.nan],
                      'REGEMAC 2025': [np.nan, 258, 108.7, 0.007],
                      f"simulado {SIM['anio_inicio']}": [b1.fierro_t, b1.madera_t, b1.ingresos_clp/1e6, b1.pct_recuperado]},
                     index=['fierro vendido (t)', 'madera a RECUPAC (t)', 'ventas reciclaje ($ MM)', '% recuperado sobre lo recibido']).style.format('{:,.1f}', na_rep='–').forma
print('El modelo recupera algo menos que el 0,7% medido porque el registro limpio solo cuenta lo vendido (fierro, aluminio, cartón) y la madera;')
print('las ~875 t de Rodrigo incluyen material separado que no se vende de inmediato (plástico, neumáticos).')
print('Capacidad de extracción por trabajador (t/h):', {m: round(v, 4) for m, v in TASA_EXTR.items()})
```

	REGEMAC 2026 (ene-ago × 12/8)	REGEMAC 2025	simulado 2027
fierro vendido (t)	601.5	—	560.3
madera a RECUPAC (t)	139.5	258.0	133.2
ventas reciclaje ($ MM)	122.1	108.7	122.0
% recuperado sobre lo recibido	—	0.70%	0.44%

```
El modelo recupera algo menos que el 0,7% medido porque el registro limpio solo cuenta lo vendido (fierro, aluminio, cartón) y la madera;
las ~875 t de Rodrigo incluyen material separado que no se vende de inmediato (plástico, neumáticos).
Capacidad de extracción por trabajador (t/h): {'fierro': np.float64(0.0328), 'aluminio': np.float64(0.0003), 'madera': np.float64(0.0078), 'carton': np.float64(0.0006), 'plastic
o': np.float64(0.0)}
```

6.2 CBO: qué agregan el imán y la chipeadora

Cómo leerlo:

- **Imán.** Muestra cuánto fierro adicional saca el imán y cuántos años tarda en pagarse solo.
- **Madera.** Compara las salidas posibles: seguir pagando a RECUPAC, comprar chipeadora o arrendar a Raico. El resultado operativo ya descuenta flete, chipeo y mantención.

Ojo con la opción "todo el turno" del imán: supone que la retro se dedica al imán y deja de hacer su trabajo actual, y que el imán no compite con los trabajadores. Es un techo optimista.

In [18]:
```
rb, rc = R_['Caso base'], R_['CBO']
d_cbo = delta(rc, rb)
print('CBO − Caso base, promedio anual 10 años:'); display(d_cbo.mean().to_frame('promedio').T.round(1))
cx_cbo = capex(CBO); print('CAPEX CBO ($):', {k: f'{v:,.0f}' for k, v in cx_cbo.items()})
print(f"Fierro que llega al año: {(ESC['CBO'].entrada_t * cr['cont']['fierro']).sum()/SIM['horizonte']:,.0f} t | lo saca la gente: {rb.fierro_t.mean():,.0f} t | el imán agrega:
```

```
CBO − Caso base, promedio anual 10 años:
```

	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
promedio	88.30	88.30	0.00	93.90	-88.30	216.00	16.80	1.50	0.00	0.00	18.30	7.40	-0.00	0.00	10.90

```
CAPEX CBO ($): {'Electroimán': '17,658,000', 'Losa acopio astilla': '24,694,458', 'Chipeadora': '30,975,000'}
Fierro que llega al año: 3,808 t | lo saca la gente: 580 t | el imán agrega: 88 t
```

In [19]:
```
# 6.2a Sensibilidad del imán (como en el Excel de Nicolás): recuperación r × horas de trabajo del imán (sin chipeadora, para aislar el imán)
filas = []
for r_ in (0.12, 0.20, 0.30, 0.50):
    for etiqueta, horas in [('valle 13 y 17 h', [13, 17]), ('valle + 2 h', [12, 13, 17, 18]), ('todo el turno', list(range(8, 18)))]:
        sc2 = dict(CBO, iman=dict(CBO['iman'], recuperacion=r_, horas=horas), madera=BASE['madera'])
        d_ = delta(resumen(caso_cbo(cr, sc2)), rb).mean()
        inv = sum(capex(sc2).values()) / 1e6
        filas.append({'r': r_, 'horas imán': etiqueta, 'Δ fierro imán (t/año)': d_['Δ fierro imán (t)'],
                      'Δ resultado ($MM/año)': d_['Δ resultado operativo ($MM)'], 'CAPEX ($MM)': inv,
                      'payback (años)': inv / d_['Δ resultado operativo ($MM)'] if d_['Δ resultado operativo ($MM)'] > 0 else np.nan})
display(pd.DataFrame(filas).style.format({'r': '{:.0%}', 'Δ fierro imán (t/año)': '{:,.0f}', 'Δ resultado ($MM/año)': '{:,.1f}', 'CAPEX ($MM)': '{:,.1f}', 'payback (años)': '{:.
```

	r	horas imán	Δ fierro imán (t/año)	Δ resultado ($MM/año)	CAPEX ($MM)	payback (años)
0	12%	valle 13 y 17 h	53	5.7	17.7	3.1
1	12%	valle + 2 h	100	11.2	17.7	1.6
2	12%	todo el turno	347	47.7	17.7	0.4
3	20%	valle 13 y 17 h	88	12.4	17.7	1.4
4	20%	valle + 2 h	167	23.9	17.7	0.7
5	20%	todo el turno	579	91.8	17.7	0.2
6	30%	valle 13 y 17 h	132	20.8	17.7	0.8
7	30%	valle + 2 h	251	39.8	17.7	0.4
8	30%	todo el turno	868	146.8	17.7	0.1
9	50%	valle 13 y 17 h	221	37.6	17.7	0.5
10	50%	valle + 2 h	418	71.6	17.7	0.2
11	50%	todo el turno	1,446	256.8	17.7	0.1

In [20]:
```
# 6.2b Destino de la madera: RECUPAC vs astilla (compra / arriendo Raico), en el CBO y en la planta KS514
OPC_MADERA = {'RECUPAC (hoy)': dict(destino='recupac'), 'Astilla — compra chipeadora': dict(destino='astilla', modo_chipeo='compra'),
              'Astilla — arriendo Raico': dict(destino='astilla', modo_chipeo='arriendo')}
filas = []
for esc, f_ in [('CBO', lambda m: caso_cbo(cr, dict(CBO, madera=m))), ('Planta KS514', lambda m: caso_planta(cr, dict(PLANTAS['KS514'], madera=m)))]:
    for k, m in OPC_MADERA.items():
        o = f_(m); a = resumen(o).mean(); sc_ = dict(CBO, madera=m)
        cm = (o.costo_madera_clp.sum() - o.ingreso_astilla_clp.sum()) / max(o.rec_madera_t.sum(), 1e-9)
        filas.append({'escenario': esc, 'destino madera': k, 'madera recuperada (t/año)': a.madera_t, 'astilla (t/año)': a.astilla_t,
```

```
                       'costo neto madera ($/t)': cm, 'CAPEX chipeadora ($MM)': capex(sc_).get('Chipeadora', 0)/1e6,
                       'resultado operativo ($MM/año)': a.resultado_clp/1e6})
tm = pd.DataFrame(filas)
display(tm.style.format({c: '{:,.0f}' for c in tm.columns if c not in ('escenario', 'destino madera')} | {'resultado operativo ($MM/año)': '{:,.1f}', 'CAPEX chipeadora ($MM)': '
print('Costo neto madera = (RECUPAC + flete + chipeo − venta astilla) / t recuperada; negativo = la madera deja plata. No incluye mantención ni CAPEX de la chipeadora.')
print('Arriendo Raico: 400 UF por semana (35 h, ~700 t). Con poca madera se paga la semana completa por un lote chico.')
```

	escenario	destino madera	madera recuperada (t/año)	astilla (t/año)	costo neto madera ($/t)	CAPEX chipeadora ($MM)	resultado operativo ($MM/año)
0	CBO	RECUPAC (hoy)	134	0	15,126	0.0	89.7
1	CBO	Astilla – compra chipeadora	134	94	3,654	31.0	88.1
2	CBO	Astilla – arriendo Raico	134	94	120,365	0.0	75.6
3	Planta KS514	RECUPAC (hoy)	1,485	0	15,126	0.0	56.8
4	Planta KS514	Astilla – compra chipeadora	1,485	1,040	3,654	31.0	70.8
5	Planta KS514	Astilla – arriendo Raico	1,485	1,040	29,730	0.0	35.1

```
Costo neto madera = (RECUPAC + flete + chipeo − venta astilla) / t recuperada; negativo = la madera deja plata. No incluye mantención ni CAPEX de la chipeadora.
Arriendo Raico: 400 UF por semana (35 h, ~700 t). Con poca madera se paga la semana completa por un lote chico.
```

6.2c ¿Conviene chipear? Compra vs arriendo Raico vs seguir pagando a RECUPAC

La comparación es solo para la madera **apta** (70%): la no apta va a RECUPAC en las tres opciones, y la madera sale del pozo en las tres, así que el espacio de pozo no cambia la decisión.

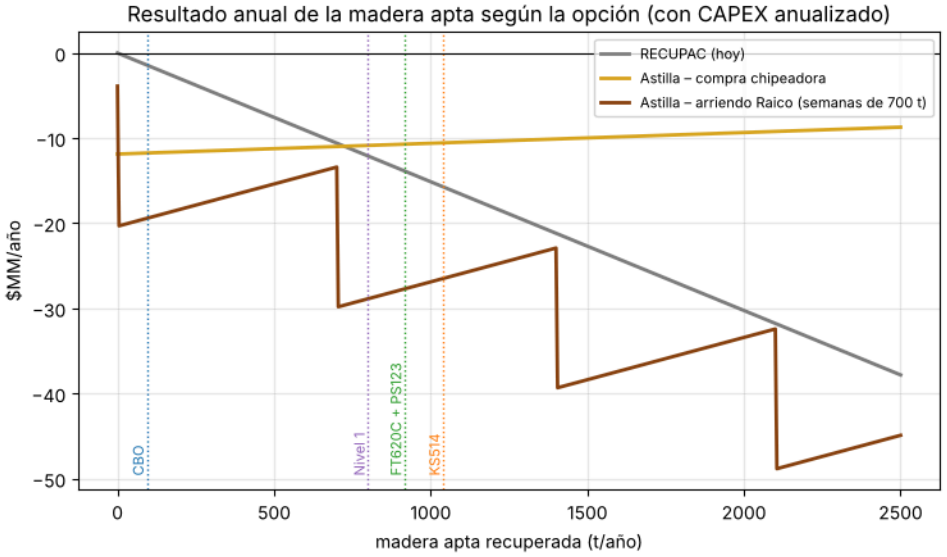
Unidades (lo que cambia al chipear):

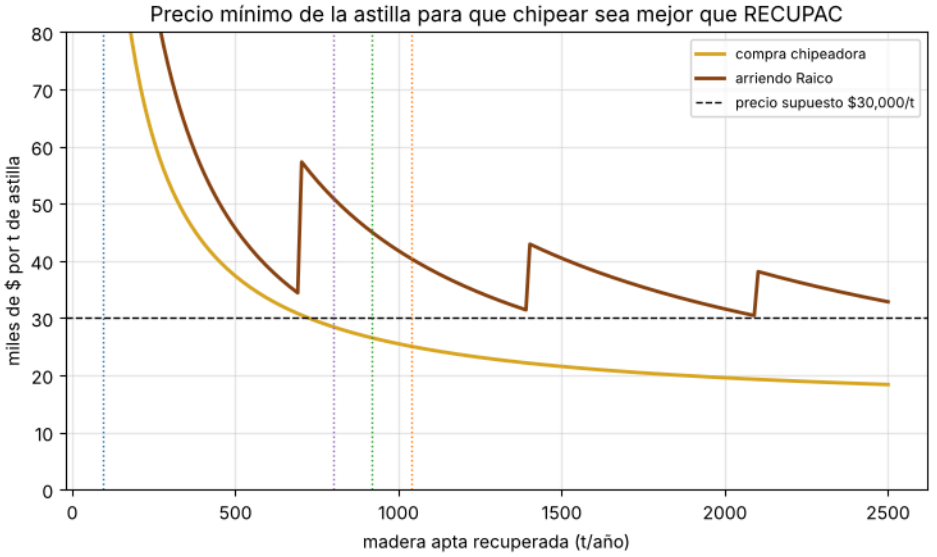
- **RECUPAC** cobra por kilo: la masa no cambia al chipear → $15.126 por t de madera (neto).
- **Flete a Teno** se cobra por m³ de **astilla**: el camión de Arauco lleva 100 m³ de astilla ≈ 100 × 0,22 = **22 t**, y cuesta 4,5 USD × 100 m³ × $981 ≈ **$441 mil por viaje ≈ $20.000/t**. Se supone que el precio del flete ya incluye la vuelta (es el precio del servicio; Arauco retira en planta).
- **Pozo:** 1 t de madera suelta ocupa 1 ÷ 0,34 ≈ 2,9 m³; la misma tonelada como astilla ocupa 1 ÷ 0,22 ≈ 4,5 m³ en el acopio (por eso se necesita una losa de acopio).
- **Venta:** $30.000 por t de astilla (Rodrigo y Pedro; unidad t, por confirmar con Arauco).

Para cada volumen anual de madera apta se calcula el costo neto anual de cada opción, con el CAPEX anualizado a 10 años al 9,25% (chipeadora $31 MM y losa 600 UF). Se marcan los volúmenes del CBO y de cada planta.

In [21]:
```
r_an = 0.0925; crf = r_an / (1 - (1 + r_an) ** -SIM['horizonte'])     # factor de recuperación del capital (WACC del §8.1)
pc, pa = MADERA['chipeo']['compra'], MADERA['chipeo']['arriendo']
energia_t = pc['factor_horas'] / pc['cap_th'] * pc['kw'] * pc['carga'] * COSTOS['precio_kwh']
losa = EXTRA_PLANTA['losa_astilla_uf'] * COSTOS['uf']
semana = pa['uf_semana'] * COSTOS['uf']
def neto_madera_apta(t, modo):
    # $/año (positivo = deja plata) para t toneladas de madera apta al año
    t = np.asarray(t, float)
    if modo == 'recupac': return -t * MERCADO['costo_recupac'] * 1000
    venta = t * (MADERA['precio_astilla'] - MADERA['flete_astilla'])
    if modo == 'compra':
        return venta - t * energia_t - pc['capex_clp'] * pc['mantencion'] - (pc['capex_clp'] + losa) * crf
    return venta - np.maximum(np.ceil(t / pa['t_semana']), (t > 0) * 1.0) * semana - losa * crf   # arriendo: semanas de 700 t, al menos 1 por año
tt_ = np.linspace(0, 2_500, 501)
fig, ax = plt.subplots(1, 2, figsize=(15, 4.6))
for modo, col, lab in [('recupac', '#7f7f7f', 'RECUPAC (hoy)'), ('compra', '#DAA520', 'Astilla — compra chipeadora'), ('arriendo', '#8B4513', 'Astilla — arriendo Raico (semanas
    ax[0].plot(tt_, neto_madera_apta(tt_, modo) / 1e6, color=col, label=lab, lw=2)
vol = {k: R_[k].madera_t.mean() * MADERA['frac_apta'] for k in ['CBO'] + [sc['nombre'] for sc in PLANTAS.values()]}
y0, y1 = ax[0].get_ylim()
for k, v in vol.items():
    ax[0].axvline(v, color=COLOR[k], ls=':', lw=1); ax[0].annotate(k.replace('Planta ', ''), (v, y0 + 0.03 * (y1 - y0)), rotation=90, fontsize=8, color=COLOR[k], ha='right', va=
ax[0].axhline(0, color='k', lw=.8); ax[0].set_xlabel('madera apta recuperada (t/año)'); ax[0].set_ylabel('$MM/año'); ax[0].legend(fontsize=8)
```

```
ax[0].set_title('Resultado anual de la madera apta según la opción (con CAPEX anualizado)')
# precio de la astilla que hace igual la compra y RECUPAC, según el volumen
tv = np.linspace(100, 2_500, 200)
p_eq = MADERA['flete_astilla'] + energia_t - MERCADO['costo_recupac'] * 1000 + (pc['capex_clp'] * pc['mantencion'] + (pc['capex_clp'] + losa) * crf) / tv
ax[1].plot(tv, p_eq / 1e3, color='#DAA520', lw=2, label='compra chipeadora')
p_eq_a = MADERA['flete_astilla'] - MERCADO['costo_recupac'] * 1000 + (np.maximum(np.ceil(tv / pa['t_semana']), 1) * semana + losa * crf) / tv
ax[1].plot(tv, p_eq_a / 1e3, color='#8B4513', lw=2, label='arriendo Raico')
ax[1].axhline(MADERA['precio_astilla'] / 1e3, color='k', ls='--', lw=1, label=f"precio supuesto ${MADERA['precio_astilla']:,.0f}/t")
for k, v in vol.items(): ax[1].axvline(v, color=COLOR[k], ls=':', lw=1)
ax[1].set_ylim(0, 80); ax[1].set_xlabel('madera apta recuperada (t/año)'); ax[1].set_ylabel('miles de $ por t de astilla'); ax[1].legend(fontsize=8)
ax[1].set_title('Precio mínimo de la astilla para que chipear sea mejor que RECUPAC')
plt.tight_layout(); plt.show()
filas = []
for k, v in vol.items():
    res = {m: neto_madera_apta(v, m) / 1e6 for m in ('recupac', 'compra', 'arriendo')}
    filas.append({'escenario': k, 'madera apta (t/año)': v, 'astilla (m3/año)': v / MADERA['densidad_astilla'], 'camiones/año': v / MADERA['densidad_astilla'] / MADERA['camion_m
                  'RECUPAC ($MM/año)': res['recupac'], 'compra ($MM/año)': res['compra'], 'arriendo ($MM/año)': res['arriendo'],
                  'mejor opción': max(res, key=res.get), 'ganancia vs RECUPAC ($MM/año)': max(res.values()) - res['recupac']})
t_chip = pd.DataFrame(filas).set_index('escenario')
display(t_chip.style.format({c: '{:,.1f}' for c in t_chip.columns if c != 'mejor opción'} | {'madera apta (t/año)': '{:,.0f}', 'astilla (m3/año)': '{:,.0f}'}))
print(f"Por t de madera apta: astilla ${MADERA['precio_astilla']:,.0f} − flete ${MADERA['flete_astilla']:,.0f} − energía de chipeo ${energia_t:,.0f} = ${MADERA['precio_astilla']
      f"contra pagar ${MERCADO['costo_recupac']*1000:,.0f} a RECUPAC → diferencia ${MADERA['precio_astilla'] - MADERA['flete_astilla'] - energia_t + MERCADO['costo_recupac']*100
      f"(mantención ${pc['capex_clp']*pc['mantencion']/1e6:,.1f} MM/año + CAPEX anualizado ${(pc['capex_clp']+losa)*crf/1e6:,.1f} MM/año).")
print(f"Arriendo Raico: 400 UF por semana = ${semana/1e6:,.1f} MM → ${semana/pa['t_semana']:,.0f}/t si se llena la semana (700 t).")
```





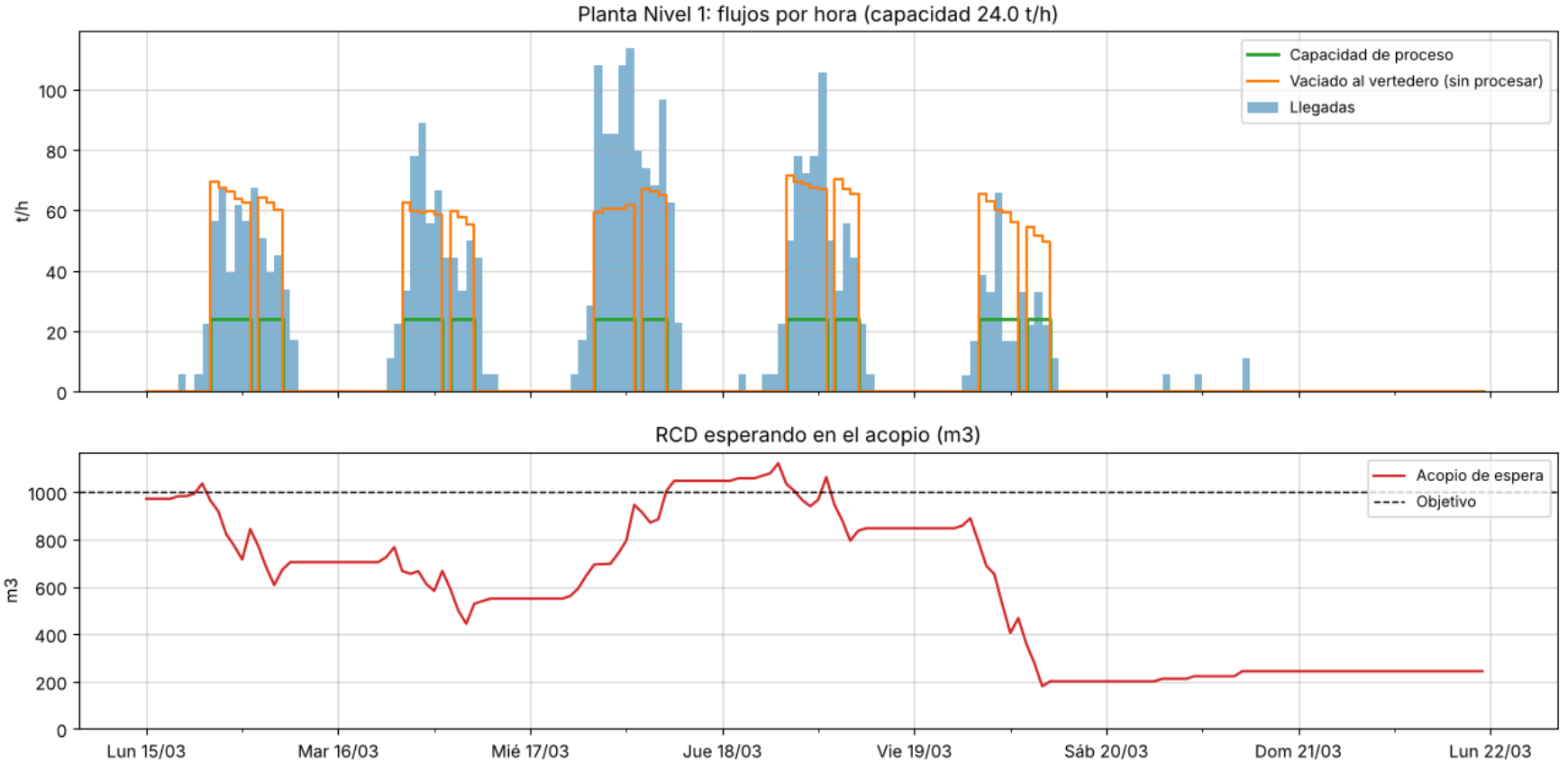
escenario	madera apta (t/año)	astilla (m3/año)	camiones/año	RECUPAC ($MM/año)	compra ($MM/año)	arriendo ($MM/año)	mejor opción	ganancia vs RECUPAC ($MM/año)
CBO	94	427	4.3	-1.4	-11.7	-19.4	recupac	0.0
Planta Nivel 1	800	3,635	36.3	-12.1	-10.9	-28.9	compra	1.2
Planta FT620C + PS123	917	4,166	41.7	-13.9	-10.7	-27.7	compra	3.2
Planta KS514	1,040	4,726	47.3	-15.7	-10.6	-26.5	compra	5.2

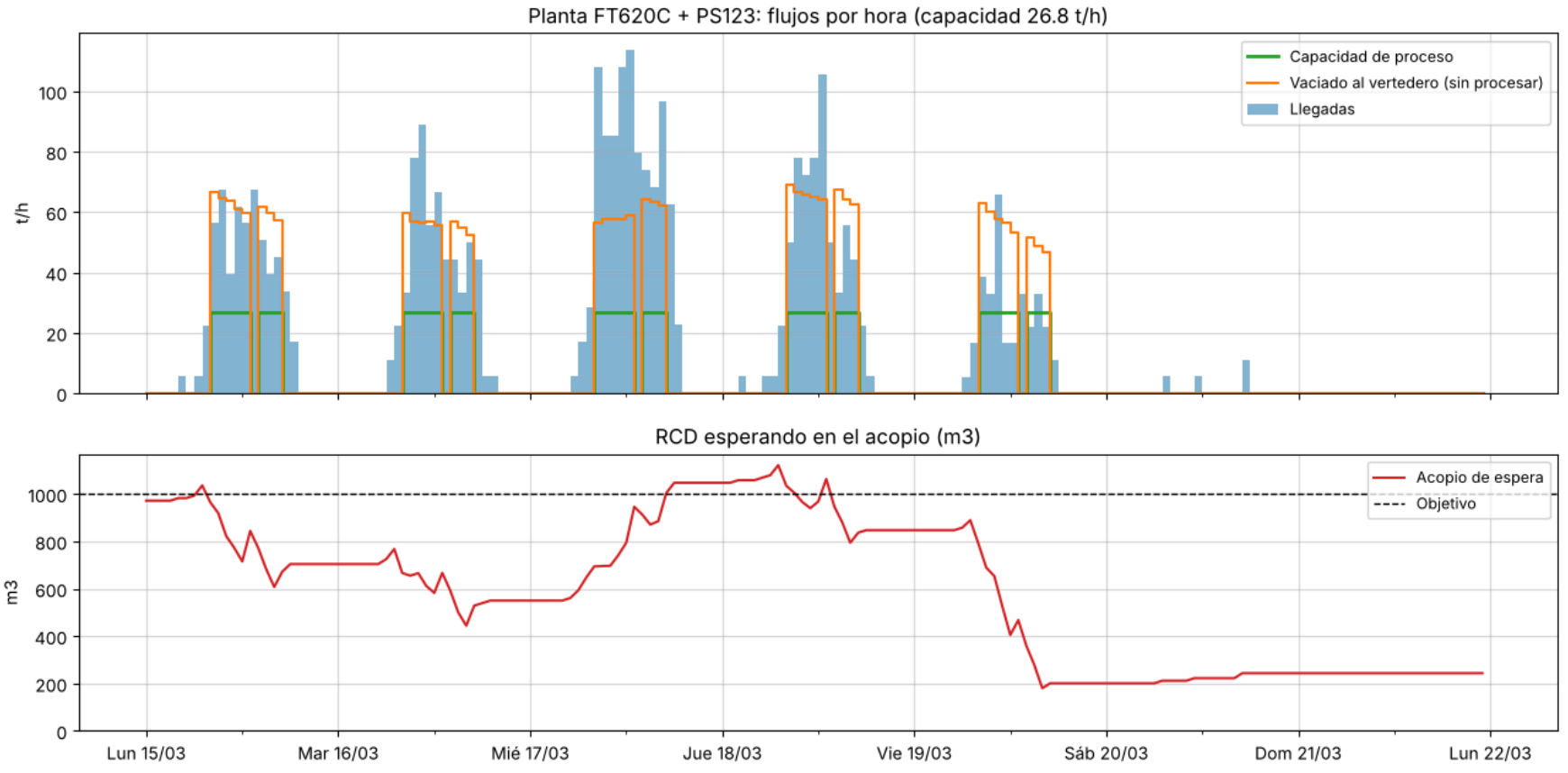
```
Por t de madera apta: astilla $30,000 − flete $20,066 − energía de chipeo $8,672 = $1,262, contra pagar $15,126 a RECUPAC → diferencia $16,388/t antes de los costos fijos (manten
ción $3.1 MM/año + CAPEX anualizado $8.8 MM/año).
Arriendo Raico: 400 UF por semana = $16.5 MM → $23,519/t si se llena la semana (700 t).
```

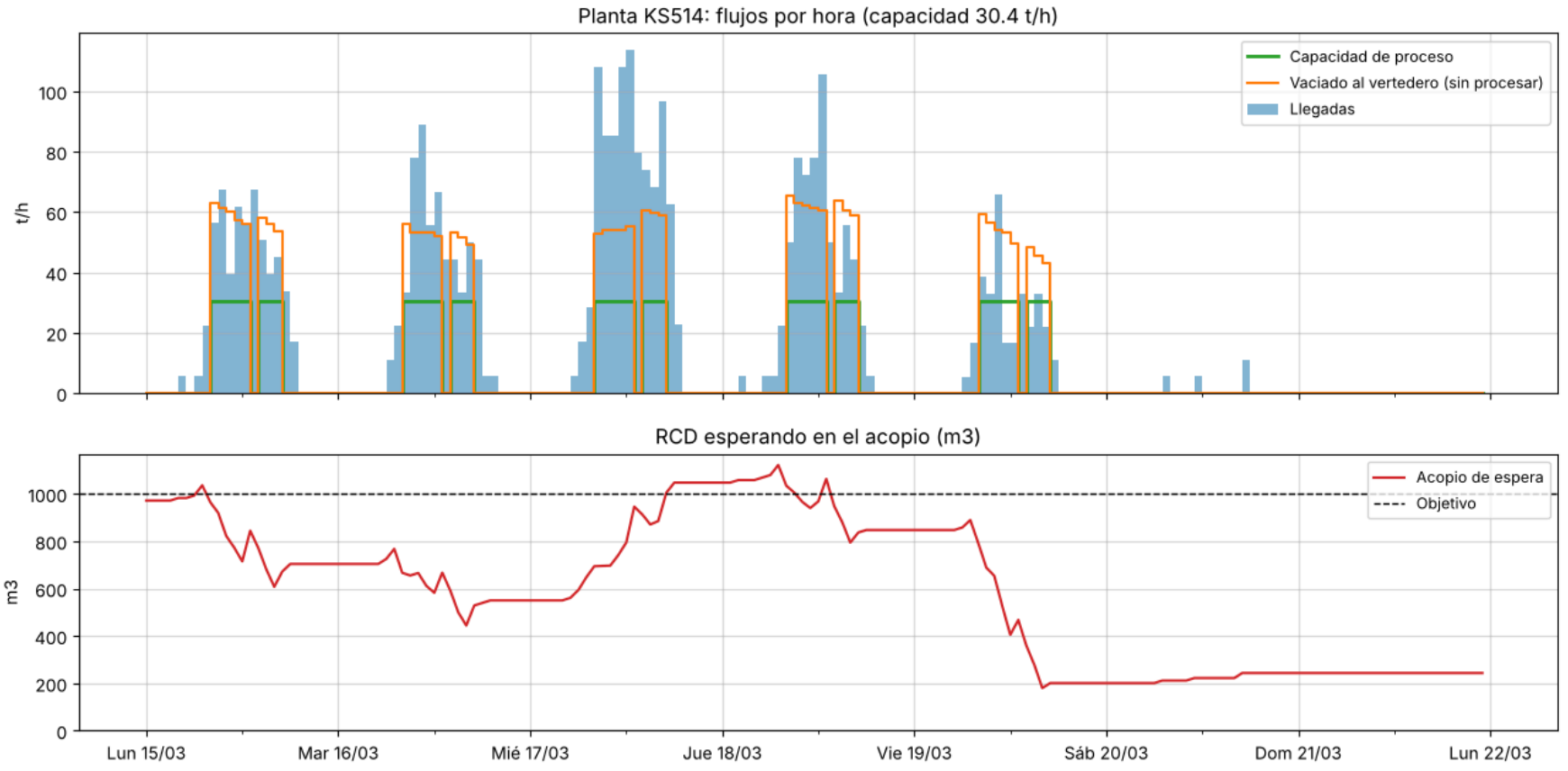
6.3 Plantas: una semana típica

Arriba: t/h que llegan (barras), capacidad de la línea (verde, solo en horas de turno) y lo que se vacía del acopio al vertedero sin procesar (naranjo, solo en horario laboral). **Abajo:** m³ en el acopio de espera (el montón frente a la planta) y el objetivo que buscan mantener los trabajadores. El montón sube cuando llega más de lo que se procesa y vacía, se queda quieto de noche y el fin de semana, y vuelve al objetivo durante el día.

In [22]:
```python
import matplotlib.dates as mdates
from matplotlib.ticker import FuncFormatter
t_idx = pd.date_range(f"{SIM['anio_inicio']}-01-01", periods=len(cr['t_h'].ravel()), freq='h')
d0 = 70 + (7 - t_idx[70*24].dayofweek) % 7                # primer lunes desde el día 70 (mediados de marzo)
sem = slice(d0*24, (d0+7)*24); horas_idx = t_idx[sem]
fmt_dia = FuncFormatter(lambda x, _: (lambda d: f"{DIAS[d.weekday()]} {d.day:02d}/{d.month:02d}")(mdates.num2date(x)))
for k, sc in PLANTAS.items():
    o = ESC[sc['nombre']]; p = sc['acopio_espera']
    cap_h = (horas_operacion(cr['lleg'], sc) * o.attrs['cap']).ravel()[sem]
    fig, ax = plt.subplots(2, 1, figsize=(13, 6.5), sharex=True, gridspec_kw={'height_ratios': [1.3, 1]})
    ax[0].bar(horas_idx, cr['t_h'].ravel()[sem], width=1/24, align='edge', color='tab:blue', alpha=.55, label='Llegadas')
    ax[0].step(horas_idx, cap_h, where='post', color='tab:green', lw=2, label='Capacidad de proceso')
    ax[0].step(horas_idx, o.attrs['vaciado_h'][sem], where='post', color='tab:orange', lw=1.5, label='Vaciado al vertedero (sin procesar)')
    ax[0].set_ylabel('t/h'); ax[0].legend(loc='upper right', fontsize=9); ax[0].set_title(f"{sc['nombre']}: flujos por hora (capacidad {o.attrs['cap']:.1f} t/h)")
    ax[1].plot(horas_idx, o.attrs['acopio_h'][sem], color='tab:red', label='Acopio de espera')
    if p['modo'] == 'control': ax[1].axhline(p['objetivo_m3'], ls='--', color='k', lw=1, label='Objetivo')
    ax[1].set_ylabel('m3'); ax[1].set_ylim(bottom=0); ax[1].legend(loc='upper right', fontsize=9); ax[1].set_title('RCD esperando en el acopio (m3)')
    for a_ in ax:
        a_.xaxis.set_major_locator(mdates.DayLocator()); a_.xaxis.set_major_formatter(fmt_dia)
        a_.xaxis.set_minor_locator(mdates.HourLocator(byhour=[12])); a_.grid(True, which='major', alpha=.5)
    plt.tight_layout(); plt.show()
```


Planta Nivel 1: flujos por hora (capacidad 24.0 t/h)
Capacidad de proceso
Vaciado al vertedero (sin procesar)
Llegadas
t/h
0
20
40
60
80
100
RCD esperando en el acopio (m3)
Acopio de espera
Objetivo
m3
0
200
400
600
800
1000
Lun 15/03
Mar 16/03
Mié 17/03
Jue 18/03
Vie 19/03
Sáb 20/03
Dom 21/03
Lun 22/03


Planta FT620C + PS123: flujos por hora (capacidad 26.8 t/h)
t/h
0
20
40
60
80
100
Capacidad de proceso
Vaciado al vertedero (sin procesar)
Llegadas
RCD esperando en el acopio (m3)
m3
0
200
400
600
800
1000
Acopio de espera
Objetivo
Lun 15/03
Mar 16/03
Mié 17/03
Jue 18/03
Vie 19/03
Sáb 20/03
Dom 21/03
Lun 22/03



6.4 Acopio de espera a lo largo del horizonte: ¿por qué un vaciado que reacciona?

Se compara la **tasa fija** (`kp = 0` , alternativa simple) con el **vaciado que reacciona** al tamaño del montón (`kp > 0`).

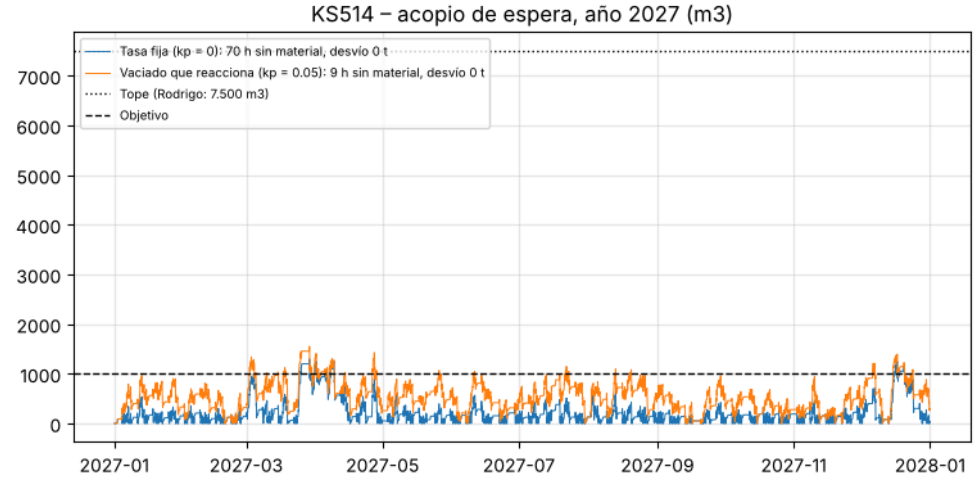
Con tasa fija, el balance cuadra solo en el promedio de los 10 años. Como las llegadas crecen ~2,8% al año, en los primeros años se vacía de más (el montón queda casi vacío y la planta a veces se queda sin material) y en los últimos se vacía de menos (el montón crece hasta el tope y los camiones se desvían).

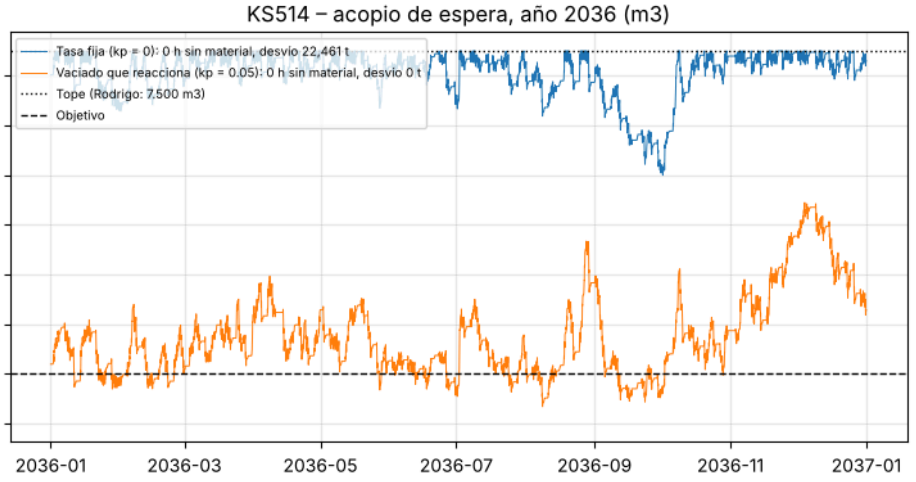
Con reacción, el montón vuelve al objetivo en cualquier año. La tabla de abajo muestra que el valor del objetivo casi no cambia lo procesado, siempre que no sea tan chico que la planta se quede sin material al partir el turno.

Restricción de Rodrigo: el acopio temporal no puede pasar de 7.500 m³, por espacio, permisos y riesgo (`tope_m3`).

In [23]:
```
sc0 = PLANTAS['KS514']
fig, ax = plt.subplots(1, 2, figsize=(15, 4), sharey=True)
for etiqueta, kp, col in [('Tasa fija (kp = 0)', 0.0, 'tab:blue'), (f"Vaciado que reacciona (kp = {sc0['acopio_espera']['kp']})", sc0['acopio_espera']['kp'], 'tab:orange')]:
    o_ = caso_planta(cr, dict(sc0, acopio_espera=dict(sc0['acopio_espera'], kp=kp)))
    for a_, anio in zip(ax, (SIM['anio_inicio'], SIM['anio_inicio'] + SIM['horizonte'] - 1)):
        m_ = t_idx.year == anio; y_ = o_[o_.anio == anio]
        a_.plot(t_idx[m_], o_.attrs['acopio_h'][m_], lw=.7, color=col,
```

```
                label=f"{etiqueta}: {y_.horas_sin_material.sum():,.0f} h sin material, desvío {y_.desvio_tope_t.sum():,.0f} t")
for a_, anio in zip(ax, (SIM['anio_inicio'], SIM['anio_inicio'] + SIM['horizonte'] - 1)):
    a_.axhline(sc0['acopio_espera']['tope_m3'], color='k', ls=':', lw=1, label='Tope (Rodrigo: 7.500 m3)'); a_.axhline(sc0['acopio_espera']['objetivo_m3'], color='k', ls='--', l
    a_.set_title(f'KS514 — acopio de espera, año {anio} (m3)'); a_.legend(fontsize=7, loc='upper left')
plt.tight_layout(); plt.show()
filas = []
for obj in (250, 500, 1000, 2000, 4000):
    for kp in (0.02, 0.05, 0.2):
        o_ = caso_planta(cr, dict(sc0, acopio_espera=dict(sc0['acopio_espera'], objetivo_m3=obj, kp=kp)))
        filas.append({'objetivo (m3)': obj, 'kp': kp, '% procesado': o_.procesado_t.sum()/o_.entrada_t.sum(),
                      'horas sin material/año': o_.horas_sin_material.sum()/SIM['horizonte'], 'acopio máx (m3)': o_.attrs['acopio_h'].max(),
                      'desvío por tope (t/año)': o_.desvio_tope_t.sum()/SIM['horizonte']})
display(pd.DataFrame(filas).style.format({'% procesado': '{:.1%}', 'horas sin material/año': '{:,.0f}', 'acopio máx (m3)': '{:,.0f}', 'desvío por tope (t/año)': '{:,.0f}'}))
```





	objetivo (m3)	kp	% procesado	horas sin material/año	acopio máx (m3)	desvío por tope (t/año)
0	250	0.020000	32.7%	24	4,661	0
1	250	0.050000	32.7%	26	4,257	0
2	250	0.200000	32.8%	17	3,988	0
3	500	0.020000	32.8%	20	4,827	0
4	500	0.050000	32.8%	15	4,474	0
5	500	0.200000	32.8%	1	4,237	0
6	1000	0.020000	32.8%	12	5,327	0
7	1000	0.050000	32.8%	2	4,963	0
8	1000	0.200000	32.8%	0	4,737	0
9	2000	0.020000	32.8%	1	6,327	0
10	2000	0.050000	32.8%	0	5,963	0
11	2000	0.200000	32.8%	0	5,737	0
12	4000	0.020000	32.8%	0	7,500	93
13	4000	0.050000	32.8%	0	7,470	35
14	4000	0.200000	32.8%	0	7,462	20

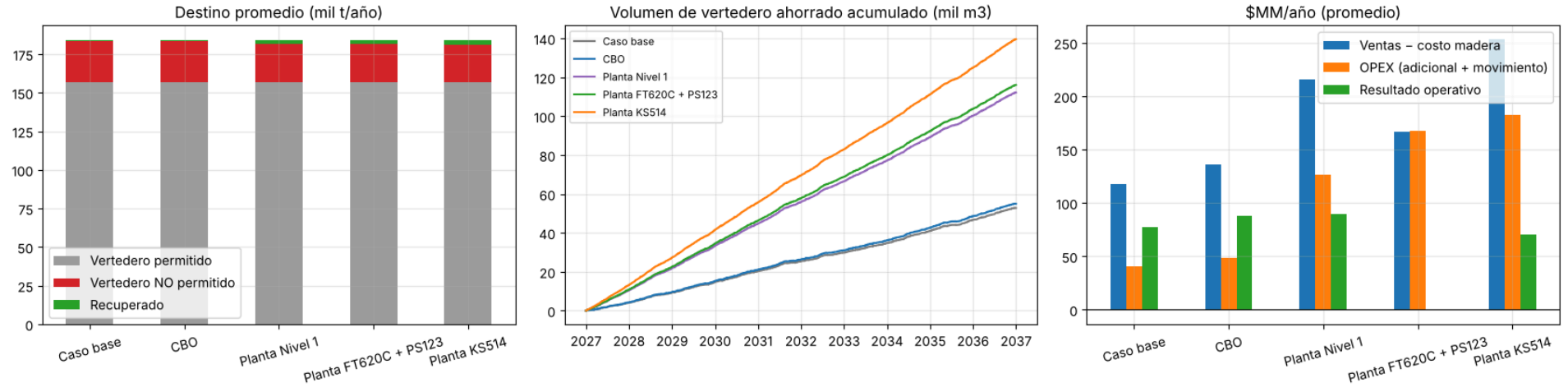
6.5 Comparación de todos los escenarios contra el caso base

Primero, la foto de cada escenario (promedio anual en 10 años). Después, la diferencia de cada uno contra el caso base, con:

- **Payback simple:** CAPEX ÷ Δ resultado operativo promedio. No descuenta ni considera impuestos.
- **Años de pozo ganados:** m³ que se dejaron de disponer en 10 años ÷ m³ que se disponen por año al final del horizonte. Rodrigo cree que es el principal beneficio.
- **Valor del m³ para recuperar el CAPEX en 10 años:** cuánto tendría que valer cada m³ de pozo ahorrado, sin descontar. Rodrigo: afuera cobran $3.500-4.000/m³.
- **Costo por t procesada:** (OPEX adicional + CAPEX ÷ 10) ÷ t procesadas. Se compara con lo que cuesta hoy disponer 1 t (movimiento $/t + valor del espacio, este último pendiente).

In [24]:
```
comp = pd.concat({k: r_.mean() for k, r_ in R_.items()}, axis=1).T
cols = ['entrada_t','procesado_t','pct_procesado','recuperado_t','pct_recuperado','fierro_t','iman_t','madera_t','astilla_t','vert_noperm_t','vertedero_m3','neto_clp','opex_clp'
display(comp[cols].style.format({'pct_procesado':'{:.1%}', 'pct_recuperado':'{:.2%}', **{c:'{:,.0f}' for c in cols if c not in ('pct_procesado','pct_recuperado')}}))
fig, ax = plt.subplots(1, 3, figsize=(17, 4.4))
dest = pd.DataFrame({k: [o.vert_permitido_t.sum(), o.vert_noperm_t.sum(), o.recuperado_t.sum()] for k, o in ESC.items()}, index=['Vertedero permitido','Vertedero NO permitido','
dest.plot(kind='bar', stacked=True, ax=ax[0], color=['#9e9e9e','#d62728','#2ca02c'], rot=15); ax[0].set_title('Destino promedio (mil t/año)')
for k, o in ESC.items(): ax[1].plot(o.fecha, o.acum_ahorro_vertedero_m3/1e3, label=k, color=COLOR[k])
ax[1].set_title('Volumen de vertedero ahorrado acumulado (mil m3)'); ax[1].legend(fontsize=8)
(comp[['neto_clp','opex_clp','resultado_clp']]/1e6).rename(columns={'neto_clp':'Ventas − costo madera','opex_clp':'OPEX (adicional + movimiento)','resultado_clp':'Resultado oper
ax[2].axhline(0, color='k', lw=.8); ax[2].set_title('$MM/año (promedio)')
plt.tight_layout(); plt.show()
```

	entrada_t	procesado_t	pct_procesado	recuperado_t	pct_recuperado	fierro_t	iman_t	madera_t	astilla_t	vert_noperm_t	vertedero_m3	neto_clp	opex_clp	resultado_clp
Caso base	184,253	0	0.0%	731	0.40%	580	0	134	0	26,470	345,982	118,049,622	40,786,812	77,262,810
CBO	184,253	0	0.0%	819	0.45%	669	88	134	94	26,382	345,766	136,352,028	48,229,392	88,122,636
Planta Nivel 1	184,253	47,799	26.1%	2,426	1.33%	893	78	1,142	800	24,725	339,376	215,874,585	126,338,466	89,536,119
Planta FT620C + PS123	184,253	53,322	29.2%	2,351	1.29%	620	75	1,309	917	24,815	339,186	166,359,114	167,263,430	-904,316
Planta KS514	184,253	60,486	33.1%	3,006	1.64%	1,043	71	1,485	1,040	24,179	337,082	253,460,229	182,699,033	70,761,196



In [25]:
```
def indicadores(nom, ref):
    r_, r0 = R_[nom], R_[ref]; d_ = delta(r_, r0)
    cx = sum(capex(SC_DE[nom]).values()) - sum(capex(SC_DE[ref]).values())
    dr = d_['Δ resultado operativo ($MM)'].mean() * 1e6; dm3 = d_['Δ m3 ahorrados'].mean()
    proc = r_.procesado_t.mean()
    return {'comparación': f'{nom} vs {ref}', 'CAPEX incremental ($MM)': cx/1e6, 'Δ resultado operativo ($MM/año)': dr/1e6,
            'payback simple (años)': cx / dr if dr > 0 else np.nan, 'Δ % recuperado (pp)': (r_.pct_recuperado - r0.pct_recuperado).mean()*100,
            'Δ m3 ahorrados/año': dm3, 'años de pozo ganados': anios_pozo(r_, r0),
```

```
            'valor m3 para recuperar CAPEX en 10 años ($/m3)': max(cx - dr * SIM['horizonte'], 0) / (dm3 * SIM['horizonte']) if dm3 > 0 else np.nan,
            'costo por t procesada ($/t)': ((r_.opex_clp - r0.opex_clp).mean() + cx / SIM['horizonte']) / proc if proc > 0 else np.nan}
FMT_IND = {'Δ m3 ahorrados/año': '{:,.0f}', 'valor m3 para recuperar CAPEX en 10 años ($/m3)': '{:,.0f}', 'costo por t procesada ($/t)': '{:,.0f}',
           'años de pozo ganados': '{:.2f}', 'Δ % recuperado (pp)': '{:.2f}'}
inc_base = {f'{k} vs Caso base': delta(R_[k], R_['Caso base']) for k in ESC if k != 'Caso base'}
ind_base = pd.DataFrame([indicadores(k, 'Caso base') for k in ESC if k != 'Caso base']).set_index('comparación')
display(ind_base.style.format('{:,.1f}').format(FMT_IND))
print(f"Costo actual de mover 1 t al pozo: ${costo_mov_t():,.0f}/t (+ espacio de pozo: ${POZO['valor_m3']:,.0f}/m3 afuera cuando el pozo se llene en {POZO['anio_lleno']})")
for sc in PLANTAS.values():
    o = ESC[sc['nombre']]
    print(f"{sc['nombre']}: {sc['kw']} kW → {o.opex_energia_clp.sum() / COSTOS['precio_kwh'] / o.procesado_t.sum():.2f} kWh/t procesada (Ferronato, Excel de Pedro: 1,45-4,52)")
for k, d_ in inc_base.items():
    print(k); display(d_.round(1))
```

comparación	CAPEX incremental ($MM)	Δ resultado operativo ($MM/año)	payback simple (años)	Δ % recuperado (pp)	Δ m3 ahorrados/año	años de pozo ganados	valor m3 para recuperar CAPEX en 10 años ($/m3)	costo por t procesada ($/t)
CBO vs Caso base	73.327458	10.859825	6.752177	0.05	216	0.01	0	nan
Planta Nivel 1 vs Caso base	407.046863	12.273309	33.165210	0.93	5,936	0.17	4,790	2,641
Planta FT620C + PS123 vs Caso base	1014.928906	-78.167127	nan	0.89	6,326	0.17	28,401	4,275
Planta KS514 vs Caso base	1549.988656	-6.501614	nan	1.24	8,679	0.23	18,608	4,909

```
Costo actual de mover 1 t al pozo: $222/t (+ espacio de pozo: $4,200/m3 afuera cuando el pozo se llene en 2034)
Planta Nivel 1: 25 kW → 0.83 kWh/t procesada (Ferronato, Excel de Pedro: 1,45-4,52)
Planta FT620C + PS123: 46 kW → 1.37 kWh/t procesada (Ferronato, Excel de Pedro: 1,45-4,52)
Planta KS514: 37 kW → 0.97 kWh/t procesada (Ferronato, Excel de Pedro: 1,45-4,52)
CBO vs Caso base
```

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	76.10	76.10	0.00	93.20	-75.90	186.10	15.10	1.50	0.00	0.00	16.60	7.50	-0.00	0.00	9.10
2028	76.50	76.50	0.00	101.30	-76.60	187.20	15.20	1.70	-0.00	0.00	16.80	7.40	-0.00	0.00	9.40
2029	78.10	78.10	0.00	86.70	-78.10	191.10	16.00	1.40	0.00	0.00	17.40	7.40	-0.00	0.00	10.00
2030	83.70	83.70	0.00	102.70	-83.80	204.80	17.20	1.70	0.00	0.00	18.90	7.40	-0.00	0.00	11.50
2031	87.00	87.00	0.00	87.70	-87.00	212.70	16.90	1.40	0.00	0.00	18.30	7.50	-0.00	0.00	10.90
2032	90.30	90.30	0.00	84.60	-90.40	220.80	17.40	1.40	0.00	0.00	18.80	7.50	-0.00	0.00	11.30
2033	90.90	90.90	0.00	107.50	-90.90	222.30	17.20	1.80	-0.00	0.00	19.00	7.50	-0.00	0.00	11.50
2034	95.00	95.00	0.00	80.00	-94.90	232.50	16.30	1.30	0.00	0.00	17.60	7.40	-0.00	0.00	10.20
2035	101.30	101.30	0.00	108.60	-101.30	247.80	17.10	1.80	-0.00	0.00	18.80	7.40	-0.00	0.00	11.50
2036	104.10	104.10	0.00	86.30	-104.10	254.50	19.30	1.40	0.00	0.00	20.70	7.50	-0.00	0.00	13.20

```
Planta Nivel 1 vs Caso base
```

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	325.60	65.00	1,017.60	805.60	-1,721.90	6,391.20	64.80	-2.20	40.10	0.00	102.70	85.90	-0.40	30.60	16.80
2028	288.60	66.30	992.60	796.10	-1,657.50	6,157.40	57.10	-2.00	42.50	0.00	97.70	85.50	-0.40	30.20	12.10
2029	286.00	67.90	1,004.80	790.00	-1,742.80	6,080.00	58.50	-2.30	40.70	0.00	97.00	84.90	-0.50	30.00	12.10
2030	271.40	75.50	991.30	796.60	-1,589.70	5,817.10	55.60	-1.90	38.80	0.00	92.40	85.50	-0.30	30.20	6.90
2031	341.20	75.10	1,021.90	803.00	-1,796.30	6,092.20	66.20	-2.30	34.40	0.00	98.40	85.70	-0.50	30.50	12.70
2032	352.30	79.70	1,040.10	812.70	-1,755.60	6,147.60	68.10	-2.40	43.40	0.00	109.10	86.40	-0.40	30.80	22.70
2033	291.90	81.20	997.90	806.00	-1,652.10	5,813.80	55.30	-1.90	46.20	0.00	99.60	86.00	-0.30	30.60	13.60
2034	286.00	86.10	1,019.10	793.40	-1,674.40	5,703.40	49.60	-2.40	40.10	0.00	87.20	85.30	-0.40	30.10	2.00
2035	323.60	89.90	973.70	790.20	-1,903.50	5,422.30	54.10	-1.80	37.00	0.00	89.30	84.70	-0.70	30.00	4.50
2036	358.70	92.30	1,023.90	803.00	-1,955.80	5,733.30	66.50	-2.30	40.70	0.00	104.80	85.60	-0.70	30.50	19.20

Planta FT620C + PS123 vs Caso base

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	50.90	61.80	1,185.50	923.00	-1,641.30	6,781.10	10.30	-2.80	43.50	0.00	51.00	126.90	-0.40	30.60	-75.80
2028	17.10	63.10	1,158.60	912.30	-1,583.80	6,543.40	3.20	-2.60	45.60	0.00	46.30	126.50	-0.30	30.20	-80.20
2029	16.60	64.80	1,169.70	905.50	-1,669.50	6,465.00	3.30	-2.90	44.00	0.00	44.50	125.70	-0.50	30.00	-81.30
2030	-0.20	72.30	1,157.70	913.10	-1,511.90	6,206.20	-0.20	-2.50	41.80	0.00	39.10	126.40	-0.30	30.20	-87.30
2031	67.60	72.00	1,189.60	920.40	-1,704.40	6,484.60	13.10	-2.90	37.00	0.00	47.30	126.60	-0.40	30.50	-79.30
2032	75.20	76.40	1,209.90	931.50	-1,698.80	6,544.40	14.70	-3.00	46.80	0.00	58.50	127.50	-0.40	30.80	-69.00
2033	17.10	78.00	1,166.20	923.80	-1,580.30	6,207.20	3.20	-2.50	50.00	0.00	50.70	126.90	-0.30	30.60	-76.20
2034	15.40	82.90	1,184.80	909.40	-1,601.00	6,090.70	3.00	-3.00	43.20	0.00	43.20	126.10	-0.30	30.10	-82.90
2035	54.80	87.50	1,138.70	905.70	-1,658.30	5,809.80	8.70	-2.40	39.70	0.00	46.00	125.70	-0.50	30.00	-79.70
2036	84.80	89.10	1,191.60	920.40	-1,901.40	6,125.20	15.80	-2.90	43.80	0.00	56.70	126.50	-0.70	30.50	-69.90

Planta KS514 vs Caso base

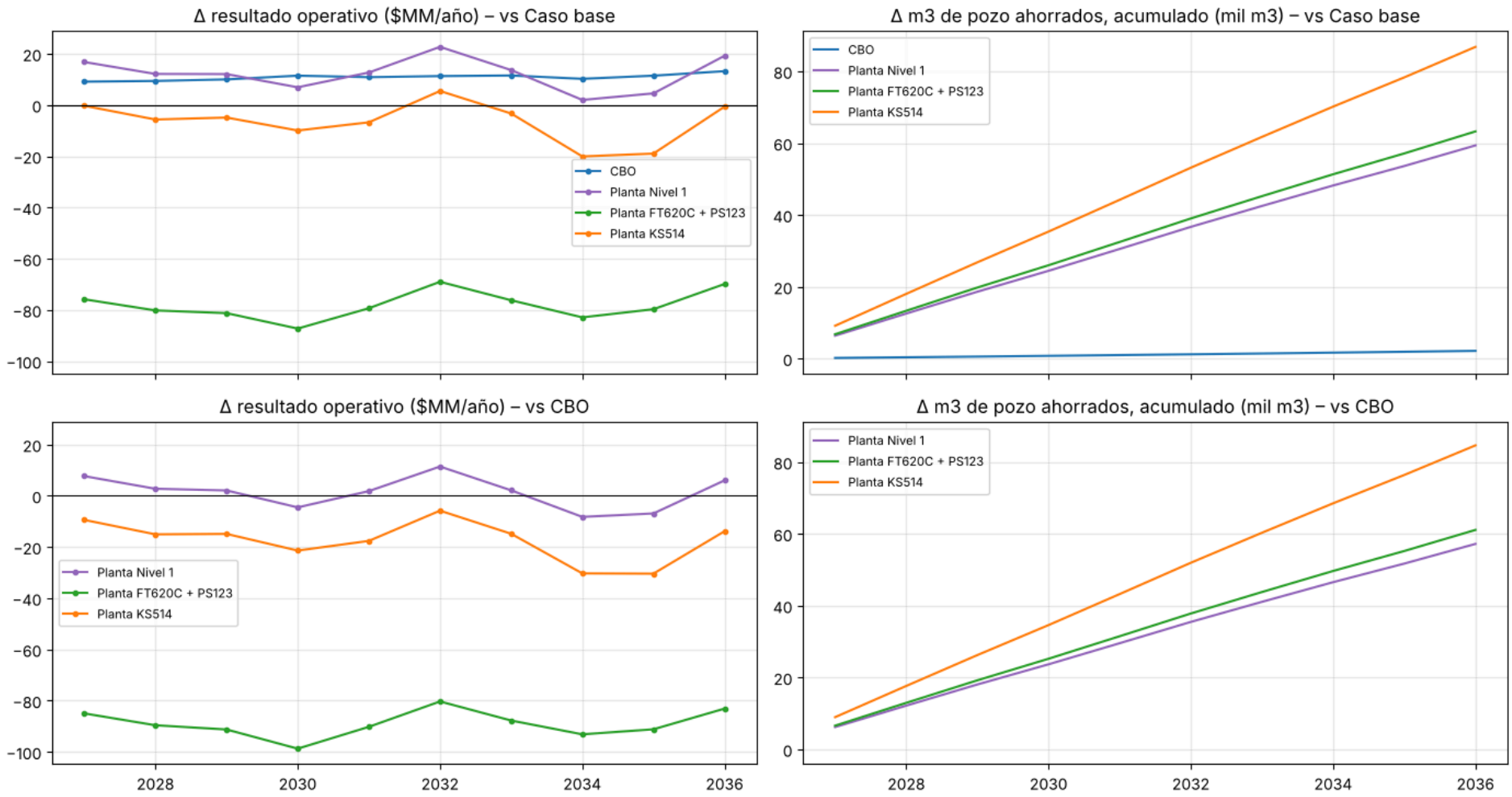
anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	475.90	57.70	1,362.20	1,046.70	-2,302.80	9,147.20	94.50	-3.50	50.80	0.00	141.90	142.20	-0.50	30.60	-0.30
2028	437.10	59.10	1,333.20	1,034.50	-2,234.50	8,882.00	86.60	-3.20	52.90	0.00	136.30	141.90	-0.50	30.20	-5.70
2029	433.80	60.70	1,343.40	1,027.10	-2,316.20	8,789.40	88.80	-3.50	50.90	0.00	136.30	141.10	-0.60	30.00	-4.90
2030	420.60	68.20	1,333.10	1,035.80	-2,162.50	8,552.10	86.30	-3.20	48.70	0.00	131.80	141.80	-0.40	30.20	-9.90
2031	491.80	68.00	1,366.40	1,044.20	-2,356.20	8,849.50	95.50	-3.60	43.30	0.00	135.20	142.00	-0.60	30.50	-6.80
2032	504.40	72.20	1,388.80	1,056.70	-2,372.70	8,937.50	97.40	-3.70	54.60	0.00	148.30	142.90	-0.50	30.80	5.50
2033	442.90	73.90	1,343.60	1,048.00	-2,241.20	8,580.50	83.90	-3.10	58.20	0.00	139.00	142.30	-0.50	30.60	-3.30
2034	434.50	78.70	1,359.40	1,031.60	-2,250.70	8,427.00	75.10	-3.70	50.00	0.00	121.40	141.50	-0.50	30.10	-20.10
2035	472.40	83.50	1,312.60	1,027.50	-2,243.30	8,137.10	79.20	-3.00	46.20	0.00	122.30	141.30	-0.50	30.00	-19.00
2036	509.40	85.30	1,368.40	1,044.20	-2,428.40	8,490.80	94.40	-3.60	50.70	0.00	141.60	142.20	-0.60	30.50	-0.60

6.6 Plantas contra el CBO

Lo que agrega cada planta por sobre las mejoras baratas (imán + chipeadora). Siguiendo a Rodrigo, la astilla de la madera que hoy ya se recupera queda en el CBO, y solo la astilla adicional que segrega la planta (y el RECUPAC que evita) se le imputa al proyecto.

In [26]:
```
inc_cbo = {f"{sc['nombre']} vs CBO": delta(R_[sc['nombre']], R_['CBO']) for sc in PLANTAS.values()}
ind_cbo = pd.DataFrame([indicadores(sc['nombre'], 'CBO') for sc in PLANTAS.values()]).set_index('comparación')
display(ind_cbo.style.format('{:,.1f}').format(FMT_IND))
fig, ax = plt.subplots(2, 2, figsize=(14, 7.5), sharex=True, sharey='col')   # misma escala arriba y abajo
for fila, (titulo, dic) in enumerate([('vs Caso base', inc_base), ('vs CBO', inc_cbo)]):
    for k, d_ in dic.items():
        nom = k.split(' vs ')[0]
        ax[fila, 0].plot(d_.index, d_['Δ resultado operativo ($MM)'], marker='.', color=COLOR[nom], label=nom)
        ax[fila, 1].plot(d_.index, d_['Δ m3 ahorrados'].cumsum()/1e3, color=COLOR[nom], label=nom)
    ax[fila, 0].axhline(0, color='k', lw=.8); ax[fila, 0].set_title(f'Δ resultado operativo ($MM/año) — {titulo}')
    ax[fila, 1].set_title(f'Δ m3 de pozo ahorrados, acumulado (mil m3) — {titulo}')
    for a_ in ax[fila]: a_.legend(fontsize=8)
plt.tight_layout(); plt.show()
for k, d_ in inc_cbo.items():
    print(k); display(d_.round(1))
```

comparación	CAPEX incremental ($MM)	Δ resultado operativo ($MM/año)	payback simple (años)	Δ % recuperado (pp)	Δ m3 ahorrados/año	años de pozo ganados	valor m3 para recuperar CAPEX en 10 años ($/m3)	costo por t procesada ($/t)
Planta Nivel 1 vs CBO	333.719405	1.413484	236.097126	0.88	5,720	0.16	5,587	2,332
Planta FT620C + PS123 vs CBO	941.601448	-89.026952	nan	0.84	6,110	0.17	29,983	3,998
Planta KS514 vs CBO	1476.661198	-17.361439	nan	1.20	8,463	0.22	19,499	4,665



Planta Nivel 1 vs CBO

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	249.50	-11.10	1,017.60	712.30	-1,646.00	6,205.10	49.70	-3.70	40.10	0.00	86.10	78.40	-0.40	30.60	7.70
2028	212.10	-10.20	992.60	694.80	-1,580.90	5,970.30	41.90	-3.60	42.50	0.00	80.80	78.10	-0.30	30.20	2.70
2029	207.90	-10.20	1,004.80	703.40	-1,664.70	5,889.00	42.50	-3.70	40.70	0.00	79.60	77.50	-0.50	30.00	2.10
2030	187.60	-8.30	991.30	693.90	-1,505.90	5,612.30	38.40	-3.60	38.80	0.00	73.50	78.10	-0.30	30.20	-4.50
2031	254.30	-11.90	1,021.90	715.30	-1,709.30	5,879.40	49.40	-3.70	34.40	0.00	80.00	78.20	-0.50	30.50	1.80
2032	262.00	-10.50	1,040.10	728.10	-1,665.20	5,926.90	50.70	-3.80	43.40	0.00	90.30	78.90	-0.40	30.80	11.40
2033	201.00	-9.70	997.90	698.50	-1,561.20	5,591.50	38.10	-3.60	46.20	0.00	80.60	78.50	-0.30	30.60	2.10
2034	190.90	-8.90	1,019.10	713.30	-1,579.50	5,470.90	33.20	-3.70	40.10	0.00	69.60	77.80	-0.30	30.10	-8.20
2035	222.20	-11.40	973.70	681.60	-1,802.20	5,174.50	37.00	-3.60	37.00	0.00	70.40	77.30	-0.70	30.00	-6.90
2036	254.60	-11.70	1,023.90	716.70	-1,851.80	5,478.70	47.20	-3.70	40.70	0.00	84.10	78.10	-0.70	30.50	6.00

Planta FT620C + PS123 vs CB0

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	-25.20	-14.20	1,185.50	829.80	-1,565.30	6,595.00	-4.80	-4.30	43.50	0.00	34.40	119.40	-0.40	30.60	-85.00
2028	-59.50	-13.40	1,158.60	811.00	-1,507.30	6,356.30	-12.00	-4.20	45.60	0.00	29.40	119.00	-0.30	30.20	-89.60
2029	-61.50	-13.30	1,169.70	818.80	-1,591.40	6,274.00	-12.60	-4.30	44.00	0.00	27.10	118.30	-0.50	30.00	-91.30
2030	-83.90	-11.40	1,157.70	810.40	-1,428.20	6,001.40	-17.40	-4.20	41.80	0.00	20.20	118.90	-0.20	30.20	-98.70
2031	-19.40	-14.90	1,189.60	832.70	-1,617.40	6,271.80	-3.80	-4.30	37.00	0.00	28.90	119.10	-0.40	30.50	-90.20
2032	-15.10	-13.80	1,209.90	846.90	-1,608.40	6,323.60	-2.70	-4.40	46.80	0.00	39.70	120.00	-0.40	30.80	-80.30
2033	-73.70	-12.90	1,166.20	816.30	-1,489.40	5,984.90	-14.10	-4.30	50.00	0.00	31.70	119.40	-0.30	30.60	-87.80
2034	-79.60	-12.20	1,184.80	829.30	-1,506.10	5,858.20	-13.30	-4.30	43.20	0.00	25.50	118.70	-0.30	30.10	-93.10
2035	-46.50	-13.90	1,138.70	797.10	-1,557.00	5,562.00	-8.30	-4.20	39.70	0.00	27.20	118.40	-0.40	30.00	-91.20
2036	-19.20	-15.00	1,191.60	834.10	-1,797.30	5,870.70	-3.50	-4.40	43.80	0.00	36.00	119.10	-0.70	30.50	-83.10

Planta KS514 vs CB0

anio	Δ fierro (t)	Δ fierro imán (t)	Δ madera (t)	Δ astilla (t)	Δ NO permitido al vertedero (t)	Δ m3 ahorrados	Δ fierro ($MM)	Δ madera neta ($MM)	Δ aluminio y otros ($MM)	Δ áridos ($MM)	Δ neto materiales ($MM)	Δ OPEX ($MM)	Δ mov. al pozo ($MM)	Δ alimentación planta ($MM)	Δ resultado operativo ($MM)
2027	399.80	-18.40	1,362.20	953.50	-2,226.80	8,961.20	79.50	-5.00	50.80	0.00	125.30	134.70	-0.50	30.60	-9.40
2028	360.60	-17.50	1,333.20	933.30	-2,158.00	8,694.80	71.40	-4.90	52.90	0.00	119.40	134.50	-0.50	30.20	-15.10
2029	355.60	-17.40	1,343.40	940.40	-2,238.20	8,598.30	72.80	-4.90	50.90	0.00	118.90	133.70	-0.60	30.00	-14.90
2030	336.90	-15.50	1,333.10	933.10	-2,078.70	8,347.30	69.10	-4.90	48.70	0.00	112.90	134.30	-0.40	30.20	-21.40
2031	404.90	-19.00	1,366.40	956.50	-2,269.20	8,636.80	78.60	-5.00	43.30	0.00	116.90	134.50	-0.60	30.50	-17.60
2032	414.20	-18.00	1,388.80	972.10	-2,282.30	8,716.80	80.00	-5.10	54.60	0.00	129.50	135.40	-0.50	30.80	-5.80
2033	352.00	-17.00	1,343.60	940.50	-2,150.30	8,358.20	66.70	-4.90	58.20	0.00	120.00	134.80	-0.50	30.60	-14.80
2034	339.40	-16.30	1,359.40	951.60	-2,155.80	8,194.50	58.80	-5.00	50.00	0.00	103.80	134.10	-0.50	30.10	-30.30
2035	371.10	-17.80	1,312.60	918.80	-2,142.00	7,889.30	62.10	-4.80	46.20	0.00	103.50	133.90	-0.50	30.00	-30.40
2036	405.30	-18.80	1,368.40	957.80	-2,324.40	8,236.30	75.10	-5.00	50.70	0.00	120.90	134.70	-0.60	30.50	-13.80

Movimiento de material: ¿cuánto cambia con la planta?

En el caso base todo lo que llega se mueve una vez al pozo. Con planta, lo procesado se mueve dos veces (a la tolva y el rechazo al pozo), y lo recuperado ya no se mueve. La tabla compara tres formas de alimentar la planta: por tonelada (v1.0, usa la retro que ya existe), cargador propio con operador (base v1.1) y cargador arrendado con operador. El CAPEX del cargador propio ($100 MM) no aparece aquí: entra en el §8.

In [27]:
```
filas = []
for k, sc in PLANTAS.items():
    for modo in ('por_tonelada', 'cargador_propio', 'arriendo'):
        a = resumen(caso_planta(cr, dict(sc, alimentacion=modo))).mean()
        filas.append({'planta': sc['nombre'], 'alimentación': modo, 't al pozo/año': a.vertedero_t, 'mov. al pozo ($MM)': a.opex_mov_clp/1e6,
                      'alimentación ($MM)': a.opex_alim_clp/1e6, 'Δ resultado vs Caso base ($MM/año)': (a.resultado_clp - R_['Caso base'].resultado_clp.mean())/1e6,
                      'Δ resultado vs CBO ($MM/año)': (a.resultado_clp - R_['CBO'].resultado_clp.mean())/1e6})
print(f"Costo de mover 1 t al pozo: ${costo_mov_t():,.0f}/t")
display(pd.DataFrame(filas).style.format({c: '{:,.1f}' for c in ['mov. al pozo ($MM)', 'alimentación ($MM)', 'Δ resultado vs Caso base ($MM/año)', 'Δ resultado vs CBO ($MM/año)'
```

```
Costo de mover 1 t al pozo: $222/t
```

	planta	alimentación	t al pozo/año	mov. al pozo ($MM)	alimentación ($MM)	Δ resultado vs Caso base ($MM/año)	Δ resultado vs CBO ($MM/año)
0	Planta Nivel 1	por_tonelada	181,470	40.3	10.6	38.0	27.2
1	Planta Nivel 1	cargador_propio	181,470	40.3	30.4	12.3	1.4
2	Planta Nivel 1	arriendo	181,470	40.3	120.0	-71.4	-82.2
3	Planta FT620C + PS123	por_tonelada	181,649	40.4	11.9	-53.7	-64.5
4	Planta FT620C + PS123	cargador_propio	181,649	40.4	30.4	-78.2	-89.0
5	Planta FT620C + PS123	arriendo	181,649	40.4	120.0	-161.8	-172.7
6	Planta KS514	por_tonelada	181,125	40.3	13.4	16.4	5.6
7	Planta KS514	cargador_propio	181,125	40.3	30.4	-6.5	-17.4
8	Planta KS514	arriendo	181,125	40.3	120.0	-90.1	-101.0

6.7 Sensibilidad rápida de las plantas: turnos, operarios y productividad (año 1)

Muestra cuándo deja de mandar la máquina y pasan a mandar los operarios, y cómo cambia el resultado contra el caso base y contra el CBO. Con 2 turnos se duplica el personal de la planta en el OPEX (`turnos`). Con 4 operarios de picking la planta usa menos gente que los 8 actuales, así que el personal adicional puede quedar negativo (se reasignan o salen trabajadores).

In [28]:
```
filas = []
b_1, c_1 = R_['Caso base'].iloc[0], R_['CBO'].iloc[0]
for k, sc in PLANTAS.items():
    for turnos in (1, 2):
        for ops in (4, 8):
            for tasa in (2.9, 4.0, 7.0):
                sc2 = dict(sc, horas=T1 + (T2 if turnos == 2 else []), turnos=turnos, operarios_picking=ops, tasa_revision_th=tasa)
                c, cm, co = capacidad_th(sc2); r_ = resumen(caso_planta(cr, sc2)).iloc[0]
                filas.append(dict(planta=sc['nombre'], turnos=turnos, operarios_por_turno=ops, t_h_operario=tasa, cap_th=c, manda='máquina' if cm <= co else 'operarios',
                                  pct_procesado=r_.pct_procesado, pct_recuperado=r_.pct_recuperado,
                                  d_resultado_vs_base_MM=(r_.resultado_clp - b_1.resultado_clp)/1e6, d_resultado_vs_CBO_MM=(r_.resultado_clp - c_1.resultado_clp)/1e6))
display(pd.DataFrame(filas).style.format({'cap_th':'{:.1f}','pct_procesado':'{:.0%}','pct_recuperado':'{:.2%}','d_resultado_vs_base_MM':'{:,.1f}','d_resultado_vs_CBO_MM':'{:,.1f
```

	planta	turnos	operarios_por_turno	t_h_operario	cap_th	manda	pct_procesado	pct_recuperado	d_resultado_vs_base_MM	d_resultado_vs_CBO_MM
0	Planta Nivel 1	1	4	2.900000	11.6	operarios	14%	0.75%	-36.2	-45.3
1	Planta Nivel 1	1	4	4.000000	16.0	operarios	20%	1.02%	1.4	-7.8
2	Planta Nivel 1	1	4	7.000000	28.0	operarios	35%	1.74%	103.6	94.5
3	Planta Nivel 1	1	8	2.900000	23.2	operarios	29%	1.45%	10.0	0.8
4	Planta Nivel 1	1	8	4.000000	32.0	operarios	40%	1.98%	84.9	75.7
5	Planta Nivel 1	1	8	7.000000	55.2	máquina	68%	3.36%	281.5	272.4
6	Planta Nivel 1	2	4	2.900000	11.6	operarios	25%	1.28%	-84.5	-93.7
7	Planta Nivel 1	2	4	4.000000	16.0	operarios	35%	1.74%	-18.8	-27.9
8	Planta Nivel 1	2	4	7.000000	28.0	operarios	61%	3.00%	160.5	151.3
9	Planta Nivel 1	2	8	2.900000	23.2	operarios	50%	2.49%	-22.1	-31.2
10	Planta Nivel 1	2	8	4.000000	32.0	operarios	69%	3.42%	109.2	100.0
11	Planta Nivel 1	2	8	7.000000	55.2	máquina	100%	4.91%	320.3	311.2
12	Planta FT620C + PS123	1	4	2.900000	26.8	máquina	33%	1.45%	-23.0	-32.2
13	Planta FT620C + PS123	1	4	4.000000	26.8	máquina	33%	1.45%	-23.0	-32.2
14	Planta FT620C + PS123	1	4	7.000000	26.8	máquina	33%	1.45%	-23.0	-32.2
15	Planta FT620C + PS123	1	8	2.900000	26.8	máquina	33%	1.45%	-75.8	-85.0
16	Planta FT620C + PS123	1	8	4.000000	26.8	máquina	33%	1.45%	-75.8	-85.0
17	Planta FT620C + PS123	1	8	7.000000	26.8	máquina	33%	1.45%	-75.8	-85.0
18	Planta FT620C + PS123	2	4	2.900000	26.8	máquina	58%	2.50%	-49.6	-58.7
19	Planta FT620C + PS123	2	4	4.000000	26.8	máquina	58%	2.50%	-49.6	-58.7
20	Planta FT620C + PS123	2	4	7.000000	26.8	máquina	58%	2.50%	-49.6	-58.7
21	Planta FT620C + PS123	2	8	2.900000	26.8	máquina	58%	2.50%	-160.5	-169.6
22	Planta FT620C + PS123	2	8	4.000000	26.8	máquina	58%	2.50%	-160.5	-169.6
23	Planta FT620C + PS123	2	8	7.000000	26.8	máquina	58%	2.50%	-160.5	-169.6
24	Planta KS514	1	4	2.900000	30.4	máquina	38%	1.86%	52.5	43.4
25	Planta KS514	1	4	4.000000	30.4	máquina	38%	1.86%	52.5	43.4
26	Planta KS514	1	4	7.000000	30.4	máquina	38%	1.86%	52.5	43.4
27	Planta KS514	1	8	2.900000	30.4	máquina	38%	1.86%	-0.3	-9.4
28	Planta KS514	1	8	4.000000	30.4	máquina	38%	1.86%	-0.3	-9.4
29	Planta KS514	1	8	7.000000	30.4	máquina	38%	1.86%	-0.3	-9.4
30	Planta KS514	2	4	2.900000	30.4	máquina	66%	3.21%	90.1	81.0
31	Planta KS514	2	4	4.000000	30.4	máquina	66%	3.21%	90.1	81.0
32	Planta KS514	2	4	7.000000	30.4	máquina	66%	3.21%	90.1	81.0
33	Planta KS514	2	8	2.900000	30.4	máquina	66%	3.21%	-20.7	-29.9
34	Planta KS514	2	8	4.000000	30.4	máquina	66%	3.21%	-20.7	-29.9
35	Planta KS514	2	8	7.000000	30.4	máquina	66%	3.21%	-20.7	-29.9

6.8 Sensibilidad a la densidad de lo recibido (pedida por Rodrigo)

Rodrigo: el contenedor de 9 m³ viene con ~3.000 kg (~0,33 t/m³) y el de 10 m³ con ~7.000 kg (~0,70 t/m³); el modelo pondera por la mezcla de contenedores de las guías (~0,525 t/m³). Las 118.457 t del RETC usan ~4,8 t por viaje, así que no son una calibración independiente.

Aquí se cambia la densidad media manteniendo los mismos viajes y m³: con más densidad llegan más toneladas, la planta (que limita por t/h) procesa una fracción menor y cambian los m³ ahorrados.

In [29]:
```
filas = []
for rho in (0.33, 0.43, round(t_viaje_medio / m3_viaje_medio, 3), 0.60, 0.70):
    cr_d = con_densidad(cr, rho)
    rb_d, rc_d = resumen(caso_base(cr_d)), resumen(caso_cbo(cr_d))
    for k, sc in PLANTAS.items():
        rp = resumen(caso_planta(cr_d, sc))
        filas.append({'densidad (t/m3)': rho, 'planta': sc['nombre'], 'entrada (t/año)': rp.entrada_t.mean(), '% procesado': rp.pct_procesado.mean(),
                      'Δ m3 ahorrados vs base': (rp.ahorro_m3 - rb_d.ahorro_m3).mean(), 'años pozo ganados vs base': anios_pozo(rp, rb_d),
                      'Δ resultado vs base ($MM/año)': (rp.resultado_clp - rb_d.resultado_clp).mean()/1e6,
                      'Δ resultado vs CBO ($MM/año)': (rp.resultado_clp - rc_d.resultado_clp).mean()/1e6})
t_dens = pd.DataFrame(filas)
display(t_dens.style.format({'densidad (t/m3)': '{:.3f}', 'entrada (t/año)': '{:,.0f}', '% procesado': '{:.1%}', 'Δ m3 ahorrados vs base': '{:,.0f}',
                             'años pozo ganados vs base': '{:.2f}', 'Δ resultado vs base ($MM/año)': '{:,.1f}', 'Δ resultado vs CBO ($MM/año)': '{:,.1f}'}))
```

	densidad (t/m3)	planta	entrada (t/año)	% procesado	Δ m3 ahorrados vs base	años pozo ganados vs base	Δ resultado vs base ($MM/año)	Δ resultado vs CBO ($MM/año)
0	0.330	Planta Nivel 1	115,890	41.5%	6,998	0.20	4.7	1.2
1	0.330	Planta FT620C + PS123	115,890	46.3%	7,387	0.21	-85.7	-89.2
2	0.330	Planta KS514	115,890	52.5%	9,738	0.28	-14.1	-17.6
3	0.430	Planta Nivel 1	151,008	31.9%	6,459	0.17	8.6	1.3
4	0.430	Planta FT620C + PS123	151,008	35.6%	6,848	0.18	-81.8	-89.1
5	0.430	Planta KS514	151,008	40.3%	9,201	0.24	-10.2	-17.5
6	0.525	Planta Nivel 1	184,370	26.1%	5,934	0.17	12.3	1.4
7	0.525	Planta FT620C + PS123	184,370	29.1%	6,324	0.17	-78.2	-89.0
8	0.525	Planta KS514	184,370	33.0%	8,677	0.23	-6.5	-17.4
9	0.600	Planta Nivel 1	210,708	22.9%	5,519	0.16	15.2	1.5
10	0.600	Planta FT620C + PS123	210,708	25.5%	5,909	0.17	-75.3	-89.0
11	0.600	Planta KS514	210,708	28.9%	8,263	0.23	-3.6	-17.3
12	0.700	Planta Nivel 1	245,826	19.6%	4,966	0.15	19.0	1.5
13	0.700	Planta FT620C + PS123	245,826	21.9%	5,357	0.16	-71.4	-88.9
14	0.700	Planta KS514	245,826	24.8%	7,712	0.22	0.3	-17.2

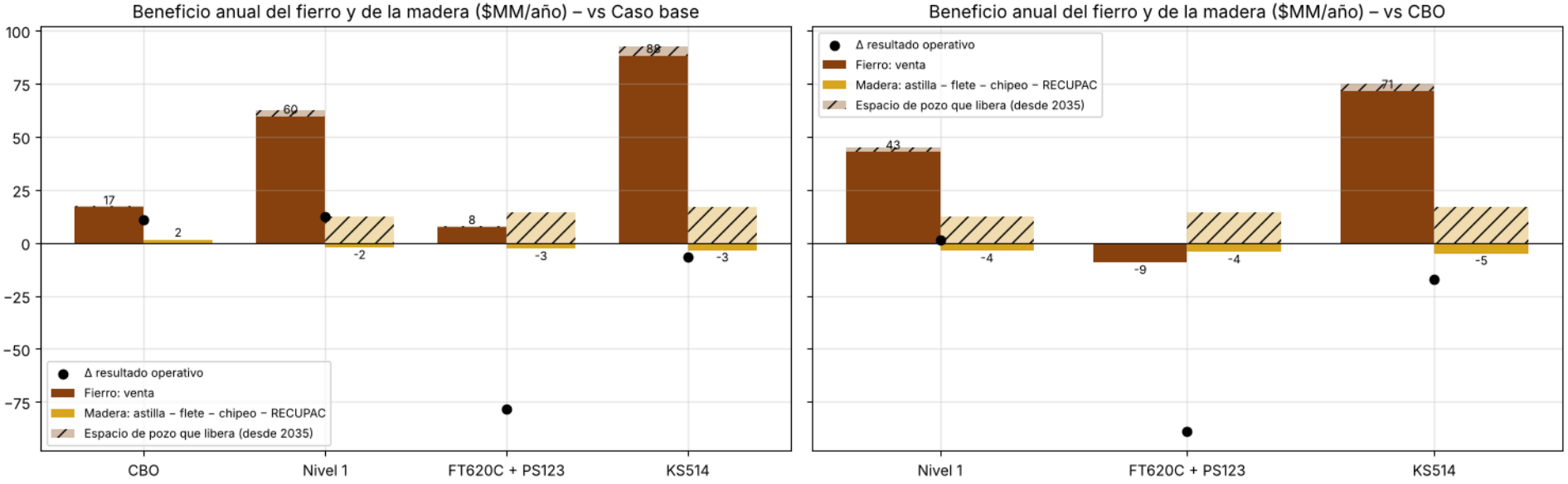
6.9 ¿Cuánto aporta el fierro y cuánto la madera?

Beneficio monetario anual promedio (10 años) de cada material, contra el caso base y contra el CBO:

- **Fierro:** ventas adicionales a Ecociclo (incluye lo que saca el imán de la retro).
- **Madera (neta):** venta de astilla − flete a Teno − chipeo, más el pago a RECUPAC que se evita (o que aumenta, si se recupera madera no apta).
- **Aluminio, cartón y plástico**, y **áridos** si el interruptor está activo.
- **Rayado:** el espacio de pozo que libera cada material (t ÷ su densidad × $4.200/m³). No es caja hoy: se cobra cuando el pozo se llena (2035), por eso pesa menos en el VAN (§8.4b).
- El punto negro es el **Δ resultado operativo** (materiales − OPEX adicional, sin el pozo).

In [30]:
```
ORDEN_PL = [sc['nombre'] for sc in PLANTAS.values()]
comp_cols = {'Fierro': 'Δ fierro ($MM)', 'Madera (neta)': 'Δ madera neta ($MM)', 'Aluminio, cartón, plástico': 'Δ aluminio y otros ($MM)', 'Áridos (neto)': 'Δ áridos ($MM)'}
fig, ax = plt.subplots(1, 2, figsize=(15, 4.8), sharey=True)
filas = []
for a_, (titulo, dic) in zip(ax, [('vs Caso base', inc_base), ('vs CBO', inc_cbo)]):
    noms = [k for k in dic]; x = np.arange(len(noms)); w = 0.38
    fi = [dic[k]['Δ fierro ($MM)'].mean() for k in noms]; ma = [dic[k]['Δ madera neta ($MM)'].mean() for k in noms]
    ot = [dic[k]['Δ aluminio y otros ($MM)'].mean() + dic[k]['Δ áridos ($MM)'].mean() for k in noms]
    vm_ = POZO['valor_m3'] / 1e6
    fi_p = [dic[k]['Δ fierro (t)'].mean() / DENS['fierro'] * vm_ for k in noms]; ma_p = [dic[k]['Δ madera (t)'].mean() / DENS['madera'] * vm_ for k in noms]
```

```
    a_.bar(x - w/2, fi, w, color=COLOR_COMP['Fierro'], label='Fierro: venta')
    a_.bar(x + w/2, ma, w, color=COLOR_COMP['Madera (neta)'], label='Madera: astilla − flete − chipeo − RECUPAC')
    a_.bar(x - w/2, fi_p, w, bottom=np.maximum(fi, 0), color=COLOR_COMP['Fierro'], alpha=.35, hatch='//', label='Espacio de pozo que libera (desde 2035)')
    a_.bar(x + w/2, ma_p, w, bottom=np.maximum(ma, 0), color=COLOR_COMP['Madera (neta)'], alpha=.35, hatch='//')
    a_.scatter(x, [dic[k]['Δ resultado operativo ($MM)'].mean() for k in noms], color='k', zorder=3, label='Δ resultado operativo')
    for xi, f_, m_ in zip(x, fi, ma):
        a_.annotate(f'{f_:,.0f}', (xi - w/2, f_), ha='center', va='bottom' if f_ >= 0 else 'top', fontsize=8)
        a_.annotate(f'{m_:,.0f}', (xi + w/2, m_), ha='center', va='bottom' if m_ >= 0 else 'top', fontsize=8)
    a_.set_xticks(x); a_.set_xticklabels([k.split(' vs ')[0].replace('Planta ', '') for k in noms]); a_.axhline(0, color='k', lw=.8)
    a_.set_title(f'Beneficio anual del fierro y de la madera ($MM/año) — {titulo}'); a_.legend(fontsize=8)
    for k in noms:
        d_ = dic[k].mean()
        filas.append({'comparación': k, 'Δ fierro (t/año)': d_['Δ fierro (t)'], 'Δ fierro ($MM/año)': d_['Δ fierro ($MM)'],
                      '$ por t de fierro': d_['Δ fierro ($MM)'] * 1e6 / d_['Δ fierro (t)'] if d_['Δ fierro (t)'] > 0 else np.nan,
                      'pozo que libera el fierro ($MM/año)': d_['Δ fierro (t)'] / DENS['fierro'] * POZO['valor_m3'] / 1e6,
                      'Δ madera (t/año)': d_['Δ madera (t)'], 'Δ astilla (t/año)': d_['Δ astilla (t)'], 'Δ madera neta ($MM/año)': d_['Δ madera neta ($MM)'],
                      'pozo que libera la madera ($MM/año)': d_['Δ madera (t)'] / DENS['madera'] * POZO['valor_m3'] / 1e6,
                      '$ neto por t de madera': d_['Δ madera neta ($MM)'] * 1e6 / d_['Δ madera (t)'] if d_['Δ madera (t)'] > 0 else np.nan,
                      'Δ otros + áridos ($MM/año)': d_['Δ aluminio y otros ($MM)'] + d_['Δ áridos ($MM)'], 'Δ OPEX ($MM/año)': d_['Δ OPEX ($MM)'],
                      'Δ resultado operativo ($MM/año)': d_['Δ resultado operativo ($MM)']})
plt.tight_layout(); plt.show()
ben_fm = pd.DataFrame(filas).set_index('comparación')
display(ben_fm.style.format('{:,.1f}').format({c: '{:,.0f}' for c in ['Δ fierro (t/año)', '$ por t de fierro', 'Δ madera (t/año)', 'Δ astilla (t/año)', '$ neto por t de madera']
p_ = MADERA
print(f"Valor unitario: 1 t de fierro = ${MERCADO['precio']['fierro']*1000:,.0f}. 1 t de madera apta a astilla = ${p_['precio_astilla']:,.0f} venta − ${p_['flete_astilla']:,.0f}
      f"+ ${MERCADO['costo_recupac']*1000:,.0f} de RECUPAC evitado − chipeo (compra ≈ ${MADERA['chipeo']['compra']['factor_horas']/MADERA['chipeo']['compra']['cap_th']*MADERA['c
      f" El fierro vale ~{MERCADO['precio']['fierro']*1000/(p_['precio_astilla']-p_['flete_astilla']+MERCADO['costo_recupac']*1000):.0f} veces más por tonelada.")
```



comparación	Δ fierro (t/año)	Δ fierro ($MM/año)	$ por t de fierro	pozo que libera el fierro ($MM/año)	Δ madera (t/año)	Δ astilla (t/año)	Δ madera neta ($MM/año)	pozo que libera la madera ($MM/año)	$ neto por t de madera	Δ otros + áridos ($MM/año)	Δ OPEX ($MM/año)	Δ resultado operativo ($MM/año)
CBO vs Caso base	88	16.764227	189,857	0.882992	0	94	1.538179	0.000000	—	0.000000	7.442581	10.859825
Planta Nivel 1 vs Caso base	313	59.580119	190,646	3.125175	1,008	800	-2.146371	12.455319	-2,129	40.391214	85.551654	12.273309
Planta FT620C + PS123 vs Caso base	40	7.523866	188,375	0.399408	1,175	917	-2.756330	14.517236	-2,345	43.541956	126.476619	-78.167127
Planta KS514 vs Caso base	462	88.176369	190,746	4.622713	1,351	1,040	-3.399100	16.690068	-2,516	50.633338	141.912221	-6.501614
Planta Nivel 1 vs CBO	224	42.815892	190,956	2.242183	1,008	706	-3.684549	12.455319	-3,654	40.391214	78.109074	1.413484
Planta FT620C + PS123 vs CBO	-48	-9.240361	—	-0.483584	1,175	823	-4.294509	14.517236	-3,654	43.541956	119.034038	-89.026952
Planta KS514 vs CBO	374	71.412142	190,956	3.739721	1,351	946	-4.937279	16.690068	-3,654	50.633338	134.469641	-17.361439

```
Valor unitario: 1 t de fierro = $195,000. 1 t de madera apta a astilla = $30,000 venta − $20,066 flete + $15,126 de RECUPAC evitado − chipeo (compra ≈ $8,672/t de energía). El fi
erro vale ~8 veces más por tonelada.
```

6.10 Interruptor de áridos (`ARIDOS['activo']`)

Por defecto está apagado. Aquí se prende solo para mostrar qué cambia en cada planta, con chancado por campañas arrendadas y con chancador propio. Solo se chanca el **hormigón que llega separado** (~10% de lo recibido, RETC 2024): el RCD mixto no cumple como base granular (CBR 41 en el tramo de prueba del Centro del Hormigón UC). El árido vendido sale del predio y libera espacio de pozo; el que se usa en el predio (caminos, cobertura) no libera espacio y hoy vale $0 porque falta saber cuánto material compra REGEMAC. El VAN con y sin áridos está en el §8.6c.

In [31]:
```
filas = []
for modo in ('arriendo', 'compra'):
    with temporal(ARIDOS, 'activo', True), temporal(ARIDOS, 'modo', modo):
        for sc in PLANTAS.values():
            a = resumen(caso_planta(cr, sc)).mean(); b = R_[sc['nombre']].mean()
            filas.append({'modo': modo, 'planta': sc['nombre'], 'hormigón chancado → árido (t/año)': a.aridos_t, 'vendido (t/año)': a.aridos_venta_t,
                          'neto áridos ($MM/año)': a.neto_aridos_clp / 1e6, 'Δ mov. al pozo ($MM/año)': (a.opex_mov_clp - b.opex_mov_clp) / 1e6,
                          'Δ m3 de pozo ahorrados/año': b.vertedero_m3 - a.vertedero_m3, 'CAPEX chancador ($MM)': capex(sc).get('Chancador', 0) / 1e6})
t_ar = pd.DataFrame(filas)
display(t_ar.style.format({c: '{:,.1f}' for c in t_ar.columns if c not in ('modo', 'planta')} | {'hormigón chancado → árido (t/año)': '{:,.0f}', 'vendido (t/año)': '{:,.0f}', 'Δ
A_ = ARIDOS; ar_ = A_['arriendo']
c_t = ar_['usd_dia'] * COSTOS['tc_usd'] / ar_['t_dia'] + A_['diesel_l_t'] * COSTOS['precio_diesel'] + A_['personal_t']
print(f"Costo de chancar por campaña ≈ ${c_t:,.0f}/t (+ cargo por campaña) vs precio de venta ${A_['precio_venta_t']:,.0f}/t × {A_['frac_venta']:.0%} vendido. "
      f"Con chancador propio: ${A_['compra']['opex_t']:,.0f}/t + CAPEX.")
```

	modo	planta	hormigón chancado → árido (t/año)	vendido (t/año)	neto áridos ($MM/año)	Δ mov. al pozo ($MM/año)	Δ m3 de pozo ahorrados/año	CAPEX chancador ($MM)
0	arriendo	Planta Nivel 1	16,583	8,291	-63.2	-1.8	12,522	0.0
1	arriendo	Planta FT620C + PS123	16,583	8,291	-63.2	-1.8	12,522	0.0
2	arriendo	Planta KS514	16,583	8,291	-63.2	-1.8	12,522	0.0
3	compra	Planta Nivel 1	16,583	8,291	-46.0	-1.8	12,522	376.7
4	compra	Planta FT620C + PS123	16,583	8,291	-46.0	-1.8	12,522	376.7
5	compra	Planta KS514	16,583	8,291	-46.0	-1.8	12,522	376.7

```
Costo de chancar por campaña ≈ $4,705/t (+ cargo por campaña) vs precio de venta $4,500/t × 50% vendido. Con chancador propio: $3,500/t + CAPEX.
```

Lectura de resultados (v1.2, semilla 42; cambia si se cambian los supuestos)

Validación. El caso base reproduce las ventas de reciclaje ($122 MM simulado vs $122 MM anualizado 2026) y la madera a RECUPAC (133 vs 140 t). Recupera 0,44% de lo recibido, contra el 0,7% medido por Rodrigo en 2025.

Capacidad y personal (1 turno). Nivel 1: 24 t/h (mandan los 8 operarios de la cabina, a 3 t/h cada uno). FT620C: 27 t/h y KS514: 30 t/h (manda la máquina). Procesan 26%, 29% y 33% de lo que llega. Cada planta usa 10 personas por turno (8 en la cabina + 2 de transferencia): **2 más que los 8 de hoy** (+$26 MM/año).

Todos contra el caso base (promedio anual):

- **CBO:** +$10,9 MM/año (el imán agrega ~88 t de fierro). CAPEX $73 MM. Payback simple ~7 años.
- **Nivel 1:** materiales +$98 MM/año (fierro +313 t = +$60 MM; aluminio, cartón y plástico +$40 MM; madera −$2 MM); costos +$86 MM/año (personal 26, energía 10, diésel 33, mantención 16) → **+$12 MM/año**. CAPEX $407 MM.
- **FT620C + PS123:** el fierro casi no sube (+40 t), porque el trómel no tiene imán bajo malla y los trabajadores dejan la fosa → **−$78 MM/año**; CAPEX $1.015 MM.
- **KS514:** fierro +462 t (+$88 MM), pero la mantención de un equipo de $1.140 MM pesa → **−$6,5 MM/año**; CAPEX $1.550 MM.
- **Pozo:** 6.000-8.700 m³/año ahorrados, 0,17-0,23 años más de vida útil en 10 años.

¿Conviene la chipeadora? (§6.2c).

- Por tonelada de madera apta, chipear deja $30.000 − $20.066 de flete − $8.672 de energía = **+$1.262**, contra **−$15.126** de RECUPAC: $16.400/t a favor de chipear antes de los costos fijos. Los costos fijos de comprar la chipeadora son ~$12 MM/año (mantención $3 MM + CAPEX y losa anualizados $9 MM).
- Por eso **comprar conviene desde ~730 t de madera apta al año**. Con la madera de hoy (CBO, ~94 t apta) conviene seguir con RECUPAC: chipear pierde ~$10 MM/año. Con una planta (800-1.040 t apta) comprar gana $1-5 MM/año.
- **El arriendo de Raico nunca conviene** a estos volúmenes: cada semana cuesta $16,5 MM y solo se paga si se llenan las 700 t; necesitaría una astilla de más de ~$31.000-45.000/t.
- El volumen sí cambia: 1 t de madera suelta ocupa 2,9 m³ en el pozo y 4,5 m³ como astilla. Un camión de 100 m³ lleva ~22 t, y el flete de 4,5 USD/m³ (supuesto sin ida y vuelta) da ~$20.000/t. Si el flete fuera el doble, chipear no conviene en ningún escenario.

Fierro vs madera (§6.9). Una tonelada de fierro vale $195.000; la madera adicional que recupera una planta **resta** ~$2.000/t en caja (el 30% no apto va a RECUPAC). Su valor está en el pozo: 2,9 m³/t × $4.200 ≈ $12.400/t, pero recién desde 2035.

Alimentación (§6.6). El cargador propio (diésel ~$30 MM/año, mantención y $100 MM de CAPEX; lo opera uno de los 2 de transferencia) baja el resultado de cada planta ~$25 MM/año contra la forma de la v1.0. Con arriendo con operador ($45 mil/h) baja ~$110 MM/año.

Áridos (§6.10). Con los precios encontrados no conviene: chancar por campaña cuesta ~$4.700/t y solo la mitad se vende, a ~$4.500/t.

7. Monte Carlo: incrementales con números aleatorios comunes

Cada corrida sortea otra historia (llegadas, contenido, precios con riesgo de perder al comprador) y aplica todos los escenarios a esa misma historia. Se reportan los totales en 10 años: P10 / P50 / P90 y la probabilidad de que el resultado operativo incremental sea negativo. Primero contra el caso base y después las plantas contra el CBO.

In [32]:
```
N_MC = SIM['n_montecarlo']
filas = []
def fila_mc(i, nombre, r_, r0):
    return pd.DataFrame({'corrida': i, 'comparacion': nombre, 'anio': r_.index, 'pct_procesado': r_.pct_procesado,
        'd_fierro_t': r_.fierro_t - r0.fierro_t, 'd_noperm_t': r_.vert_noperm_t - r0.vert_noperm_t, 'd_m3': r_.ahorro_m3 - r0.ahorro_m3,
        'd_neto_MM': (r_.neto_clp - r0.neto_clp)/1e6, 'd_opex_MM': (r_.opex_clp - r0.opex_clp)/1e6,
        'd_resultado_MM': (r_.resultado_clp - r0.resultado_clp)/1e6,
        # para el flujo de caja del §8
        'd_neto_clp': r_.neto_clp - r0.neto_clp, 'd_opex_clp': r_.opex_clp - r0.opex_clp,
        'd_ing_clp': r_.ingresos_clp - r0.ingresos_clp, 'd_m3disp': r0.vertedero_m3 - r_.vertedero_m3,
        'd_fierro_clp': r_.ing_fierro_clp - r0.ing_fierro_clp, 'd_madera_clp': r_.neto_madera_clp - r0.neto_madera_clp,
        'd_otros_clp': r_.ing_aluminio_clp + r_.ing_otros_clp - r0.ing_aluminio_clp - r0.ing_otros_clp,
        'd_aridos_clp': r_.neto_aridos_clp - r0.neto_aridos_clp}).reset_index(drop=True)
for i in range(N_MC):
    cr_i = preparar_corrida(SIM['semilla'] + 1 + i)
    rb_i, rc_i = resumen(caso_base(cr_i)), resumen(caso_cbo(cr_i))
    filas.append(fila_mc(i, 'CBO vs Caso base', rc_i, rb_i))
```

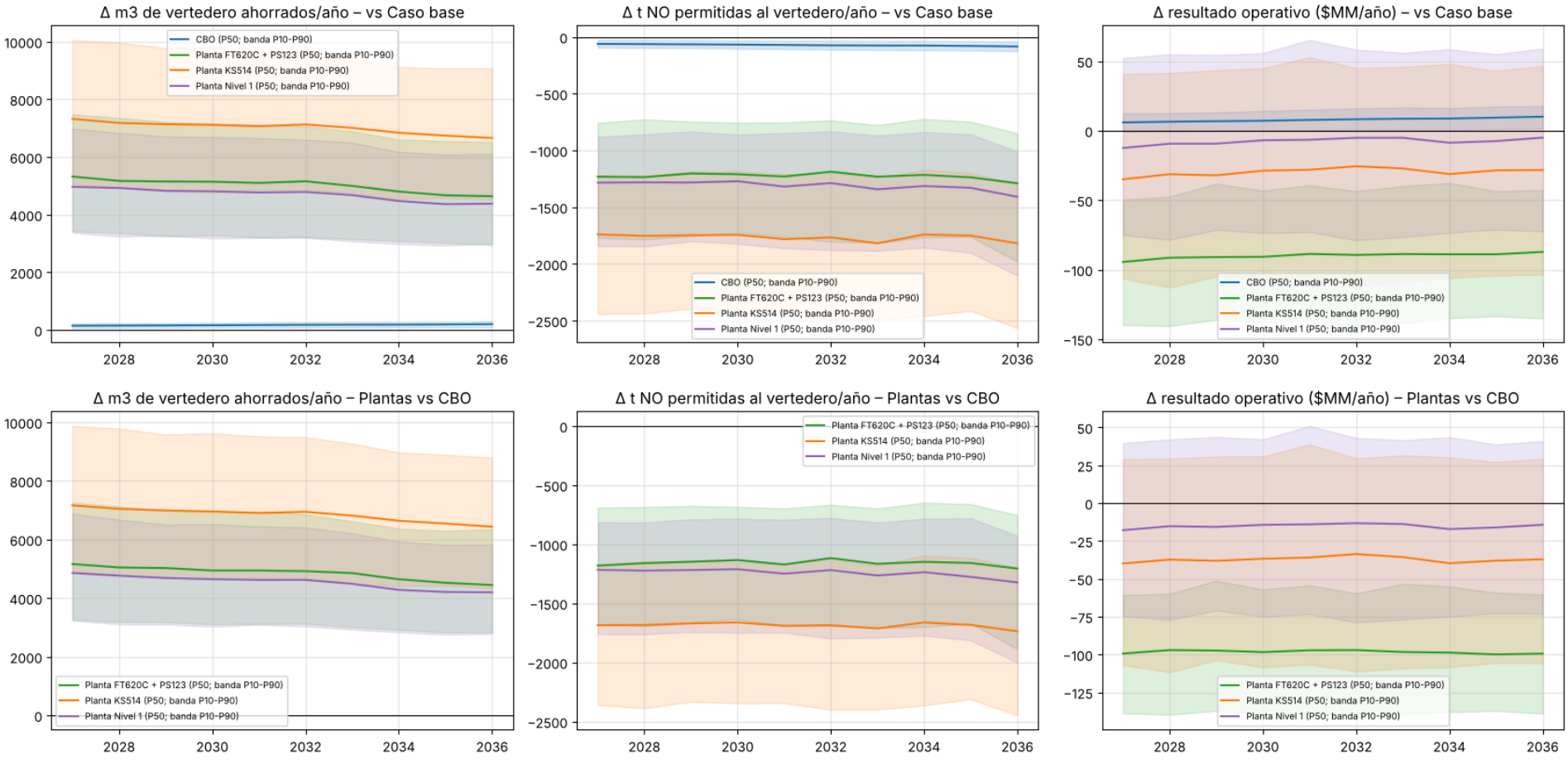
```
    for k, sc in PLANTAS.items():
        rp_i = resumen(caso_planta(cr_i, sc))
        filas.append(fila_mc(i, f"{sc['nombre']} vs Caso base", rp_i, rb_i))
        filas.append(fila_mc(i, f"{sc['nombre']} vs CBO", rp_i, rc_i))
mc = pd.concat(filas)
tot = mc.groupby(['comparacion','corrida'])[['d_fierro_t','d_noperm_t','d_m3','d_neto_MM','d_opex_MM','d_resultado_MM']].sum()
q = tot.groupby('comparacion').quantile([.1, .5, .9]).unstack()
orden = ['CBO vs Caso base'] + [f"{sc['nombre']} vs Caso base" for sc in PLANTAS.values()] + [f"{sc['nombre']} vs CBO" for sc in PLANTAS.values()]
q = q.loc[orden]
print('Totales en 10 años (P10 / P50 / P90):'); display(q.round(0))
print('P(Δ resultado operativo 10 años < 0):'); display(tot.groupby('comparacion').d_resultado_MM.apply(lambda x: (x < 0).mean()).loc[orden].to_frame('P(<0)').style.format('{:.0
for titulo, filtro in [('vs Caso base', 'Caso base'), ('Plantas vs CBO', 'vs CBO')]:
    fig, ax = plt.subplots(1, 3, figsize=(17, 4.2))
    for a_, col, t_ in [(ax[0],'d_m3','Δ m3 de vertedero ahorrados/año'), (ax[1],'d_noperm_t','Δ t NO permitidas al vertedero/año'), (ax[2],'d_resultado_MM','Δ resultado operati
        for pl, gq in mc[mc.comparacion.str.endswith(filtro)].groupby('comparacion'):
            qq = gq.groupby('anio')[col].quantile([.1, .5, .9]).unstack()
            a_.fill_between(qq.index, qq[.1], qq[.9], alpha=.15, color=col_de(pl)); a_.plot(qq.index, qq[.5], color=col_de(pl), label=pl.split(' vs ')[0] + ' (P50; banda P10-P90
        a_.axhline(0, color='k', lw=.8); a_.set_title(f'{t_} — {titulo}'); a_.legend(fontsize=7)
    plt.tight_layout(); plt.show()
```

Totales en 10 años (P10 / P50 / P90):

	d_fierro_t			d_noperm_t			d_m3			d_neto_MM			d_opex_MM			d_resultado_MM		
	0.10	0.50	0.90	0.10	0.50	0.90	0.10	0.50	0.90	0.10	0.50	0.90	0.10	0.50	0.90	0.10	0.50	0.90
comparacion																		
CBO vs Caso base	323.00	710.00	1,079.00	-1,079.00	-709.00	-323.00	791.00	1,735.00	2,640.00	79.00	152.00	224.00	74.00	74.00	75.00	4.00	78.00	150.00
Planta Nivel 1 vs Caso base	-1,722.00	1,644.00	4,820.00	-18,641.00	-13,235.00	-8,870.00	31,038.00	47,035.00	65,301.00	116.00	785.00	1,454.00	855.00	857.00	858.00	-741.00	-72.00	597.00
Planta FT620C + PS123 vs Caso base	-2,968.00	-674.00	1,563.00	-17,732.00	-12,265.00	-7,546.00	32,124.00	50,332.00	69,547.00	-115.00	367.00	845.00	1,265.00	1,266.00	1,267.00	-1,381.00	-898.00	-420.00
Planta KS514 vs Caso base	-1,028.00	2,907.00	6,604.00	-24,532.00	-17,604.00	-12,113.00	46,966.00	70,336.00	95,658.00	343.00	1,137.00	1,905.00	1,419.00	1,420.00	1,421.00	-1,079.00	-283.00	486.00
Planta Nivel 1 vs CBO	-2,037.00	927.00	3,739.00	-17,848.00	-12,509.00	-8,166.00	29,700.00	45,419.00	62,861.00	33.00	647.00	1,232.00	781.00	782.00	783.00	-751.00	-136.00	451.00
Planta FT620C + PS123 vs CBO	-3,306.00	-1,398.00	492.00	-16,789.00	-11,718.00	-7,123.00	30,985.00	48,739.00	67,592.00	-182.00	210.00	618.00	1,190.00	1,191.00	1,192.00	-1,374.00	-980.00	-572.00
Planta KS514 vs CBO	-1,351.00	2,196.00	5,529.00	-23,810.00	-16,928.00	-11,730.00	46,278.00	68,647.00	94,012.00	269.00	993.00	1,676.00	1,344.00	1,346.00	1,347.00	-1,079.00	-353.00	332.00

P(Δ resultado operativo 10 años < 0):

	P(<0)
comparacion	
CBO vs Caso base	8%
Planta Nivel 1 vs Caso base	58%
Planta FT620C + PS123 vs Caso base	100%
Planta KS514 vs Caso base	70%
Planta Nivel 1 vs CBO	67%
Planta FT620C + PS123 vs CBO	100%
Planta KS514 vs CBO	76%



8. Módulo 4 – Evaluación económica (flujo de caja y VAN)

Qué hace. Convierte la diferencia anual entre dos escenarios (Δ resultado operativo, Δ m³ de pozo) en un **flujo de caja libre (FCL) incremental en pesos reales de 2026**, que equivalen a UF porque no hay inflación, y lo descuenta con una **tasa real**. La estructura sigue el FCL parametrizado de Pedro (depreciación, impuesto, capital de trabajo, valor residual), pero alimentada con los flujos de la simulación.

Estructura del FCL (cada año t = 0…10):

```
  Δ fierro + Δ madera (neta) + Δ aluminio, cartón, plástico + Δ áridos   (primer año de operación × rampa)
− Δ OPEX                     (personal, energía, diésel, mantención, movimiento al pozo, alimentación)
+ Valor del espacio de pozo (modo 'vida_util': los m³ ahorrados se cobran cuando el pozo se llena, al costo de disponer afuera)
− Seguros (% del CAPEX)      − Permisos, consulta de pertinencia SEIA y (escenario) DIA (año 0, gasto)
= EBITDA
− Depreciación (tabla SII)  = EBIT
− Impuesto (27% del EBIT; si es negativo es ahorro, porque REGEMAC tiene utilidades)
= Utilidad después de impuesto
```

```
+ Depreciación − CAPEX (año 0) − Δ capital de trabajo + valor residual neto (año 10)
= FCL
```

VAN = Σ FCL_t / $(1 + tasa)^t$. También se calculan TIR, payback descontado, VAN para un rango de tasas, el **valor del m³ de pozo que hace VAN = 0** (break-even) y la distribución del VAN en el Monte Carlo (P10/P50/P90, P(VAN<0)), como pidió Rodrigo.

Año 1 parcial. Las plantas operan desde el mes `inicio_meses` (6: plazo de fabricación y entrega; 18 con DIA). Mientras no operan, el escenario es igual a su referencia (el CBO): contra el caso base se cuenta lo que aporta el CBO esos meses.

Supuestos que vale la pena discutir:

1. **Espacio de pozo.** Al pozo le quedan 8 años (Rodrigo), así que se llena a fines de 2034. Cada m³ suelto ahorrado vale lo que REGEMAC cobra a externos ($4.200), y se cobra cuando el pozo se llena: en 2035 todo lo acumulado y después lo ahorrado cada año. Es conservador en el tiempo; si REGEMAC hoy rechaza clientes por falta de espacio, el m³ se podría vender antes (`POZO['modo']='anual'`). Con `POZO['modo']='anual'` se vuelve a la v1.0 (cada m³ vale lo mismo el año en que se ahorra).
2. La evaluación es del **proyecto puro**, sin financiamiento. El leasing que ofrece SKC queda como escenario.
3. El incremental contra el CBO no incluye la inversión del CBO, porque está en los dos escenarios.

8.0 De dónde sale cada número del módulo 4

Parámetro	Valor	Fuente	Tipo	Confianza
Tasa libre de riesgo real	2,74%	BCU/BTU-10 del Banco Central, 9-oct-2026	Fuente externa	alta
Beta desapalancada	0,82	Damodaran, ene-2026, *Environmental & Waste Services*	Fuente externa	media
D/E de la industria	21,45%	Damodaran, ene-2026, misma industria	Fuente externa	media
Prima por riesgo de mercado Chile	5,33% (4,23% madura + 1,10% país)	Damodaran, ene-2026	Fuente externa	media
Premio por proyecto	+2 pp	Tecnología nueva para REGEMAC y venta de material que hoy no vende	SUPUESTO justificado	baja
Costo de deuda real	9,2%	Deuda de REGEMAC: 4 leasing y crédito Itaú, 10,9-13,9% nominal (Rodrigo 13-oct) → real con 3% de inflación (IPoM)	Dato REGEMAC + calculado	media
Rango de tasas	6%–12% real	Rodrigo: "muestren el VAN para un rango de tasas"	Pedido del mandante	—
Impuesto	27%	Régimen 14A (Rodrigo 13-oct)	Dato REGEMAC	alta
Vida útil tributaria	Maquinaria 15 años (acelerada 5); obras 10 (acelerada 3)	Tabla de vida útil SII	Norma	media
Valor residual al año 10	KS514 15%, FT620C+PS123 30%, Nivel 1 15% de la máquina; cargador 25%; obras 0%	Usados TTS620 (web); KS514 estática sin mercado	Fuente externa / SUPUESTO	media / baja
Capital de trabajo	30 días de cobro y 30 de pago	Supuesto	SUPUESTO	baja
Seguros	1% del CAPEX por año	Supuesto	SUPUESTO	baja
Permisos y estudios	$20 MM + consulta de pertinencia SEIA 75 UF (año 0); escenario DIA: +1.250 UF y +12 meses	PAS 140, Seremi, OGUC; web (SEA, SOFOFA)	SUPUESTO / web	baja
Inicio de operación	Mes 6 de 2027 (las tres plantas)	Cotización Kiverco: 12-16 + 6-8 semanas	Cotización / SUPUESTO	media
Puesta en marcha	85% del margen de materiales en el primer año de operación	Supuesto (curva de aprendizaje)	SUPUESTO	baja
CAPEX plantas	KS514 GBP 883.000; FT620C+PS123 GBP 530.000; Nivel 1 ver §2.3	Cotizaciones SKC 12-09-2025 (vencidas); planos REGEMAC + web	Cotización / SUPUESTO	media / baja

Parámetro	Valor	Fuente	Tipo	Confianza
Tipo de cambio	1.290 CLP/GBP; 981 CLP/USD; UF $41.157	14-oct-2026	Fuente externa	alta / media
Inversiones fuera de la cotización	Obras 12%, internación 2,5% (arancel 0%), empalme 565 UF, polvo $15 MM, repuestos 2-3%, cargador $100 MM, losa de astilla 600 UF	Rodrigo; web	Varios	alta-baja
Valor del m³ de pozo	$4.200 por m³ suelto (lo que REGEMAC cobra a externos), cobrado cuando el pozo se llena (2035); sensibilidad $3.500-8.000 y cobro anual	Rodrigo 13-oct	Dato REGEMAC	media
Compactación en el pozo	1,0	Rodrigo: solo la pasada del bulldozer y los camiones	Dato REGEMAC	media

Los parámetros operativos (precios, costos, capacidades) tienen su fuente en los comentarios de §1-§2 y en la tabla del §0.1.

In [33]:
```
# 8.1 Tasa de descuento real: WACC con CAPM (Damodaran)
c = FIN['capm']; t_ = FIN['impuesto']
beta_l = c['beta_u'] * (1 + (1 - t_) * c['d_e'])              # Hamada: beta apalancada con el D/E de la industria
ke = c['rf_real'] + beta_l * c['erp'] + c['premio']             # costo del patrimonio real (con premio por proyecto)
wd = c['d_e'] / (1 + c['d_e']); we = 1 - wd
wacc = we * ke + wd * c['kd_real'] * (1 - t_)
TASA = FIN['tasa_real_base'] if FIN['tasa_real_base'] is not None else wacc
display(pd.DataFrame({'valor': [c['rf_real'], c['beta_u'], c['d_e'], beta_l, c['erp'], c['premio'], ke, c['kd_real'], we, wd, wacc, TASA]},
                     index=['rf real', 'beta desapalancada', 'D/E', 'beta apalancada', 'prima de mercado Chile', 'premio proyecto',
                            'Ke real', 'Kd real', 'peso patrimonio', 'peso deuda', 'WACC real', 'TASA usada']).style.format('{:.4f}'))
print(f"Sin premio por proyecto el WACC real sería {we * (ke - c['premio']) + wd * c['kd_real'] * (1 - t_):.2%}. Referencia del equipo (Emilio): 10,76% nominal ≈ {(1.1076/1.03 -
```

	valor
rf real	0.0274
beta desapalancada	0.8200
D/E	0.2145
beta apalancada	0.9484
prima de mercado Chile	0.0533
premio proyecto	0.0200
Ke real	0.0979
Kd real	0.0920
peso patrimonio	0.8234
peso deuda	0.1766
WACC real	0.0925
TASA usada	0.0925

```
Sin premio por proyecto el WACC real sería 7.60%. Referencia del equipo (Emilio): 10,76% nominal ≈ 7.53% real. v1.0: 8,79%.
```

In [34]:
```
# 8.2 Funciones del flujo de caja
from scipy.optimize import brentq
OBRAS = ('Obras civiles, grúa, tablero', 'Losa acopio astilla', 'Empalme eléctrico')
def cat_de(k, sc):
    # Categoría tributaria de cada ítem de CAPEX (vida útil SII)
    if k in OBRAS: return 'obras'
    if k in sc.get('capex_items', {}): return sc['capex_items'][k][2]
    return 'maquinaria'

def residual_de(k, sc):
    # % del CAPEX que se recupera al año 10 (valor de mercado)
    if cat_de(k, sc) == 'obras': return FIN['valor_residual']['obras']
    if k in ('Repuestos iniciales', 'Internación (agente, puerto)'): return 0.0
```

```
    if k == 'Control de polvo': return 0.10
    if k == 'Cargador frontal': return ALIMENTACION['cargador_propio']['valor_residual']
    if k == 'Planta (cotización)' or k in sc.get('capex_items', {}): return sc.get('valor_residual_equipo', FIN['valor_residual']['maquinaria'])
    return FIN['valor_residual']['maquinaria']

COMPS = [('CBO', 'Caso base')] + [(sc['nombre'], 'Caso base') for sc in PLANTAS.values()] + [(sc['nombre'], 'CBO') for sc in PLANTAS.values()]
COMP_COLS = ['d_fierro', 'd_madera', 'd_otros', 'd_aridos']
DD_COLS = ['d_neto', 'd_opex', 'd_ing', 'd_m3'] + COMP_COLS

def deltas(r_, r0):
    # Diferencias anuales en $ y m3 que alimentan el FCL (r_ − r0)
    return pd.DataFrame({'d_neto': r_.neto_clp - r0.neto_clp, 'd_opex': r_.opex_clp - r0.opex_clp,
                         'd_ing': r_.ingresos_clp - r0.ingresos_clp, 'd_m3': r0.vertedero_m3 - r_.vertedero_m3,
                         'd_fierro': r_.ing_fierro_clp - r0.ing_fierro_clp, 'd_madera': r_.neto_madera_clp - r0.neto_madera_clp,
                         'd_otros': r_.ing_aluminio_clp + r_.ing_otros_clp - r0.ing_aluminio_clp - r0.ing_otros_clp,
                         'd_aridos': r_.neto_aridos_clp - r0.neto_aridos_clp})

def factor_operacion(sc_n, sc_r, N):
    # Fracción de cada año (1..N) en que opera la planta nueva; 1 si no hay planta nueva
    if not (es_planta(sc_n) and not es_planta(sc_r)): return np.ones(N)
    ini = sc_n.get('inicio_meses', 0) + (FIN['seia']['dia_retraso_meses'] if FIN['seia']['dia'] else 0)
    return np.clip((12 * np.arange(1, N + 1) - ini) / 12, 0, 1)

def valor_pozo(d_m3, vm3, N):
    # $ por año del espacio de pozo ahorrado (ver texto del §8)
    if POZO['modo'] == 'anual': return d_m3 * POZO['factor_compactacion'] * vm3
    anios = SIM['anio_inicio'] + np.arange(N); v = np.zeros(N); lleno = anios > POZO['anio_lleno']
    if lleno.any():
        k = np.flatnonzero(lleno)[0]; v[k] = d_m3[:k + 1].sum(); v[k + 1:] = d_m3[k + 1:]
    else:
        v[-1] = d_m3.sum()                        # el pozo se llena después del horizonte: se valora al final
    return v * POZO['factor_compactacion'] * vm3

def flujo_caja(dd, sc_n, sc_r, valor_m3=None, dd_prev=None):
    # FCL incremental (sc_n − sc_r), años 0..N, en $ reales de 2026. dd_prev: deltas de la referencia "previa" (CBO vs ref) para los meses sin planta
    d = dd[DD_COLS].reset_index(drop=True); N = len(d); A = list(range(N + 1)); t = FIN['impuesto']
    prev = dd_prev[DD_COLS].reset_index(drop=True) if dd_prev is not None else d * 0
    vm3 = POZO['valor_m3'] if valor_m3 is None else valor_m3
    cn, c0 = capex(sc_n), capex(sc_r)
    inv = {k: cn.get(k, 0) - c0.get(k, 0) for k in set(cn) | set(c0)}
    inv = {k: v for k, v in inv.items() if abs(v) > 1}
    capex_tot = sum(inv.values())
    planta_nueva = es_planta(sc_n) and not es_planta(sc_r)
    fr = factor_operacion(sc_n, sc_r, N); ramp = np.ones(N)
    if planta_nueva and (fr > 0).any(): ramp[np.flatnonzero(fr > 0)[0]] = FIN['rampa_anio1']
    b = {c: fr * d[c].values * (ramp if c in ['d_neto'] + COMP_COLS else 1) + (1 - fr) * prev[c].values for c in DD_COLS}
    z = lambda: np.zeros(N + 1)
    comp = {c: z() for c in COMP_COLS}
    for c in COMP_COLS: comp[c][1:] = b[c]
    opex, pozo, seg, perm = z(), z(), z(), z()
    opex[1:] = -b['d_opex']
    pozo[1:] = valor_pozo(b['d_m3'], vm3, N)
    seg[1:] = -FIN['seguro'] * capex_tot
    if planta_nueva:
        s_ = FIN['seia']
        perm[0] = -(FIN['permisos_planta'] + s_['pertinencia_uf'] * COSTOS['uf'] + (s_['dia_uf'] * COSTOS['uf'] if s_['dia'] else 0))
    ebitda = sum(comp.values()) + opex + pozo + seg + perm
    dep, libro, vr = z(), 0.0, 0.0
    for k, v in inv.items():
        sc_k = sc_n if k in cn else sc_r
```

```
        cat = cat_de(k, sc_k); vida = FIN['vida_util'][cat][1 if FIN['depreciacion_acelerada'] else 0]
        n_ = min(vida, N); dep[1:n_ + 1] += v / vida
        libro += v - v / vida * n_; vr += v * residual_de(k, sc_k)
    ebit = ebitda - dep
    imp = -t * ebit                                  # negativo = paga; positivo = ahorro tributario (REGEMAC tiene utilidades)
    util = ebit + imp
    capx = z(); capx[0] = -capex_tot
    ct = np.r_[0, (b['d_ing'] * FIN['dias_cobro'] - b['d_opex'] * FIN['dias_pago']) / 365]
    dct = -np.diff(ct, prepend=0.0); dct[N] += ct[N]          # se invierte cuando crece y se recupera al final
    res = z(); res[N] = vr - t * (vr - libro)
    fcl = util + dep + capx + dct + res
    return pd.DataFrame([comp['d_fierro'], comp['d_madera'], comp['d_otros'], comp['d_aridos'], opex, pozo, seg, perm, ebitda, -dep, ebit, imp, util, dep, capx, dct, res, fcl],
        index=['Δ fierro', 'Δ madera (neta)', 'Δ aluminio, cartón, plástico', 'Δ áridos (neto)', 'Δ OPEX', 'Valor espacio de pozo', 'Seguros', 'Permisos y estudios', 'EBITDA',
               'Depreciación', 'EBIT', 'Impuesto (−) / ahorro (+)', 'Utilidad después de impuesto', '+ Depreciación',
               'Inversión (CAPEX)', 'Δ capital de trabajo', 'Valor residual neto', 'FCL'])

def van(F, tasa):
    return (F.loc['FCL'].values / (1 + tasa) ** np.arange(F.shape[1])).sum()

def tir(F):
    f = lambda r: van(F, r)
    try: return brentq(f, -0.9, 3.0) if f(-0.9) * f(3.0) < 0 else np.nan
    except ValueError: return np.nan

def payback_desc(F, tasa):
    acum = np.cumsum(F.loc['FCL'].values / (1 + tasa) ** np.arange(F.shape[1]))
    k = np.flatnonzero(acum >= 0)
    return float(k[0]) if len(k) and k[0] > 0 else np.nan

def prev_de(nom, ref, Rd=None):
    Rd = Rd or R_
    return deltas(Rd['CBO'], Rd[ref]) if (es_planta(SC_DE[nom]) and ref == 'Caso base') else None

def fc_de(nom, ref, valor_m3=None):
    return flujo_caja(deltas(R_[nom], R_[ref]), SC_DE[nom], SC_DE[ref], valor_m3, prev_de(nom, ref))

def descomp_van(F, tasa):
    # VAN separado por componente (suma = VAN)
    disc = (1 + tasa) ** -np.arange(F.shape[1]); pv = lambda fila: (F.loc[fila].values * disc).sum()
    return {'Fierro': pv('Δ fierro'), 'Madera (neta)': pv('Δ madera (neta)'), 'Aluminio, cartón, plástico': pv('Δ aluminio, cartón, plástico'),
            'Áridos (neto)': pv('Δ áridos (neto)'), 'Espacio de pozo': pv('Valor espacio de pozo'), 'OPEX': pv('Δ OPEX'),
            'Seguros y permisos': pv('Seguros') + pv('Permisos y estudios'), 'Impuestos': pv('Impuesto (−) / ahorro (+)'),
            'CAPEX, residual y capital de trabajo': pv('Inversión (CAPEX)') + pv('Valor residual neto') + pv('Δ capital de trabajo')}
```

8.3 Flujo de caja de ejemplo: Planta Nivel 1 vs caso base ($MM reales, con el valor del espacio de pozo)

In [35]:
```
F_ej = fc_de('Planta Nivel 1', 'Caso base')
display((F_ej / 1e6).style.format('{:,.1f}').set_properties(subset=pd.IndexSlice[['EBITDA', 'FCL'], :], **{'font-weight': 'bold'}))
t_ = tir(F_ej)
print(f"VAN al {TASA:.2%} real: ${van(F_ej, TASA)/1e6:,.0f} MM | TIR: " + (f"{t_:.2%}" if t_ == t_ else "no existe"))
print(f"Pozo: lleno en {POZO['anio_lleno']} → el espacio ahorrado se cobra desde {POZO['anio_lleno'] + 1} (año {POZO['anio_lleno'] + 1 - SIM['anio_inicio'] + 1}) a ${POZO['valor
```

	0	1	2	3	4	5	6	7	8	9	10
Δ fierro	0.0	35.1	57.1	58.5	55.6	66.2	68.1	55.3	49.6	54.1	66.5
Δ madera (neta)	0.0	-0.2	-2.0	-2.3	-1.9	-2.3	-2.4	-1.9	-2.4	-1.8	-2.3
Δ aluminio, cartón, plástico	0.0	17.1	42.5	40.7	38.8	34.4	43.4	46.2	40.1	37.0	40.7
Δ áridos (neto)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Δ OPEX	0.0	-46.7	-85.5	-84.9	-85.5	-85.7	-86.4	-86.0	-85.3	-84.7	-85.6
Valor espacio de pozo	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	229.2	34.7
Seguros	0.0	-4.1	-4.1	-4.1	-4.1	-4.1	-4.1	-4.1	-4.1	-4.1	-4.1
Permisos y estudios	-23.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
EBITDA	**-23.1**	**1.2**	**8.1**	**8.0**	**2.8**	**8.6**	**18.6**	**9.6**	**-2.1**	**229.7**	**49.9**
Depreciación	-0.0	-31.3	-31.3	-31.3	-31.3	-31.3	-31.3	-31.3	-31.3	-31.3	-31.3
EBIT	-23.1	-30.2	-23.3	-23.3	-28.5	-22.7	-12.7	-21.8	-33.4	198.3	18.5
Impuesto (–) / ahorro (+)	6.2	8.1	6.3	6.3	7.7	6.1	3.4	5.9	9.0	-53.6	-5.0
Utilidad después de impuesto	-16.9	-22.0	-17.0	-17.0	-20.8	-16.6	-9.3	-15.9	-24.4	144.8	13.5
+ Depreciación	0.0	31.3	31.3	31.3	31.3	31.3	31.3	31.3	31.3	31.3	31.3
Inversión (CAPEX)	-407.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Δ capital de trabajo	-0.0	-2.2	-0.9	-0.0	0.4	-0.5	-0.9	0.8	0.9	-0.1	2.5
Valor residual neto	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	66.1
FCL	**-423.9**	**7.1**	**13.4**	**14.3**	**11.0**	**14.2**	**21.2**	**16.3**	**7.9**	**176.0**	**113.4**

```
VAN al 9.25% real: $-227 MM | TIR: -0.87%
Pozo: lleno en 2034 → el espacio ahorrado se cobra desde 2035 (año 9) a $4,200/m3.
```

8.4 Indicadores de todas las comparaciones (primero vs caso base, después vs CBO)

VAN a la tasa base, **con** el valor del espacio de pozo ($4.200 por m³ suelto, lo que REGEMAC cobra a externos, cobrado desde 2035) y **sin** él. La última columna es el **break-even**: cuánto tendría que valer cada m³ de pozo ahorrado (en el modo de vida útil) para que el VAN sea 0.

In [36]:
```
filas = []
for nom, ref in COMPS:
    F0, F1, Fb = fc_de(nom, ref, 0), fc_de(nom, ref, 1000), fc_de(nom, ref)
    v0 = van(F0, TASA); pend = (van(F1, TASA) - v0) / 1000
    filas.append({'comparación': f'{nom} vs {ref}', 'CAPEX ($MM)': -Fb.loc['Inversión (CAPEX)', 0] / 1e6,
                  f'VAN {TASA:.1%} con pozo ($MM)': van(Fb, TASA) / 1e6, f'VAN {TASA:.1%} sin pozo ($MM)': v0 / 1e6, 'TIR (con pozo)': tir(Fb),
                  'payback descontado (años)': payback_desc(Fb, TASA), 'break-even valor m3 ($/m3)': max(-v0 / pend, 0) if pend > 0 else np.nan})
ind_fin = pd.DataFrame(filas).set_index('comparación')
display(ind_fin.style.format({'CAPEX ($MM)': '{:,.0f}', f'VAN {TASA:.1%} con pozo ($MM)': '{:,.0f}', f'VAN {TASA:.1%} sin pozo ($MM)': '{:,.0f}', 'TIR (con pozo)': '{:.1%}', 'pa
print('CAPEX por ítem ($MM):'); display(pd.DataFrame({k: {i: v / 1e6 for i, v in capex(sc).items()} for k, sc in SC_DE.items() if k != 'Caso base'}).fillna(0).style.format('{:,.
van_tasas = pd.DataFrame({f'{r:.0%}': [van(fc_de(n, rf), r) / 1e6 for n, rf in COMPS] for r in FIN['tasas_rango']}, index=[f'{n} vs {rf}' for n, rf in COMPS])
print(f"VAN ($MM) para el rango de tasas reales, con pozo a ${POZO['valor_m3']:,.0f}/m3:"); display(van_tasas.style.format('{:,.0f}'))
van_m3 = pd.DataFrame({f'${v:,.0f}/m3': [van(fc_de(n, rf, v), TASA) / 1e6 for n, rf in COMPS] for v in FIN['valores_m3']}, index=van_tasas.index)
print(f'VAN ($MM) al {TASA:.1%} según el valor del m3 de pozo ahorrado:'); display(van_m3.style.format('{:,.0f}'))
```

comparación	CAPEX ($MM)	VAN 9.3% con pozo ($MM)	VAN 9.3% sin pozo ($MM)	TIR (con pozo)	payback descontado (años)	break-even valor m3 ($/m3)
CBO vs Caso base	73	-10	-13	6.6%	—	18,066
Planta Nivel 1 vs Caso base	407	-227	-313	-0.9%	—	15,298
Planta FT620C + PS123 vs Caso base	1,015	-1,097	-1,185	-15.2%	—	56,432
Planta KS514 vs Caso base	1,550	-1,254	-1,369	-9.1%	—	49,760
Planta Nivel 1 vs CBO	334	-216	-299	-2.3%	—	15,157
Planta FT620C + PS123 vs CBO	942	-1,086	-1,172	-17.2%	—	57,721
Planta KS514 vs CBO	1,477	-1,243	-1,356	-10.1%	—	50,561

CAPEX por ítem ($MM):

	CBO	Planta Nivel 1	Planta FT620C + PS123	Planta KS514
Electroimán	17.7	17.7	17.7	17.7
Losa acopio astilla	24.7	24.7	24.7	24.7
Chipeadora	31.0	31.0	31.0	31.0
Edificio y estructura (Pinilla 2024)	0.0	78.2	0.0	0.0
Tolva, grizzly y alimentador vibratorio	0.0	29.4	0.0	0.0
Cinta de alimentación inclinada (~12 m)	0.0	21.6	0.0	0.0
Cinta de triaje 1,2 m (~15 m)	0.0	34.3	0.0	0.0
Imán sobrebanda autolimpiante	0.0	19.6	0.0	0.0
Motores, tablero, montaje	0.0	10.0	0.0	0.0
Repuestos iniciales	0.0	2.3	20.5	34.2
Empalme eléctrico	0.0	23.3	23.3	23.3
Control de polvo	0.0	15.0	15.0	15.0
Cargador frontal	0.0	100.0	100.0	100.0
Planta (cotización)	0.0	0.0	683.7	1,139.1
Internación (agente, puerto)	0.0	0.0	17.1	28.5
Obras civiles, grúa, tablero	0.0	0.0	82.0	136.7

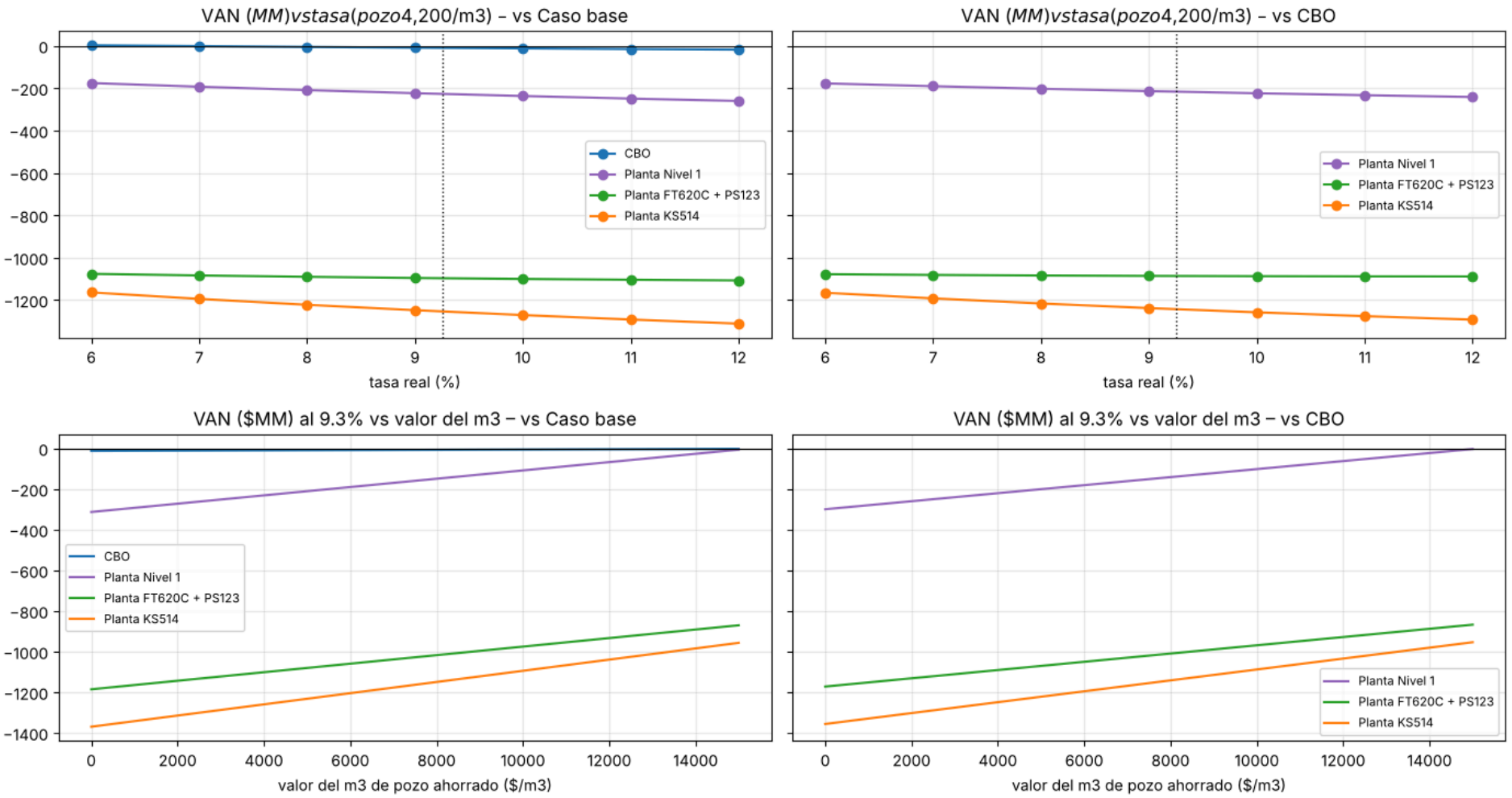
VAN ($MM) para el rango de tasas reales, con pozo a $4,200/m3:

	6%	7%	8%	9%	10%	11%	12%
CBO vs Caso base	3	-2	-5	-9	-12	-15	-18
Planta Nivel 1 vs Caso base	-176	-193	-209	-224	-237	-249	-260
Planta FT620C + PS123 vs Caso base	-1,076	-1,084	-1,090	-1,096	-1,100	-1,104	-1,108
Planta KS514 vs Caso base	-1,164	-1,195	-1,222	-1,248	-1,271	-1,292	-1,311
Planta Nivel 1 vs CBO	-177	-191	-203	-214	-224	-233	-242
Planta FT620C + PS123 vs CBO	-1,078	-1,082	-1,084	-1,086	-1,087	-1,088	-1,089
Planta KS514 vs CBO	-1,166	-1,192	-1,216	-1,238	-1,258	-1,276	-1,293

VAN ($MM) al 9.3% según el valor del m3 de pozo ahorrado:

	$0/m3	$1,740/m3	$3,750/m3	$4,200/m3	$8,000/m3	$15,000/m3
CBO vs Caso base	-13	-11	-10	-10	-7	-2
Planta Nivel 1 vs Caso base	-313	-277	-236	-227	-149	-6
Planta FT620C + PS123 vs Caso base	-1,185	-1,149	-1,107	-1,097	-1,017	-870
Planta KS514 vs Caso base	-1,369	-1,322	-1,266	-1,254	-1,149	-957
Planta Nivel 1 vs CBO	-299	-265	-225	-216	-141	-3
Planta FT620C + PS123 vs CBO	-1,172	-1,136	-1,096	-1,086	-1,009	-867
Planta KS514 vs CBO	-1,356	-1,309	-1,255	-1,243	-1,141	-954

In [37]:
```
fig, ax = plt.subplots(2, 2, figsize=(14, 7.5), sharey='row')
for col, sufijo in enumerate(['Caso base', 'CBO']):
    for nom, ref in COMPS:
        if ref != sufijo: continue
        ax[0, col].plot([r * 100 for r in FIN['tasas_rango']], van_tasas.loc[f'{nom} vs {ref}'], marker='o', color=COLOR[nom], label=nom)
        vs = np.linspace(0, max(FIN['valores_m3']), 31)
        ax[1, col].plot(vs, [van(fc_de(nom, ref, v), TASA) / 1e6 for v in vs], color=COLOR[nom], label=nom)
        be = ind_fin.loc[f'{nom} vs {ref}', 'break-even valor m3 ($/m3)']
        if be == be and be <= vs.max(): ax[1, col].plot(be, 0, 'o', color=COLOR[nom], mfc='white'); ax[1, col].annotate(f'${be:,.0f}', (be, 0), xytext=(4, 6), textcoords='offset
    ax[0, col].axvline(TASA * 100, color='k', ls=':', lw=1); ax[0, col].set_xlabel('tasa real (%)'); ax[0, col].set_title(f"VAN ($MM) vs tasa (pozo ${POZO['valor_m3']:,.0f}/m3)
    ax[1, col].set_xlabel('valor del m3 de pozo ahorrado ($/m3)'); ax[1, col].set_title(f'VAN ($MM) al {TASA:.1%} vs valor del m3 — vs {sufijo}')
    for a_ in ax[:, col]: a_.axhline(0, color='k', lw=.8); a_.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

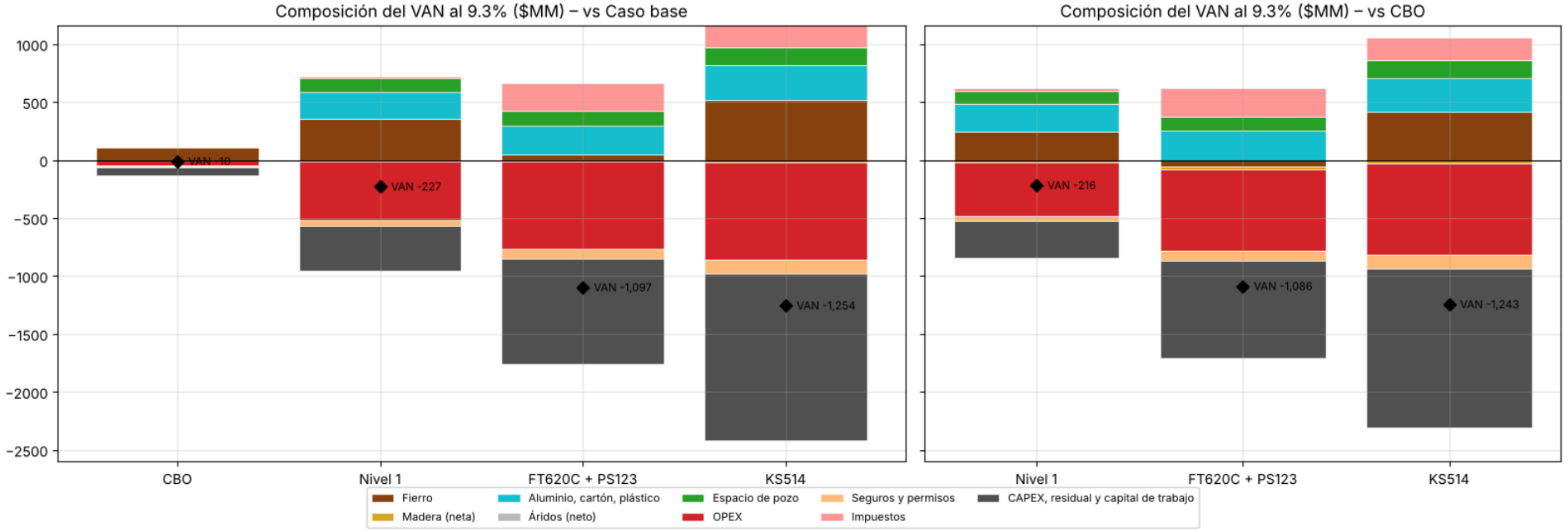


8.4b ¿De dónde sale el VAN? Fierro, madera, pozo, costos e inversión

Cada barra es el valor presente (a la tasa base) de un componente del flujo de caja; la suma es el VAN (rombo negro). Así se ve cuánto aporta el **fierro** y cuánto la **madera** en valor presente, y contra qué costos compiten.

In [38]:
```
filas = {}
for nom, ref in COMPS: filas[f'{nom} vs {ref}'] = descomp_van(fc_de(nom, ref), TASA)
van_dec = pd.DataFrame(filas).T / 1e6
fig, ax = plt.subplots(1, 2, figsize=(16, 5.2), sharey=True, gridspec_kw={'width_ratios': [4, 3]})
for a_, suf in zip(ax, ['Caso base', 'CBO']):
    sub = van_dec[[k.endswith(suf) for k in van_dec.index]]
    x = np.arange(len(sub)); pos = np.zeros(len(sub)); neg = np.zeros(len(sub))
    for c in van_dec.columns:
        v = sub[c].values
```

```
        a_.bar(x, np.where(v > 0, v, 0), bottom=pos, color=COLOR_COMP[c], label=c if suf == 'Caso base' else None, edgecolor='white', lw=.5)
        a_.bar(x, np.where(v < 0, v, 0), bottom=neg, color=COLOR_COMP[c], edgecolor='white', lw=.5)
        pos += np.where(v > 0, v, 0); neg += np.where(v < 0, v, 0)
    a_.scatter(x, sub.sum(axis=1), marker='D', color='k', zorder=3, s=40)
    for xi, vv in zip(x, sub.sum(axis=1)): a_.annotate(f'VAN {vv:,.0f}', (xi, vv), xytext=(8, 0), textcoords='offset points', fontsize=8, va='center')
    a_.set_xticks(x); a_.set_xticklabels([k.split(' vs ')[0].replace('Planta ', '') for k in sub.index]); a_.axhline(0, color='k', lw=.8)
    a_.set_title(f'Composición del VAN al {TASA:.1%} ($MM) — vs {suf}')
fig.legend(loc='lower center', ncol=5, fontsize=8, bbox_to_anchor=(0.5, -0.06)); plt.tight_layout(); plt.show()
display(van_dec.style.format('{:,.0f}'))
```



	Fierro	Madera (neta)	Aluminio, cartón, plástico	Áridos (neto)	Espacio de pozo	OPEX	Seguros y permisos	Impuestos	CAPEX, residual y capital de trabajo
CBO vs Caso base	105	10	0	0	4	-47	-5	-8	-68
Planta Nivel 1 vs Caso base	353	-12	235	0	118	-507	-49	17	-381
Planta FT620C + PS123 vs Caso base	47	-15	254	0	121	-748	-88	239	-907
Planta KS514 vs Caso base	520	-19	295	0	158	-839	-121	189	-1,437
Planta Nivel 1 vs CBO	249	-21	235	0	114	-460	-44	24	-313
Planta FT620C + PS123 vs CBO	-57	-25	254	0	117	-701	-83	247	-838
Planta KS514 vs CBO	417	-29	295	0	154	-792	-117	197	-1,369

8.5 VAN en el Monte Carlo: P10 / P50 / P90 y probabilidad de VAN negativo

Cada corrida del §7 trae sus propias llegadas, composición y precios (con riesgo de perder al comprador). Con ellas se arma el FCL y se calcula el VAN a la tasa base, sin valor del espacio de pozo y con el valor base (`POZO['valor_m3']`).

In [39]:
```
REN = {'d_neto_clp': 'd_neto', 'd_opex_clp': 'd_opex', 'd_ing_clp': 'd_ing', 'd_m3disp': 'd_m3',
       'd_fierro_clp': 'd_fierro', 'd_madera_clp': 'd_madera', 'd_otros_clp': 'd_otros', 'd_aridos_clp': 'd_aridos'}
a_dd = lambda grp: grp.sort_values('anio').drop(columns=['d_m3']).rename(columns=REN)
```

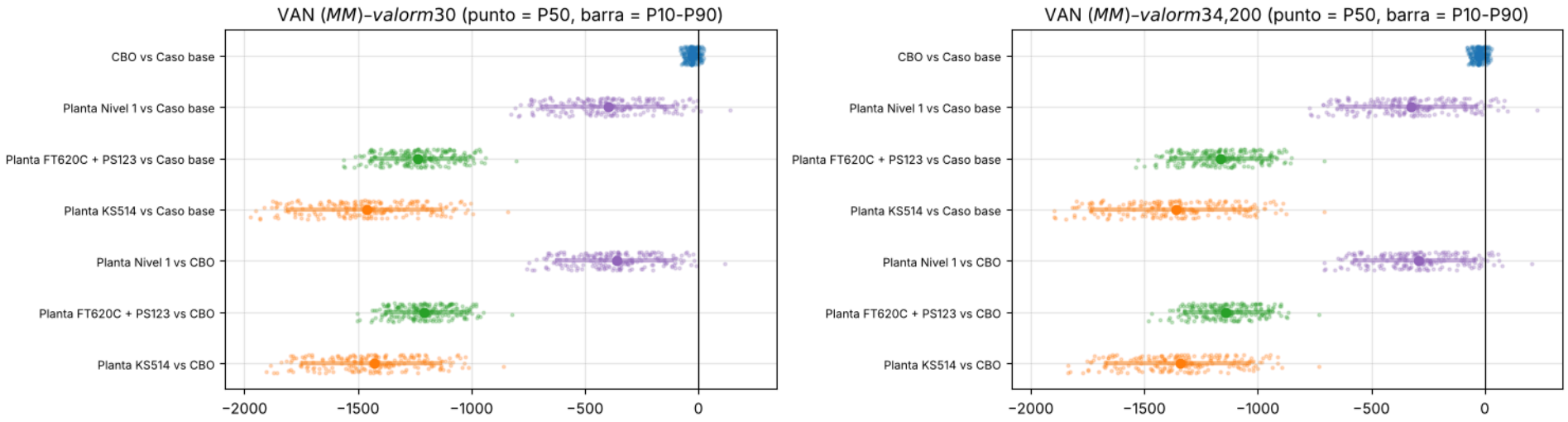
```python
prevs = {c: a_dd(gq) for c, gq in mc[mc.comparacion == 'CBO vs Caso base'].groupby('corrida')}
VM_MC = (0, POZO['valor_m3'])
filas = []
for (comp, corrida), grp in mc.groupby(['comparacion', 'corrida']):   # (no usar 'g': es la tasa de crecimiento)
    nom, ref = comp.split(' vs ')
    dd = a_dd(grp); prev = prevs[corrida] if (ref == 'Caso base' and nom != 'CBO') else None
    for vm in VM_MC:
        filas.append({'comparacion': comp, 'corrida': corrida, 'valor_m3': vm, 'VAN_MM': van(flujo_caja(dd, SC_DE[nom], SC_DE[ref], vm, prev), TASA) / 1e6})
mc_van = pd.DataFrame(filas)
orden_c = [f'{n} vs {r}' for n, r in COMPS]
res_mc = mc_van.groupby(['valor_m3', 'comparacion']).VAN_MM.agg(P10=lambda x: x.quantile(.1), P50='median', P90=lambda x: x.quantile(.9),
                                                                media='mean', P_VAN_neg=lambda x: (x < 0).mean())
for vm in VM_MC:
    print(f'VAN ($MM) al {TASA:.1%} real, valor del m3 = ${vm:,.0f}:')
    display(res_mc.loc[vm].loc[orden_c].style.format({'P10': '{:,.0f}', 'P50': '{:,.0f}', 'P90': '{:,.0f}', 'media': '{:,.0f}', 'P_VAN_neg': '{:.0%}'}))
fig, ax = plt.subplots(1, 2, figsize=(15, 4.2), sharex=True)
for a_, vm in zip(ax, VM_MC):
    for y, comp in enumerate(orden_c[::-1]):
        x = mc_van[(mc_van.valor_m3 == vm) & (mc_van.comparacion == comp)].VAN_MM
        a_.plot([x.quantile(.1), x.quantile(.9)], [y, y], color=col_de(comp), lw=3, alpha=.5)
        a_.plot(x.median(), y, 'o', color=col_de(comp))
        a_.scatter(x, np.full(len(x), y) + np.random.default_rng(0).uniform(-.18, .18, len(x)), s=4, color=col_de(comp), alpha=.25)
    a_.set_yticks(range(len(orden_c))); a_.set_yticklabels(orden_c[::-1], fontsize=8); a_.axvline(0, color='k', lw=.8)
    a_.set_title(f'VAN ($MM) — valor m3 ${vm:,.0f} (punto = P50, barra = P10-P90)')
plt.tight_layout(); plt.show()
```

```
VAN ($MM) al 9.3% real, valor del m3 = $0:
```

comparacion	P10	P50	P90	media	P_VAN_neg
CBO vs Caso base	-61	-27	5	-26	82%
Planta Nivel 1 vs Caso base	-684	-396	-114	-397	99%
Planta FT620C + PS123 vs Caso base	-1,440	-1,235	-1,029	-1,235	100%
Planta KS514 vs Caso base	-1,804	-1,462	-1,137	-1,467	100%
Planta Nivel 1 vs CBO	-621	-360	-115	-370	100%
Planta FT620C + PS123 vs CBO	-1,374	-1,207	-1,033	-1,208	100%
Planta KS514 vs CBO	-1,739	-1,430	-1,141	-1,440	100%

```
VAN ($MM) al 9.3% real, valor del m3 = $4,200:
```

comparacion	P10	P50	P90	media	P_VAN_neg
CBO vs Caso base	-59	-25	9	-24	80%
Planta Nivel 1 vs Caso base	-634	-323	-41	-329	94%
Planta FT620C + PS123 vs Caso base	-1,379	-1,166	-948	-1,166	100%
Planta KS514 vs Caso base	-1,730	-1,361	-1,033	-1,373	100%
Planta Nivel 1 vs CBO	-572	-295	-48	-304	95%
Planta FT620C + PS123 vs CBO	-1,317	-1,143	-956	-1,141	100%
Planta KS514 vs CBO	-1,660	-1,342	-1,039	-1,348	100%



8.6 Tornado: qué mueve el VAN (Planta Nivel 1 y Planta KS514 contra el caso base)

Cada barra cambia un supuesto a su valor bajo y alto, deja el resto igual y recalcula la simulación y el FCL (misma historia, semilla del §6). Los rangos salen del registro v3 (columna Confianza): los supuestos de confianza baja tienen rangos más anchos.

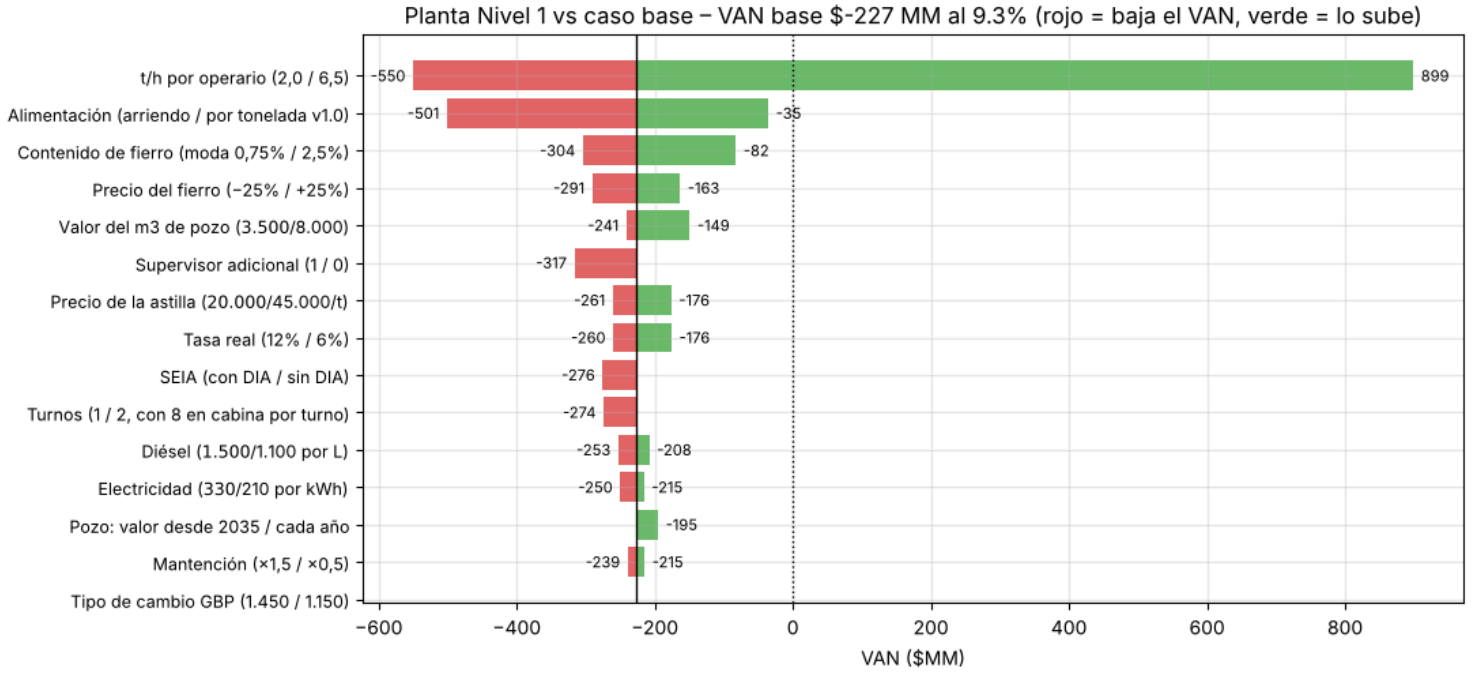
In [40]:
```
def van_caso(nom='Planta Nivel 1', ref='Caso base', sc_mod=None, tasa=None, valor_m3=None, nueva_corrida=False):
    crx = preparar_corrida(SIM['semilla']) if nueva_corrida else cr
    sc_n = dict(SC_DE[nom], **(sc_mod or {}))
    r0 = resumen(caso_base(crx)) if ref == 'Caso base' else resumen(caso_cbo(crx))
    prev = deltas(resumen(caso_cbo(crx)), r0) if ref == 'Caso base' else None
    r_ = resumen(caso_planta(crx, sc_n))
    return van(flujo_caja(deltas(r_, r0), sc_n, SC_DE[ref], valor_m3, prev), TASA if tasa is None else tasa) / 1e6

def con_global(d, k, v, **kw):
    with temporal(d, k, v): return van_caso(**kw)

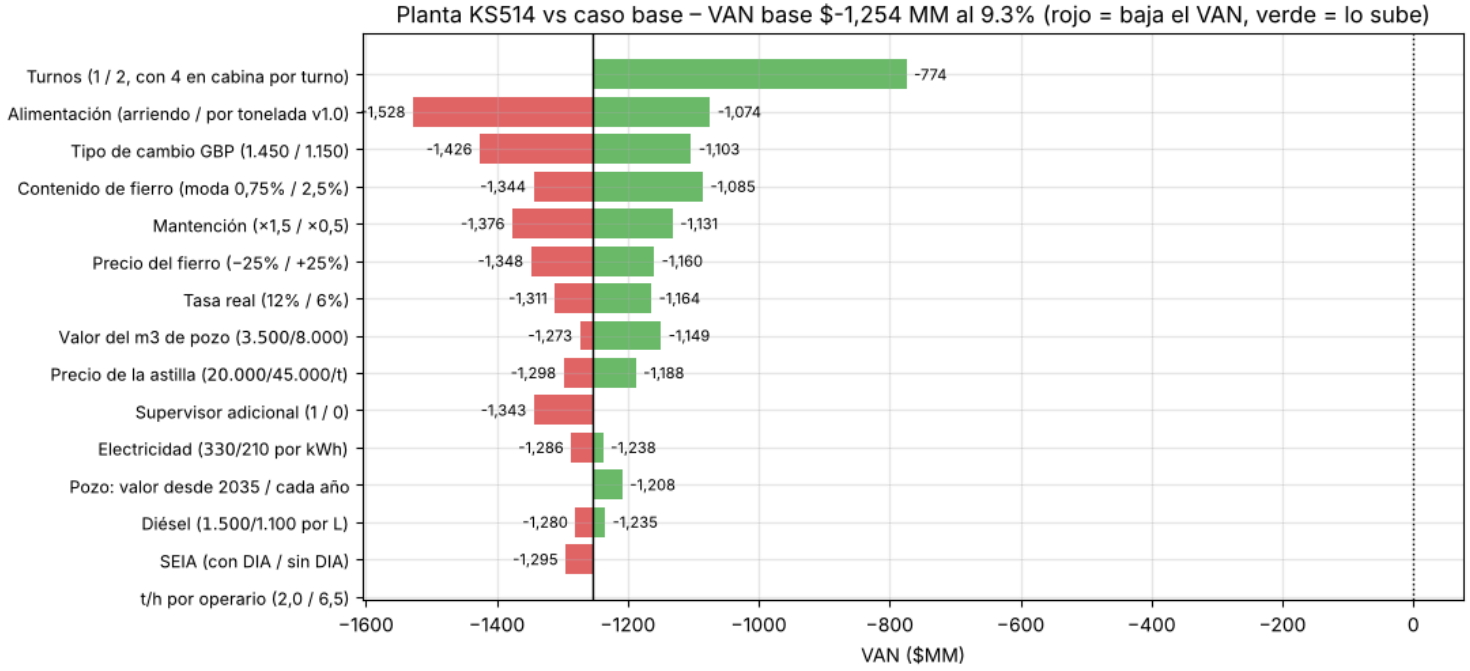
def tornado(nom):
    sc = SC_DE[nom]; v_b = van_caso(nom)
    pf = MERCADO['precio']['fierro']; cf = COMP['contenido']['fierro']; ops2 = sc['operarios_picking'] if sc['fraccion_a_picking'] >= 1 else 4
    it = [
        ('Tasa real (12% / 6%)',                        van_caso(nom, tasa=0.12), van_caso(nom, tasa=0.06)),
        ('Tipo de cambio GBP (1.450 / 1.150)',           con_global(COSTOS, 'tc_gbp', 1450, nom=nom), con_global(COSTOS, 'tc_gbp', 1150, nom=nom)),
        ('Precio del fierro (−25% / +25%)',              con_global(MERCADO['precio'], 'fierro', pf * .75, nom=nom, nueva_corrida=True), con_global(MERCADO['precio'], 'fierro',
        ('Contenido de fierro (moda 0,75% / 2,5%)',      con_global(COMP['contenido'], 'fierro', (cf[0], .0075, cf[2]), nom=nom, nueva_corrida=True), con_global(COMP['contenido
        ('Precio de la astilla ($20.000 / $45.000/t)',   con_global(MADERA, 'precio_astilla', 20_000, nom=nom), con_global(MADERA, 'precio_astilla', 45_000, nom=nom)),
        ('t/h por operario (2,0 / 6,5)',                 van_caso(nom, sc_mod={'tasa_revision_th': 2.0}), van_caso(nom, sc_mod={'tasa_revision_th': 6.5})),
        ('Mantención (×1,5 / ×0,5)',                     van_caso(nom, sc_mod={'mantencion': sc['mantencion'] * 1.5}), van_caso(nom, sc_mod={'mantencion': sc['mantencion'] * .5
        ('Alimentación (arriendo / por tonelada v1.0)',  van_caso(nom, sc_mod={'alimentacion': 'arriendo'}), van_caso(nom, sc_mod={'alimentacion': 'por_tonelada'})),
        ('Valor del m3 de pozo ($3.500 / $8.000)',       van_caso(nom, valor_m3=3_500), van_caso(nom, valor_m3=8_000)),
        ('Pozo: valor desde 2035 / cada año',            v_b, con_global(POZO, 'modo', 'anual', nom=nom)),
        ('Supervisor adicional (1 / 0)',                 van_caso(nom, sc_mod={'supervisores_extra': 1}), v_b),
        ('Electricidad ($330 / $210 por kWh)',           con_global(COSTOS, 'precio_kwh', 330, nom=nom), con_global(COSTOS, 'precio_kwh', 210, nom=nom)),
        ('Diésel ($1.500 / $1.100 por L)',               con_global(COSTOS, 'precio_diesel', 1_500, nom=nom), con_global(COSTOS, 'precio_diesel', 1_100, nom=nom)),
        ('SEIA (con DIA / sin DIA)',                     con_global(FIN['seia'], 'dia', True, nom=nom), v_b),
        (f'Turnos (1 / 2, con {ops2} en cabina por turno)', v_b, van_caso(nom, sc_mod={'horas': T1 + T2, 'turnos': 2, 'operarios_picking': ops2})),
```

```
    ]
    tt = pd.DataFrame(it, columns=['supuesto', 'bajo', 'alto']); tt['rango'] = (tt.alto - tt.bajo).abs()
    return v_b, tt.sort_values('rango')

TORN = {}
for nom in ['Planta Nivel 1', 'Planta KS514']:
    v_b, tt = tornado(nom); TORN[nom] = tt
    fig, ax = plt.subplots(figsize=(11, 5.2))
    for y, r_ in enumerate(tt.itertuples()):
        for v_ in (r_.bajo, r_.alto):
            if abs(v_ - v_b) < 0.5: continue
            ax.barh(y, v_ - v_b, left=v_b, color='#d62728' if v_ < v_b else '#2ca02c', alpha=.7)
            ax.annotate(f'{v_:,.0f}', (v_, y), xytext=(-4 if v_ < v_b else 4, 0), textcoords='offset points', ha='right' if v_ < v_b else 'left', va='center', fontsize=8)
    ax.set_yticks(range(len(tt))); ax.set_yticklabels(tt.supuesto, fontsize=9); ax.axvline(v_b, color='k', lw=1)
    ax.axvline(0, color='k', ls=':', lw=1); ax.set_xlabel('VAN ($MM)')
    ax.set_title(f'{nom} vs caso base — VAN base ${v_b:,.0f} MM al {TASA:.1%} (rojo = baja el VAN, verde = lo sube)')
    plt.tight_layout(); plt.show()
    display(tt.drop(columns='rango').set_index('supuesto').style.format('{:,.0f}'))
```



supuesto	bajo	alto
Tipo de cambio GBP (1.450 / 1.150)	-227	-227
Mantención (×1,5 / ×0,5)	-239	-215
Pozo: valor desde 2035 / cada año	-227	-195
Electricidad ($330 / $210 por kWh)	-250	-215
Diésel ($1.500 / $1.100 por L)	-253	-208
Turnos (1 / 2, con 8 en cabina por turno)	-227	-274
SEIA (con DIA / sin DIA)	-276	-227
Tasa real (12% / 6%)	-260	-176
Precio de la astilla ($20.000 / $45.000/t)	-261	-176
Supervisor adicional (1 / 0)	-317	-227
Valor del m3 de pozo ($3.500 / $8.000)	-241	-149
Precio del fierro (−25% / +25%)	-291	-163
Contenido de fierro (moda 0,75% / 2,5%)	-304	-82
Alimentación (arriendo / por tonelada v1.0)	-501	-35
t/h por operario (2,0 / 6,5)	-550	899



supuesto	bajo	alto
t/h por operario (2,0 / 6,5)	-1,254	-1,254
SEIA (con DIA / sin DIA)	-1,295	-1,254
Diésel ($1.500 / $1.100 por L)	-1,280	-1,235
Pozo: valor desde 2035 / cada año	-1,254	-1,208
Electricidad ($330 / $210 por kWh)	-1,286	-1,238
Supervisor adicional (1 / 0)	-1,343	-1,254
Precio de la astilla ($20.000 / $45.000/t)	-1,298	-1,188
Valor del m3 de pozo ($3.500 / $8.000)	-1,273	-1,149
Tasa real (12% / 6%)	-1,311	-1,164
Precio del fierro (−25% / +25%)	-1,348	-1,160
Mantención (×1,5 / ×0,5)	-1,376	-1,131
Contenido de fierro (moda 0,75% / 2,5%)	-1,344	-1,085
Tipo de cambio GBP (1.450 / 1.150)	-1,426	-1,103
Alimentación (arriendo / por tonelada v1.0)	-1,528	-1,074
Turnos (1 / 2, con 4 en cabina por turno)	-1,254	-774

8.6b 1 turno vs 2 turnos

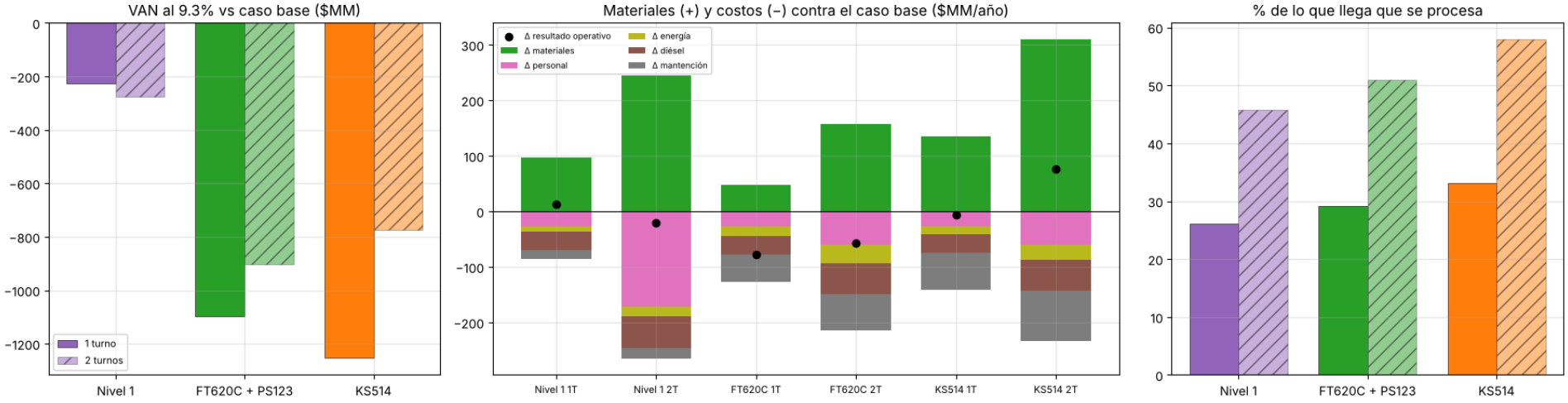
El 2° turno (17-23 h, 6 h efectivas) procesa más de lo que hoy se vacía al pozo sin procesar, pero no todo se duplica:

- **Personal:** cada turno necesita su dotación (cabina + 2 de transferencia), con 10% de recargo en el 2°; se paga la diferencia con los 8 actuales. Nivel 1 mantiene 8 en la cabina por turno (mandan los operarios); las Kiverco usan 4 (manda la máquina).
- **Energía:** 18-22 h es horario punta de abril a septiembre ($320/kWh en vez de $250).
- **Diésel del cargador:** por hora de operación.
- **Mantención:** la mitad crece con las horas.
- **CAPEX:** el mismo.

In [41]:
```
filas = []
for k, sc in PLANTAS.items():
    for tur in (1, 2):
        ops = sc['operarios_picking'] if (tur == 1 or sc['fraccion_a_picking'] >= 1) else 4
        sc2 = dict(sc, horas=T1 + (T2 if tur == 2 else []), turnos=tur, operarios_picking=ops)
        r2 = resumen(caso_planta(cr, sc2)); a, b = r2.mean(), R_['Caso base'].mean()
        dd = deltas(r2, R_['Caso base']); prev = deltas(R_['CBO'], R_['Caso base'])
        Fb = flujo_caja(dd, sc2, BASE, None, prev); F0, F1 = flujo_caja(dd, sc2, BASE, 0, prev), flujo_caja(dd, sc2, BASE, 1000, prev)
        v0 = van(F0, TASA); pend = (van(F1, TASA) - v0) / 1000
        pt = ops + sc['operarios_apoyo']
        filas.append({'planta': sc['nombre'], 'turnos': tur, 'personas por turno': pt, 'Δ personas vs hoy': pt * tur - BASE['trabajadores'],
                      'capacidad (t/h)': capacidad_th(sc2)[0], '% procesado': a.pct_procesado, 'Δ fierro (t/año)': a.fierro_t - b.fierro_t,
                      'Δ m3 de pozo/año': b.vertedero_m3 - a.vertedero_m3,
                      'Δ materiales ($MM/año)': (a.neto_clp - b.neto_clp) / 1e6, 'Δ personal ($MM/año)': (a.opex_personal_clp - b.opex_personal_clp) / 1e6,
                      'Δ energía ($MM/año)': (a.opex_energia_clp - b.opex_energia_clp) / 1e6, 'Δ diésel ($MM/año)': (a.opex_diesel_clp - b.opex_diesel_clp) / 1e6,
                      'Δ mantención ($MM/año)': (a.opex_mantencion_clp - b.opex_mantencion_clp) / 1e6, 'Δ resultado operativo ($MM/año)': (a.resultado_clp - b.resultado_clp) / 1
                      f'VAN {TASA:.1%} con pozo ($MM)': van(Fb, TASA) / 1e6, 'TIR': tir(Fb), 'break-even valor m3 ($/m3)': max(-v0 / pend, 0) if pend > 0 else np.nan})
t_tur = pd.DataFrame(filas)
display(t_tur.set_index(['planta', 'turnos']).style.format('{:,.1f}', na_rep='–').format({'% procesado': '{:.0%}', 'TIR': '{:.1%}', 'personas por turno': '{:.0f}', 'Δ personas v
        'Δ fierro (t/año)': '{:,.0f}', 'Δ m3 de pozo/año': '{:,.0f}', 'break-even valor m3 ($/m3)': '{:,.0f}', f'VAN {TASA:.1%} con pozo ($MM)': '{:,.0f}'}, na_rep='–'))
fig, ax = plt.subplots(1, 3, figsize=(18, 4.8), gridspec_kw={'width_ratios': [1, 1.6, 1]})
pls = list(dict.fromkeys(t_tur.planta)); x = np.arange(len(pls)); w = .38
```

```
for j, tur in enumerate((1, 2)):
    sub = t_tur[t_tur.turnos == tur].set_index('planta').loc[pls]
    ax[0].bar(x + (j - .5) * w, sub[f'VAN {TASA:.1%} con pozo ($MM)'], w, color=[COLOR[p] for p in pls], alpha=1 if tur == 1 else .5,
              hatch=None if tur == 1 else '//', edgecolor='k', lw=.4, label=f'{tur} turno' + ('s' if tur == 2 else ''))
    ax[2].bar(x + (j - .5) * w, sub['% procesado'] * 100, w, color=[COLOR[p] for p in pls], alpha=1 if tur == 1 else .5, hatch=None if tur == 1 else '//', edgecolor='k', lw=.4)
ax[0].set_xticks(x); ax[0].set_xticklabels([p.replace('Planta ', '') for p in pls]); ax[0].axhline(0, color='k', lw=.8)
ax[0].set_title(f'VAN al {TASA:.1%} vs caso base ($MM)'); ax[0].legend(fontsize=8)
ax[2].set_xticks(x); ax[2].set_xticklabels([p.replace('Planta ', '') for p in pls]); ax[2].set_title('% de lo que llega que se procesa')
comps_ = [('Δ materiales ($MM/año)', '#2ca02c'), ('Δ personal ($MM/año)', '#e377c2'), ('Δ energía ($MM/año)', '#bcbd22'), ('Δ diésel ($MM/año)', '#8c564b'), ('Δ mantención ($MM/
xs = np.arange(len(pls) * 2); etiquetas = []
for i, (p, tur) in enumerate([(p, tur) for p in pls for tur in (1, 2)]):
    r_ = t_tur[(t_tur.planta == p) & (t_tur.turnos == tur)].iloc[0]; base_ = 0.0; neg_ = 0.0
    for c, col in comps_:
        v = r_[c] if c == 'Δ materiales ($MM/año)' else -r_[c]
        if v >= 0: ax[1].bar(i, v, .7, bottom=base_, color=col, label=c.replace(' ($MM/año)', '') if i == 0 else None); base_ += v
        else: ax[1].bar(i, v, .7, bottom=neg_, color=col, label=c.replace(' ($MM/año)', '') if i == 0 else None); neg_ += v
    ax[1].scatter(i, r_['Δ resultado operativo ($MM/año)'], color='k', zorder=3, label='Δ resultado operativo' if i == 0 else None)
    etiquetas.append(f"{p.replace('Planta ', '').replace(' + PS123', '')} {tur}T")
ax[1].set_xticks(xs); ax[1].set_xticklabels(etiquetas, fontsize=8); ax[1].axhline(0, color='k', lw=.8); ax[1].legend(fontsize=7, ncol=2)
ax[1].set_title('Materiales (+) y costos (−) contra el caso base ($MM/año)')
plt.tight_layout(); plt.show()
ind_2t = t_tur
```

planta	turnos	personas por turno	Δ personas vs hoy	capacidad (t/h)	% procesado	Δ fierro (t/año)	Δ m3 de pozo/año	Δ materiales ($MM/año)	Δ personal ($MM/año)	Δ energía ($MM/año)	Δ diésel ($MM/año)	Δ mantención ($MM/año)	Δ resultado operativo ($MM/año)	VAN 9.3% con pozo ($MM)	TIR	break-even valor m3 ($/m3)
Planta Nivel 1	1	10	+2	24.000000	26%	313	6,606	97.824963	26.421699	9.960000	33.381606	15.788350	12.273309	-227	-0.9%	15,298
	2	10	+12	24.000000	46%	903	14,172	244.753788	171.741041	17.949120	55.832026	20.215104	-20.983503	-274	-1.3%	10,456
Planta FT620C + PS123	1	10	+2	26.775000	29%	40	6,796	48.309492	26.421699	18.326400	33.421248	48.307272	-78.167127	-1,097	-15.2%	56,432
	2	6	+4	26.775000	51%	426	14,853	158.100089	60.769907	33.026381	55.861000	64.928622	-56.485821	-903	-7.9%	23,855
Planta KS514	1	10	+2	30.375000	33%	462	8,900	135.410607	26.421699	14.740800	33.304806	67.444917	-6.501614	-1,254	-9.1%	49,760
	2	6	+4	30.375000	58%	1,165	18,975	310.555890	60.769907	26.564698	55.606101	91.242884	76.372301	-774	-0.4%	17,385



8.6c Interruptor de áridos: VAN de cada planta con y sin áridos

Mismo cálculo del §8.4 (contra el caso base, con el valor del espacio de pozo), con `ARIDOS['activo']` apagado y prendido en sus dos modos.

In [42]:
```
filas = []
for sc in PLANTAS.values():
    nom = sc['nombre']; fila = {'planta': nom, 'sin áridos': van(fc_de(nom, 'Caso base'), TASA) / 1e6}
    for modo in ('arriendo', 'compra'):
        with temporal(ARIDOS, 'activo', True), temporal(ARIDOS, 'modo', modo):
            r_ = resumen(caso_planta(cr, sc))
            fila[f'con áridos ({modo})'] = van(flujo_caja(deltas(r_, R_['Caso base']), sc, BASE, None, prev_de(nom, 'Caso base')), TASA) / 1e6
    filas.append(fila)
van_ar = pd.DataFrame(filas).set_index('planta')
van_ar['Δ por áridos (arriendo)'] = van_ar['con áridos (arriendo)'] - van_ar['sin áridos']
van_ar['Δ por áridos (compra)'] = van_ar['con áridos (compra)'] - van_ar['sin áridos']
print(f'VAN ($MM) al {TASA:.1%} vs caso base:'); display(van_ar.style.format('{:,.0f}'))
```

```
VAN ($MM) al 9.3% vs caso base:
```

planta	sin áridos	con áridos (arriendo)	con áridos (compra)	Δ por áridos (arriendo)	Δ por áridos (compra)
Planta Nivel 1	-227	-322	-560	-95	-333
Planta FT620C + PS123	-1,097	-1,192	-1,430	-95	-333
Planta KS514	-1,254	-1,349	-1,587	-95	-333

8.7 Lectura de la evaluación económica (v1.2, semilla 42; cambia con los supuestos)

- **Tasa.** WACC real de 9,25%. Sin el premio de 2 pp sería 7,6%.
- **VAN al 9,25% con el pozo a $4.200 por m³ suelto (cobrado desde 2035):**
  - CBO −$10 MM (TIR 6,6%): marginal. Con la madera de hoy la chipeadora le resta: sin ella (solo el imán, y la madera sigue a RECUPAC) el CBO se ahorra $56 MM de chipeadora y losa y mejora.
  - **Nivel 1 −$227 MM** (TIR −0,9%); FT620C+PS123 −$1.097 MM; KS514 −$1.254 MM.
  - Monte Carlo (200 corridas): Nivel 1 P10/P50/P90 = −$634 / −$323 / −$41 MM, P(VAN<0) = 94%. Kiverco: 100%.
- **Composición del VAN, Nivel 1 vs caso base (§8.4b):** fierro +$353 MM; aluminio, cartón y plástico +$235 MM; espacio de pozo +$118 MM; madera −$12 MM; OPEX −$507 MM; CAPEX neto de residual −$381 MM; seguros y permisos −$49 MM; impuestos +$17 MM.
- **Pozo.** Con $4.200/m³ cobrados desde 2035, el VAN de Nivel 1 sube $86 MM respecto de no valorar el pozo. Si el espacio se pudiera vender cada año, subiría otros ~$30 MM. El VAN llega a 0 recién con ~$15.300/m³.
- **1 vs 2 turnos (§8.6b):**
  - **Nivel 1 empeora con 2 turnos** (−$274 MM): para procesar 46% necesita 12 personas más que hoy (mandan los operarios), y el personal sube $172 MM/año.
  - **Las Kiverco mejoran:** KS514 pasa a −$774 MM (procesa 58%, resultado +$76 MM/año) y FT620C a −$903 MM. Solo agregan 4 personas (4 en la cabina por turno), porque manda la máquina.
  - La energía del 2° turno cuesta más (punta) y la mantención crece, pero lo que decide es el personal.
- **Tornado Nivel 1:** lo que más mueve el VAN es la **productividad por operario** (2,0 t/h → −$550 MM; 6,5 t/h, la cifra del PFD → **+$899 MM**). Le siguen la forma de alimentar (arriendo −$501 / por tonelada −$35 MM), el contenido de fierro (−$304 / −$82 MM), el precio del fierro y un supervisor adicional (−$90 MM). **KS514:** turnos, alimentación, tipo de cambio y mantención.
- **Conclusión para el informe:**
  - Con los supuestos actuales ninguna planta tiene VAN positivo; Nivel 1 es la más cercana y la única con resultado operativo positivo con 1 turno.
  - La decisión depende sobre todo de cuántas t/h puede revisar un operario sobre RCD sin harnear. Hay que medirlo con una prueba en la fosa.
  - El cargador debería compartir el trabajo de vaciado para no ser un costo dedicado.
  - La chipeadora solo se justifica con una planta que recupere más de ~730 t/año de madera apta.

9. Exportar

In [43]:
```
with pd.ExcelWriter('modelo_regemac_resultados.xlsx') as w:
    pd.Series({**{f'SIM.{k}': v for k, v in SIM.items()}, **{f'LLEG.{k}': v for k, v in LLEG.items()}, **{f'PLANTA.{p}.{k}': str(v) for p, sc in PLANTAS.items() for k, v in sc.i
```

```
            **{f'BASE.{k}': str(v) for k, v in BASE.items()}, **{f'CBO.{k}': str(v) for k, v in CBO.items()}, **{f'MERCADO.{k}': str(v) for k, v in MERCADO.items()},
            **{f'COSTOS.{k}': v for k, v in COSTOS.items()}, **{f'MOVIMIENTO.{k}': v for k, v in MOVIMIENTO.items()}, **{f'MADERA.{k}': str(v) for k, v in MADERA.items()},
            'COMP.contenido': str(COMP['contenido']), 'PERMITIDO_VERTEDERO': str(PERMITIDO_VERTEDERO), 'DENS': str(DENS)}).to_frame('valor').to_excel(w, sheet_name='parametro
    for k, o in ESC.items(): resumen(o).to_excel(w, sheet_name=k[:28])
    for k, d_ in {**inc_base, **inc_cbo}.items(): d_.to_excel(w, sheet_name=('inc ' + k)[:31])
    pd.concat([ind_base, ind_cbo]).to_excel(w, sheet_name='indicadores')
    t_dens.to_excel(w, sheet_name='sens_densidad', index=False)
    tm.to_excel(w, sheet_name='opciones_madera', index=False)
    q.to_excel(w, sheet_name='montecarlo_incremental')
    ind_fin.to_excel(w, sheet_name='VAN_indicadores'); van_tasas.to_excel(w, sheet_name='VAN_tasas'); van_m3.to_excel(w, sheet_name='VAN_valor_m3')
    res_mc.to_excel(w, sheet_name='VAN_montecarlo'); (F_ej / 1e6).to_excel(w, sheet_name='FCL_Nivel1_vs_base')
    ben_fm.to_excel(w, sheet_name='beneficio_fierro_madera'); van_dec.to_excel(w, sheet_name='VAN_por_componente')
    t_ar.to_excel(w, sheet_name='aridos_operacion', index=False); van_ar.to_excel(w, sheet_name='aridos_VAN'); ind_2t.to_excel(w, sheet_name='VAN_2_turnos')
    for nom, tt in TORN.items(): tt.to_excel(w, sheet_name=('tornado ' + nom.replace('Planta ', ''))[:31], index=False)
    t_chip.to_excel(w, sheet_name='chipeadora'); t_tur.to_excel(w, sheet_name='1_vs_2_turnos', index=False)
    pd.Series({**{f'ALIMENTACION.{k}': str(v) for k, v in ALIMENTACION.items()}, **{f'ARIDOS.{k}': str(v) for k, v in ARIDOS.items()},
            **{f'EXTRA_PLANTA.{k}': v for k, v in EXTRA_PLANTA.items()}, 'GRANULOMETRIA': str(GRANULOMETRIA),
            **{f'CAPTURA.{sc["nombre"]}': str({m: round(v, 3) for m, v in captura_de(sc).items()}) for sc in PLANTAS.values()}}).to_frame('valor').to_excel(w, sheet_name='par
    for n, rf in COMPS: (fc_de(n, rf) / 1e6).to_excel(w, sheet_name=('FCL ' + n.replace('Planta ', '') + ' v ' + rf)[:31])
    pd.Series({f'FIN.{k}': str(v) for k, v in FIN.items()}).to_frame('valor').to_excel(w, sheet_name='parametros_fin')
    pd.Series({**{f'POZO.{k}': str(v) for k, v in POZO.items()}}).to_frame('valor').to_excel(w, sheet_name='pozo')
    estac.to_frame('factor').to_excel(w, sheet_name='estacionalidad')
    pd.DataFrame({'lun_vie': perfil_lv, 'sabado': perfil_sab}).to_excel(w, sheet_name='perfil_horario')
    tabla_kgv.to_excel(w, sheet_name='kg_por_viaje_hist')
print('Guardado: modelo_regemac_resultados.xlsx')
```

```
Guardado: modelo_regemac_resultados.xlsx
```

Próximos pasos

Datos que más mueven el resultado (ver tornados del §8.6) y a quién pedirlos:

1. **Costo de la máquina de nivel 1** (tolva, cintas, imán, montaje): hoy son rangos web de confianza baja. Rodrigo / fabricantes locales.
2. **Productividad real de un operario sobre RCD sin harnear** (t/h revisadas) y captura de madera: muestreo o prueba en la fosa.
3. **Equipo de alimentación**: precio de un cargador o arriendo con operador (Rodrigo, arrendadoras).
4. **Precio de la astilla y unidad** (Arauco) y **precio del fierro** (Ecociclo paga sobre el mercado de oct-2026).
5. **Cotización vigente de SKC** y versión eléctrica de la FT620C (reunión con Osvaldo Pardo).
6. **SEIA:** consulta de pertinencia (¿la segregación mecánica es "tratamiento"?).
7. **Áridos:** precio local del árido reciclado (Río Claro), arriendo de chancador (Raico/Comeco) y m³ de material que REGEMAC compra para caminos.

Para el Excel con Pedro: las hojas `parametros_v11`, `beneficio_fierro_madera`, `VAN_por_componente` y los FCL del archivo `modelo_regemac_resultados.xlsx` traen los flujos anuales que el Excel puede reproducir. El registro de supuestos con fuentes y confianza es `Registro_modelo_REGEMAC_v3.xlsx`.